

Laboratory of Computational Physics

Module B

Norah Jones

2026-10-06

Table of contents

Preface	4
1 Introduction	5
I Part I: Lessons	6
2 Lesson 1 — Gradient descent	7
2.1 4. Gradient descent and its generalizations	7
2.2 Learning Goal	8
2.3 Overview	8
2.4 Surfaces	8
2.5 4.1 Gradient descent and Newton's method	13
Algorithm: gradient descent in pseudocode	14
2.6 Gradient descent with and without momentum	14
2.7 Experiments with GD, CM, and NAG	16
Algorithm: Newton's method in pseudocode	21
2.8 Gradient Descents that utilize the second moment	22
2.9 Experiments with ADAM and RMSprop	24
3 Lesson 2 — Clustering (DBSCAN)	27
3.1 13. Clustering	27
3.2 13.1 Practical clustering methods	31
3.3 2. Maximization: Given a set of cluster means , find the assignments n_k which minimizes C . Clearly, this is	32
Algorithm: K-means in pseudocode	32
Algorithm: DBSCAN in pseudocode	33
3.4 Find the typical minimum distance between points	34
3.5 t-SNE visualization, for true labels	35
3.6 Different random initial states (random_state=i)	35
3.7 DBSCAN: find labels by applying clustering	36
4 Lesson 3 — Deep neural networks	39
4.1 9. An introduction to feed-forward deep neural networks(DN Ns)	39
4.1.1 9.1 Neural network basics	41
Algorithm: forward + backward pass in pseudocode	43
5 Keras Tuner	45
5.1 Read data	45
5.2 Split data and standardize (fit on train only)	47
5.3 Define the model with Keras / Tensorflow	48
5.4 Optimization method and cost function	49
5.5 Training	49

6	Evaluation on the test set	51
6.1	Grid to show predictions	51
7	Lesson 4 — Convolutional neural networks on time series	53
7.1	10. Convolutional Neural Networks(CNN S)	53
7.1.1	10.1 The structure of convolutional neural networks	54
7.1.2	Function for the added pattern	56
7.1.3	Save data on file	57
7.2	10.2 Example:CNN s for the2DI sing model	57
	Algorithm: a single conv → activation → pool block	58
7.2.1	Read data	59
7.2.2	Split train-validation and rescale	59
7.3	Convolutional NN	60
8	Lesson 5 — Gradient boosting (XGBoost)	66
8.0.1	8.2 Bagging	68
	Algorithm: gradient boosting in pseudocode	69
8.0.1	Scatter plot on [i,j] feature plane	70
8.0.2	Classify	72
8.1	XGBoost	72
8.2	Gradient Boosting from Scikit-learn	74
8.3	AdaBoost	74
II	Part II: Labs	75
9	Exercise 2	76
9.1	1. Generate data	76
9.2	2. Study of the amplitude	79
10	3	86
11	LCPB 25-26 Exercise 3, XGBoost (XGB)	91
11.1	Importing the libraries	91
11.2	Generating and loading the data	92
11.3	Exercise 1: Grid search for different parameters (Model complexity)	93
11.4	Select different features	100
11.5	compare it to FFNN with cross validation	105
12	Grid search for different parameters	109
12.1	Select different features	117
12.2	compare it to FFNN with cross validation	122
14	Summary	132
	References	133

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Introduction

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

Part I

Part I: Lessons

2 Lesson 1 — Gradient descent

The theory in this chapter is extracted from Mehta *et al.* — *A high-bias, low-variance introduction to Machine Learning for physicists* (Physics Reports 810, 2019), §4. The accompanying notebook is adapted from the supplementary material to the same review. The full review lives under `2sem/LCPB/source/` in the repository.

Theory → Motivation and the regimes of the learning rate.

2.1 4. Gradient descent and its generalizations

Almost every problem in ML and data science starts with the same ingredients: a data set X , a model $g(\cdot)$, which is a function of the parameters, and a cost function $C(X, g(\cdot))$ that allows us to judge how well the model $g(\cdot)$ explains the observations X . The model is fit by finding the values of that minimize the cost function. In this section, we discuss one of the most powerful and widely used classes of methods for performing this minimization — gradient descent and its generalizations. The basic idea behind these methods is straightforward: iteratively adjust the parameters in the direction where the gradient of the cost function is large and negative. In this way, the training procedure ensures the parameters flow towards a local minimum of the cost function. However, in practice gradient descent is full of surprises and a series of ingenious tricks have been developed by the optimization and machine learning communities to improve the performance of these algorithms. The underlying reason why training a machine learning algorithm is difficult is that the cost functions we wish to optimize are usually complicated, rugged, non-convex functions in a high-dimensional space with many local minima. To make things even more difficult, we almost never have access to the true function we wish to minimize: instead, we must estimate this function directly from data. In modern applications, both the size of the data set and the number of

Fig. 7

Gradient descent exhibits three qualitatively different regimes as a function of the learning rate. Result of gradient descent on surface $z = x^2 + y^2 - 1$ for learning rate of $\eta = 0.1, 0.5, 1.01$. Notice that the trajectory converges to the global minima in multiple steps for small learning rates ($\eta = 0.1$). Increasing the learning rate further ($\eta = 0.5$) causes the trajectory to oscillate around the global minima before converging. For even larger learning rates ($\eta = 1.01$) the trajectory diverges from the minima. See corresponding notebook for details. parameters we wish to fit is often enormous (millions of parameters and examples). The goal of this chapter is to explain how gradient descent methods can be used to train machine learning algorithms even in these difficult settings. This chapter seeks to both introduce commonly used methods and give intuition for why they work. We also include some practical tips for improving the

performance of stochastic gradient descent (Bottou, 2012; LeCun et al., 1998b). To help the reader gain more intuition about gradient descent and its variants, we have developed a Jupyter notebook that allows the reader to visualize how these algorithms perform on two dimensional surfaces. The reader is encouraged to experiment with the accompanying notebook whenever a new method is introduced (especially to explore how changing hyper-parameters can affect performance). The reader may also wish to consult useful reviews that cover these topics (Ruder, 2016) and this blog <http://ruder.io/optimizing-gradient-descent/>.

Practice → The test surfaces and visualisation helpers we will use throughout.

[from Mehta's review, Notebook 2: Gradient Descent]

2.2 Learning Goal

The goal of this notebook is to gain intuition for various gradient descent methods by visualizing and applying these methods to some simple two-dimensional surfaces. Methods studied include ordinary gradient descent, gradient descent with momentum, NAG, ADAM, and RMSProp.

2.3 Overview

In this notebook, we will visualize what different gradient descent methods are doing using some simple surfaces. From the onset, we emphasize that doing gradient descent on the surfaces is different from performing gradient descent on a loss function in Machine Learning (ML). The reason is that in ML not only do we want to find good minima, we want to find good minima that generalize well to new data. Despite this crucial difference, we can still build intuition about gradient descent methods by applying them to simple surfaces (see related blog posts [here](#) and [here](#)).

2.4 Surfaces

We will consider three simple surfaces: a quadratic minimum of the form

$$z = ax^2 + by^2,$$

a saddle-point of the form

$$z = ax^2 - by^2,$$

and **Beale's Function**, a convex function often used to test optimization problems of the form:

$$z(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

These surfaces can be plotted using the cells below.

```

# plotting functions to visualize gradient descent routines

#Make plots interactive
#%matplotlib notebook

#Make plots static
%matplotlib inline

#Make 3D plots
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from matplotlib import cm
#from matplotlib import animation
from IPython.display import HTML
from matplotlib.colors import LogNorm
#from itertools import zip_longest

# Python
import matplotlib as mpl
import matplotlib.font_manager as fm

# Use DejaVu for text and math
mpl.rcParams['font.family'] = 'DejaVu Sans'
mpl.rcParams['mathtext.fontset'] = 'dejavusans'

# Rebuild matplotlib's font cache (safe)

#Import Numpy
import numpy as np

#Define function for plotting

def plot_surface(x, y, z, azimuth=-20, elev=40, dist=10, cmap="RdYlBu_r"):

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    plot_args = {'rstride': 1, 'cstride': 1, 'cmap':cmap,
                  'linewidth': 20, 'antialiased': True,
                  'vmin': -2, 'vmax': 2}
    ax.plot_surface(x, y, z, **plot_args)
    ax.view_init(azimuth=azimuth, elev=elev)
    ax.dist=dist
    ax.set_xlim(-1, 1)
    ax.set_ylim(-1, 1)

```

```

ax.set_zlim(-2, 2)

plt.xticks([-1, -0.5, 0, 0.5, 1], ["-1", "-1/2", "0", "1/2", "1"])
plt.yticks([-1, -0.5, 0, 0.5, 1], ["-1", "-1/2", "0", "1/2", "1"])
ax.set_zticks([-2, -1, 0, 1, 2])
ax.set_zticklabels(["-2", "-1", "0", "1", "2"])

ax.set_xlabel("x", fontsize=16)
ax.set_ylabel("y", fontsize=16)
ax.set_zlabel("z", fontsize=16)
return fig, ax;

def overlay_trajectory_quiver(ax,obj_func,trajectory, color='k'):
    # Overlay the trajectory of the optimization in gradient descent steps
    ↪ using quiver arrows
    xs=trajectory[:,0]
    ys=trajectory[:,1]
    zs=obj_func(xs,ys)
    # arrow starting points, \delta(x), \delta(y)
    ax.quiver(xs[:-1], ys[:-1], zs[:-1], xs[1:]-xs[:-1],
    ↪ ys[1:]-ys[:-1],zs[1:]-zs[:-1],color=color,arrow_length_ratio=0.3)

    return ax;

def overlay_trajectory(ax,obj_func,trajectory,label,color='k'):
    # Overlay the trajectory of the optimization process with a line
    xs=trajectory[:,0]
    ys=trajectory[:,1]
    zs=obj_func(xs,ys)
    ax.plot(xs,ys,zs, color, label=label)

    return ax;

def overlay_trajectory_contour_M(ax,trajectory, label,color='k',lw=2):
    # Overlay the trajectory of the optimization process on a contour plot
    ↪ with a marker at the end
    xs=trajectory[:,0]
    ys=trajectory[:,1]
    ax.plot(xs,ys, color, label=label,lw=lw)
    ax.plot(xs[-1],ys[-1],color+'>', markersize=14)
    return ax;

def overlay_trajectory_contour(ax,trajectory, label,color='k',lw=2):

```

```

# Overlay the trajectory of the optimization process on a contour plot
↳ without a marker at the end
xs=trajectory[:,0]
ys=trajectory[:,1]
ax.plot(xs,ys, color, label=label,lw=lw)
return ax;

```

#DEFINE SURFACES WE WILL WORK WITH

#Define monkey saddle and gradient

```

def monkey_saddle(x,y):
    return x**3 - 3*x*y**2

```

```

def grad_monkey_saddle(params):
    x=params[0]
    y=params[1]
    grad_x= 3*x**2-3*y**2
    grad_y= -6*x*y
    return [grad_x,grad_y]

```

#Define saddle surface

```

def saddle_surface(x,y,a=1,b=1):
    return a*x**2-b*y**2

```

```

def grad_saddle_surface(params,a=1,b=1):
    x=params[0]
    y=params[1]
    grad_x= 2*a*x
    grad_y= -2*b*y
    return [grad_x,grad_y]

```

Define minima_surface

```

def minima_surface(x,y,a=1,b=1):
    return a*x**2+b*y**2-1

```

```

def grad_minima_surface(params,a=1,b=1):
    x=params[0]
    y=params[1]
    grad_x= 2*a*x
    grad_y= 2*b*y

```

```

    return [grad_x,grad_y]

def beales_function(x,y):
    return np.square(1.5-x+x*y)+np.square(2.25-x+x*y*y)+np.square(2.625-x+x
        ↪ *y**3)

def grad_beales_function(params):
    x=params[0]
    y=params[1]
    grad_x=2*(1.5-x+x*y)*(-1+y)+2*(2.25-x+x*y**2)*(-1+y**2)+2*(2.625-x+x*y
        ↪ **3)*(-1+y**3)
    grad_y=2*(1.5-x+x*y)*x+4*(2.25-x+x*y**2)*x*y+6*(2.625-x+x*y**3)*x*y**2
    return [grad_x,grad_y]

def contour_beales_function():
    #plot beales function
    x, y = np.meshgrid(np.arange(-4.5, 4.5, 0.2), np.arange(-4.5, 4.5, 0.2))
    fig, ax = plt.subplots(figsize=(10, 6))
    z=beales_function(x,y)
    cax = ax.contour(x, y, z, levels=np.logspace(0, 5, 35), norm=LogNorm(),
        ↪ cmap="RdYlBu_r")
    ax.plot(3,0.5, 'r*', markersize=18)

    ax.set_xlabel('$x$')
    ax.set_ylabel('$y$')

    ax.set_xlim((-4.5, 4.5))
    ax.set_ylim((-4.5, 4.5))

    return fig,ax

#Make plots of surfaces
plt.close() # closes previous plots
x, y = np.mgrid[-1:1:31j, -1:1:31j] # create a 2d grid
fig1,ax1=plot_surface(x,y,monkey_saddle(x,y))
fig2,ax2=plot_surface(x,y,saddle_surface(x,y))
fig3,ax3=plot_surface(x,y,minima_surface(x,y,a=5,b=1))

#Contour plot of Beale's Function

fig4,ax4 =contour_beales_function()
plt.show()

```

Theory → First-order update — gradient descent and its pseudocode.

2.5 4.1 Gradient descent and Newton's method

We begin by introducing a simple first-order gradient descent method and comparing and contrasting it with another algorithm, Newton's method. Newton's method is intimately related to many algorithms (conjugate gradient, quasi-Newton methods) commonly used in physics for optimization problems. Denote the function we wish to minimize by $E(\mathbf{g})$. In the context of machine learning, $E(\mathbf{g})$ is just the cost function $E(\mathbf{g}) = C(\mathbf{X}, \mathbf{g}(\mathbf{g}))$. As we shall see for linear and logistic regression in Sections 6, 7, this energy function can almost always be written as a sum over n data points, n $E(\mathbf{g}) = \sum_{i=1}^n e(\mathbf{x}_i, \mathbf{g})$. (10) For example, for linear regression e is just the mean square-error for data point i ; for logistic regression, it is the i cross-entropy. To make analogy with physical systems, we will often refer to this function as the 'energy'. In the simplest gradient descent (GD) algorithm, we update the parameters as follows. Initialize the parameters to some value and iteratively update the parameters according to the equation $\mathbf{g}_{t+1} = \mathbf{g}_t - \eta \nabla E(\mathbf{g}_t)$, (11) where $\nabla E(\mathbf{g})$ is the gradient of $E(\mathbf{g})$ w.r.t. $\mathbf{g}$ and we have introduced a learning rate, η , that controls how big a step we should take in the direction of the gradient at time step t . It is clear that for sufficiently small choice of the learning rate η this method will converge to a local minimum (in all directions) of the cost function. However, choosing a small η comes at a huge computational cost. The smaller η , the more steps we have to take to reach the local minimum. In contrast, if η is too large, we can overshoot the minimum and the algorithm becomes unstable (it either oscillates or even moves $\mathbf{g}$ away from the minimum). This is shown in Fig. 7. In practice, one usually specifies a 'schedule' that decreases η at long t times. Common schedules include power law and exponential decay in time. To better understand this behavior and highlight some of the shortcomings of GD, it is useful to contrast GD with Newton's method which is the inspiration for many widely employed optimization methods. In Newton's method, we choose the step $\mathbf{v}$ for the parameters in such a way as to minimize a second-order Taylor expansion to the energy function $E(\mathbf{g} + \mathbf{v}) \approx E(\mathbf{g}) + \nabla E(\mathbf{g}) \cdot \mathbf{v} + \frac{1}{2} \mathbf{v}^T \mathbf{H}(\mathbf{g}) \mathbf{v}$, where $\mathbf{H}(\mathbf{g})$ is the Hessian matrix of second derivatives. Differentiating this equation respect to $\mathbf{v}$ and noting that for the optimal value $\mathbf{v}_{\text{opt}}$ we expect $\nabla E(\mathbf{g} + \mathbf{v}_{\text{opt}}) = 0$, yields the following equation $0 = \nabla E(\mathbf{g}) + \mathbf{H}(\mathbf{g}) \mathbf{v}_{\text{opt}}$. (12)

i Fig. 8

Effect of learning rate on convergence. For a one dimensional quadratic potential, one can show that there exists four different qualitative behaviors for gradient descent (GD) as a function of the learning rate η depending on the relationship between η and $\eta_{\text{opt}} = [2E''(\mathbf{g})]^{-1}$. (a) For $\eta < \eta_{\text{opt}}$, GD converges to the minimum. (b) For $\eta = \eta_{\text{opt}}$, GD converges in a single step. (c) For $\eta > \eta_{\text{opt}}$, GD oscillates around the minima and eventually converges. (d) For $\eta \gg \eta_{\text{opt}}$, GD moves away from the minima. | | | ————— | — | ————— | — |
| Rearranging this expression results in the desired update rules for Newton's method $-\mathbf{1}(\mathbf{g} + \mathbf{v} = \mathbf{H}(\mathbf{g})^{-1} \nabla E(\mathbf{g}))$ (13) | | — | ————— | — | — | | $\mathbf{g}_{t+1} = \mathbf{g}_t - \eta \nabla E(\mathbf{g}_t)$ (14) |
| Since we have no guarantee that the Hessian is well conditioned, in almost all applications of Newton's method, one

Source: This figure is adapted from (LeCun et al., 1998b).

Algorithm: gradient descent in pseudocode

```
theta = theta_0 ①
for t in range(max_iter): ②
    grad = compute_gradient(E, theta) ③
    v = eta_t * grad ④
    theta = theta - v ⑤
    if converged(theta): break ⑥
```

- ① Start from an initial guess for the parameters θ_0 .
- ② Outer optimisation loop — terminate either at a fixed iteration budget or by a convergence test (step 6).
- ③ Evaluate $\nabla_{\theta} E(\theta_t)$ at the current parameter vector.
- ④ Scale the gradient by the learning rate η_t . The schedule for η_t (constant, power-law decay, exponential decay) is the main practical knob.
- ⑤ Take a step opposite to the gradient direction: $\theta_{t+1} = \theta_t - v_t$ — eq. (11) in the source.
- ⑥ Stopping criterion — typically a norm-of-gradient threshold or a relative change in loss.

Practice → Vanilla gradient descent, classical momentum, and Nesterov accelerated gradient, with experiments.

2.6 Gradient descent with and without momentum

In this notebook, we will visualize various gradient descent algorithms used in machine learning. We will be especially interested in trying to understand how various hyperparameters – especially the learning rate – affect our performance. Here, we confine ourselves primarily to looking at the performance in the absence of noise. However, we encourage the reader to experiment with playing with the noise strength below and seeing what differences introducing stochasticity makes.

Throughout, we denote the parameters by θ and the energy function we are trying to minimize by $E(\theta)$.

Gradient Descent

We start by considering a simple gradient descent method. In this method, we will take steps in the direction of the local gradient. Given some parameters θ , we adjust the parameters at each iteration so that

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} E(\theta),$$

where we have introduced the learning rate η_t that controls how large a step we take. In general, the algorithm is extremely sensitive to the choice of η_t . If η_t is too large, then one can wildly oscillate around minima and miss important structure at small scales. This problem is amplified if our gradient computations are noisy and inexact (as is often the case in machine learning applications). If η_t is too small, then the learning/minimization procedure becomes extremely slow. This raises

the natural question: What sets the natural scale for the learning rate and how can we adaptively choose it? We discuss this extensively in Section IV of the review.

Gradient Descent with Momentum One problem with gradient descent is that it has no memory of where the “ball rolling down the hill” comes from. This can be an issue when there are many shallow minima in our landscape. If we make an analogy with a ball rolling down a hill, the lack of memory is equivalent to having no inertia or momentum (i.e. completely overdamped dynamics). Without momentum, the ball has no kinetic energy and cannot climb out of shallow minima.

Momentum becomes especially important when we start thinking about stochastic gradient descent with noisy, stochastic estimates of the gradient. In this case, we should remember where we were coming from and not react drastically to each new update.

Inspired by this, we can add a memory or momentum term to the stochastic gradient descent term above:

$$v_t = \gamma v_{t-1} + \eta_t \nabla_{\theta} E(\theta_t), \theta_{t+1} = \theta_t - v_t,$$

with $0 \leq \gamma < 1$ called the momentum parameter. When $\gamma = 0$, this reduces to ordinary gradient descent, and increasing γ increases the inertial contribution to the gradient. From the equations above, we can see that typical memory lifetimes of the gradient is given by $(1 - \gamma)^{-1}$. For $\gamma = 0$ as in gradient descent, the lifetime is just one step. For $\gamma = 0.9$, we typically remember a gradient for ten steps. We will call this gradient descent with classical momentum or CM for short.

A final widely used variant of gradient descent with momentum is called the Nesterov accelerated gradient (NAG). In NAG, rather than calculating the gradient at the current position, one calculates the gradient at the position momentum will carry us to at time $t + 1$, namely, $\theta_t - \gamma v_{t-1}$. Thus, the update becomes

$$v_t = \gamma v_{t-1} + \eta_t \nabla_{\theta} E(\theta_t - \gamma v_{t-1}), \theta_{t+1} = \theta_t - v_t$$

```
#This writes a simple gradient descent, gradient descent+ momentum,
#nesterov.

#Mean-gradient based methods
def gd(grad, init, n_epochs=1000, eta=10**-4, noise_strength=0):
    #This is a simple optimizer
    params=np.array(init) # initial condition
    param_traj=np.zeros([n_epochs+1,2]) # save each step
    param_traj[0,]=init
    v=0;
    for j in range(n_epochs):
        noise=noise_strength*np.random.randn(params.size)
        v=eta*(np.array(grad(params))+noise) # gradient step + noise
        params=params-v
        param_traj[j+1,]=params
    return param_traj
```

```

def gd_with_mom(grad, init, n_epochs=5000, eta=10**-4,
    ↪ gamma=0.9, noise_strength=0):
    params=np.array(init)
    param_traj=np.zeros([n_epochs+1,2])
    param_traj[0,]=init
    v=0
    for j in range(n_epochs):
        noise=noise_strength*np.random.randn(params.size)
        v=gamma*v+eta*(np.array(grad(params))+noise)
        params=params-v
        param_traj[j+1,]=params
    return param_traj

def NAG(grad, init, n_epochs=5000, eta=10**-4, gamma=0.9, noise_strength=0):
    params=np.array(init)
    param_traj=np.zeros([n_epochs+1,2])
    param_traj[0,]=init
    v=0
    for j in range(n_epochs):

        noise=noise_strength*np.random.randn(params.size)
        params_nesterov=params-gamma*v
        v=gamma*v+eta*(np.array(grad(params_nesterov))+noise)
        params=params-v
        param_traj[j+1,]=params
    return param_traj

```

2.7 Experiments with GD, CM, and NAG

Before introducing more complicated situations, let us experiment with these methods to gain some intuition.

Let us look at the dependence of GD on learning rate in a simple quadratic minima of the form $z = ax^2 + by^2 - 1$. Make plots below for $\eta = 0.1, 0.5, 1, 1.01$ and $a = 1$ and $b = 1$. (to do this, you would have to add additional arguments to the function `gd` above in order to pass the new values of a and b ; otherwise the default values $a=1$ and $b=1$ will be used by the gradient)

What are the qualitatively different behaviors that arise as η is increased?

What does this tell us about the importance of choosing learning parameters? How do these change if we change a and b above? In particular how does anisotropy change the learning behavior?

Make similar plots for CM and NAG? How do the learning rates for these procedures compare with those for GD?

```

# Investigate effect of learning rate in gradient descent with classical
  ↪ momentum
plt.close()
a,b = 1.0,1.0
x, y = np.meshgrid(np.arange(-4.5, 4.5, 0.2), np.arange(-4.5, 4.5, 0.2))
fig, ax = plt.subplots(figsize=(10, 6))
z=np.abs(minima_surface(x,y,a,b))
ax.contour(x, y, z, levels=np.logspace(0.0, 5, 35), norm=LogNorm(),
  ↪ cmap="RdYlBu_r")
ax.plot(0,0, 'r*', markersize=18)

#initial point
init1=[-2,4]
init2=[-1.7,4]
init3=[-1.5,4]
init4=[-3,4.5]
init5=[3,4.5]

eta1=0.1
eta2=0.5
eta3=1
eta4=1.01

gd_1=gd(grad_minima_surface,init1, n_epochs=100, eta=eta1)
gd_2=gd(grad_minima_surface,init2, n_epochs=50, eta=eta2)
gd_3=gd(grad_minima_surface,init3, n_epochs=50, eta=eta3)
gd_4=gd(grad_minima_surface,init4, n_epochs=10, eta=eta4)

#print(gd_1)
overlay_trajectory_contour(ax,gd_1,'$\eta$=%s'% eta1,'g--*', lw=0.5)
overlay_trajectory_contour(ax,gd_2,'$\eta$=%s'% eta2,'b-<', lw=0.5)
overlay_trajectory_contour(ax,gd_3,'$\eta$=%s'% eta3,'y->', lw=0.5)
overlay_trajectory_contour(ax,gd_4,'$\eta$=%s'% eta4,'c-o', lw=0.5)

plt.legend(loc=2)
plt.show()
fig.savefig("GD3regimes.pdf", bbox_inches='tight')

plt.close()
a,b = 1.0,1.0
x, y = np.meshgrid(np.arange(-4.5, 4.5, 0.2), np.arange(-4.5, 4.5, 0.2))
fig, (ax1, ax2) = plt.subplots(2,1,figsize=(10, 6))

```

```

z=np.abs(minima_surface(x,y,a,b))
#ax.contour(x, y, z, levels=np.logspace(0.0, 5, 35), norm=LogNorm(),
    ↪ cmap="RdYlBu_r")
#ax.plot(0,0, 'r*', markersize=18)
for ax in (ax1, ax2):
    ax.contour(x, y, z, levels=np.logspace(0.0, 5, 35), norm=LogNorm(),
    ↪ cmap="RdYlBu_r")
    ax.plot(0,0, 'r*', markersize=18)
    ax.set_xlim((-4.5, 4.5))
    ax.set_ylim((-4.5, 4.5))

#initial point
init1=[-2,4]
init2=[-1.7,4]
init3=[-1.5,4]
init4=[-3,4.5]
init5=[3,4.5]

#initial learning rates
eta1=0.1
eta2=0.5
eta3=1.0
eta4=1.01

gdwm_1=gd_with_mom(grad_minima_surface,init1, n_epochs=70,
    ↪ eta=eta1,gamma=0.9)
gdwm_2=gd_with_mom(grad_minima_surface,init2, n_epochs=70,
    ↪ eta=eta2,gamma=0.9)
gdwm_3=gd_with_mom(grad_minima_surface,init3, n_epochs=70,
    ↪ eta=eta3,gamma=0.9)
gdwm_4=gd_with_mom(grad_minima_surface,init4, n_epochs=70,
    ↪ eta=eta4,gamma=0.9)

nag_1=NAG(grad_minima_surface,init1, n_epochs=70, eta=eta1, gamma=0.9)
nag_2=NAG(grad_minima_surface,init2, n_epochs=70, eta=eta2, gamma=0.9)
nag_3=NAG(grad_minima_surface,init3, n_epochs=70, eta=eta3, gamma=0.9)
nag_4=NAG(grad_minima_surface,init4, n_epochs=70, eta=eta4, gamma=0.9)

#print(gd_1)
overlay_trajectory_contour(ax1,gdwm_1,r'$\eta$=%s'% eta1,'g--*', lw=0.5)
overlay_trajectory_contour(ax1,gdwm_2,r'$\eta$=%s'% eta2,'b-<', lw=0.5)

```

```

overlay_trajectory_contour(ax1,gdwm_3,r'$\eta=${s}'% eta3,'y->', lw=0.5)
overlay_trajectory_contour(ax1,gdwm_4,r'$\eta=${s}'% eta4,'c-o', lw=0.5)

```

```

overlay_trajectory_contour(ax2,nag_1,r'$\eta=${s}'% eta1,'g--*', lw=0.5)
overlay_trajectory_contour(ax2,nag_2,r'$\eta=${s}'% eta2,'b-<', lw=0.5)
overlay_trajectory_contour(ax2,nag_3,r'$\eta=${s}'% eta3,'y->', lw=0.5)
overlay_trajectory_contour(ax2,nag_4,r'$\eta=${s}'% eta4,'c-o', lw=0.5)

```

```

plt.legend(loc=2)
plt.show()
fig.savefig("gdwm_and_nag.pdf", bbox_inches='tight')

```

```

# the role of anisotropy  $a \neq b$ 
plt.close()
aa,bb = 1.0,5.0
x, y = np.meshgrid(np.arange(-4.5, 4.5, 0.2), np.arange(-4.5, 4.5, 0.2))
fig, (ax1, ax2) = plt.subplots(2,1, figsize=(10, 8))
z=np.abs(minima_surface(x,y,aa,bb))
for ax in (ax1, ax2):
    ax.contour(x, y, z, levels=np.logspace(0.0, 5, 35), norm=LogNorm(),
    ↪ cmap="RdYlBu_r")
    ax.plot(0,0, 'r*', markersize=18)
    ax.set_xlim((-4.5, 4.5))
    ax.set_ylim((-4.5, 4.5))

#initial point
init1=[-2,4]
init2=[-1.7,4]
init3=[-1.5,4]
init4=[-3,4.5]
init5=[3,4.5]

#initial learning rates
#eta1=0.1
#eta2=0.5
#eta3=1.0
#eta4=1.01

#test smaller choice for anisotropic case
def gd_asym(grad, init, n_epochs=1000, eta=10**-4, noise_strength=0, aa=1,
    ↪ bb=5):

```

```

#define a new gd function for the anisotropic case
params=np.array(init) # initial condition
param_traj=np.zeros([n_epochs+1,2]) # save each step
param_traj[0,]=init
v=0
for j in range(n_epochs):
    noise=noise_strength*np.random.randn(params.size)
    v=eta*(np.array(grad(params,aa,bb))+noise) # gradient step + noise
    params=params-v
    param_traj[j+1,]=params
return param_traj

def gd_with_mom_asym(grad, init, n_epochs=5000, eta=10**-4,noise_strength=0,
    ↪ aa=1, bb=5, gamma=0.9):
    params=np.array(init)
    param_traj=np.zeros([n_epochs+1,2])
    param_traj[0,]=init
    v=0
    for j in range(n_epochs):
        noise=noise_strength*np.random.randn(params.size)
        v=gamma*v+eta*(np.array(grad(params,aa,bb))+noise)
        params=params-v
        param_traj[j+1,]=params
    return param_traj

gd_1_asym=gd_asym(grad_minima_surface,init1, n_epochs=100, eta=eta1,aa=3,
    ↪ bb=5)
gd_2_asym=gd_asym(grad_minima_surface,init2, n_epochs=100, eta=eta2,aa=3,
    ↪ bb=5)
gd_3_asym=gd_asym(grad_minima_surface,init3, n_epochs=100, eta=eta3,aa=3,
    ↪ bb=5)
gd_4_asym=gd_asym(grad_minima_surface,init4, n_epochs=100, eta=eta4,aa=3,
    ↪ bb=5)

gdwm_1_asym=gd_with_mom_asym(grad_minima_surface,init1, n_epochs=100,
    ↪ eta=eta1,aa=3, bb=5, gamma=0.9)
gdwm_2_asym=gd_with_mom_asym(grad_minima_surface,init2, n_epochs=100,
    ↪ eta=eta2,aa=3, bb=5, gamma=0.9)
gdwm_3_asym=gd_with_mom_asym(grad_minima_surface,init3, n_epochs=100,
    ↪ eta=eta3,aa=3, bb=5, gamma=0.9)

```

```

gdwm_4_asym=gd_with_mom_asym(grad_minima_surface,init4, n_epochs=100,
    ↪ eta=eta4,aa=3, bb=5, gamma=0.9)

#print(gd_1)
overlay_trajectory_contour(ax1,gd_1_asym,r'$\eta$=%s' eta1,'g--*', lw=0.5)
overlay_trajectory_contour(ax1,gd_2_asym,r'$\eta$=%s' eta2,'b-<', lw=0.5)
overlay_trajectory_contour(ax1,gd_3_asym,r'$\eta$=%s' eta3,'y->', lw=0.5)
overlay_trajectory_contour(ax1,gd_4_asym,r'$\eta$=%s' eta4,'c-o', lw=0.5)

plt.legend(loc=2)

overlay_trajectory_contour(ax2,gdwm_1_asym,r'$\eta$=%s' eta1,'g--*',
    ↪ lw=0.5)
overlay_trajectory_contour(ax2,gdwm_2_asym,r'$\eta$=%s' eta2,'b-<', lw=0.5)
overlay_trajectory_contour(ax2,gdwm_3_asym,r'$\eta$=%s' eta3,'y->', lw=0.5)
overlay_trajectory_contour(ax2,gdwm_4_asym,r'$\eta$=%s' eta4,'c-o', lw=0.5)

plt.legend(loc=2)
plt.show()
fig.savefig("GD_GDWM_asym.pdf", bbox_inches='tight')

```

Theory → Second-order update — Newton’s method in pseudocode. The next section motivates ADAM and RMSProp as cheap approximations to this idea.

Algorithm: Newton’s method in pseudocode

```

theta = theta_0 ①
for t in range(max_iter): ②
    grad = compute_gradient(E, theta) ③
    H = compute_hessian(E, theta) ④
    v = solve(H, grad) ⑤
    theta = theta - v ⑥

```

- ① Same initialisation as plain GD.
- ② Outer loop.
- ③ First-order term: $\nabla_{\theta} E(\theta_t)$.
- ④ Second-order term: the Hessian matrix $H(\theta_t)$ of second derivatives.
- ⑤ Solve the linear system $H v = \nabla_{\theta} E$ — eq. (13). In practice the Hessian is regularised, since it

is rarely well-conditioned for ML loss landscapes.

- ⑥ Same parameter update as GD — eq. (14). Newton converges in one step on quadratic energies; on real losses it is corrected by line search.

Practice → Methods that track the second moment of the gradient — RMSProp and ADAM — with experiments.

2.8 Gradient Descents that utilize the second moment

In stochastic gradient descent, with and without momentum, we still have to specify a schedule for tuning the learning rates η_t as a function of time. As discussed in Sec. IV in the context of Newton's method, this presents a number of dilemmas. The learning rate is limited by the steepest direction which can change depending on where in the landscape we are. To circumvent this problem, ideally our algorithm would take large steps in shallow, flat directions and small steps in steep, narrow directions. Second-order methods accomplish this by calculating or approximating the Hessian and normalizing the learning rate by the curvature. However, this is very computationally expensive for extremely large models. Ideally, we would like to be able to adaptively change our step size to match the landscape without paying the steep computational price of calculating or approximating Hessians.

Recently, a number of methods have been introduced that accomplish this by tracking not only the gradient but also the second moment of the gradient. These methods include AdaGrad, AdaDelta, RMS-Prop, and ADAM. Here, we discuss the latter of these two as representatives of this class of algorithms.

In RMS prop (Root-Mean-Square propagation), in addition to keeping a running average of the first moment of the gradient, we also keep track of the second moment through a moving average. The update rule for RMS prop is given by

$$\mathbf{g}_t = \nabla_{\theta} E(\theta) \mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \mathbf{g}_t^2 \theta_{t+1} = \theta_t + \eta_t \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t + \epsilon}},$$

where β controls the averaging time of the second moment and is typically taken to be about $\beta = 0.9$, η_t is a learning rate typically chosen to be 10^{-3} , and $\epsilon \sim 10^{-8}$ is a small regularization constant to prevent divergences. It is clear from this formula that the learning rate is reduced in directions where the norm of the gradient is consistently large. This greatly speeds up the convergence by allowing us to use a larger learning rate for flat directions.

A related algorithm is the ADAM optimizer. In ADAM, we keep a running average of both the first and second moment of the gradient and use this information to adaptively change the learning rate for different parameters. In addition to keeping a running average of the first and second moments of the gradient, ADAM performs an additional bias correction to account for the fact that we are estimating the first two moments of the gradient using a running average (denoted by the hats in the update rule below). The update rule for ADAM is given by (where multiplication and division are understood to be element wise operations)

$$\mathbf{g}_t = \nabla_{\theta} E(\theta) \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1} \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2} \theta_{t+1} = \theta_t + \eta_t \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t + \epsilon}},$$

where β_1 and β_2 set the memory lifetime of the first and second moment and are typically take to be 0.9 and 0.99 respectively, and η and ϵ are identical to RMSprop.

```

}
↪ #####
# Methods that exploit first and second moments of gradient: RMS-PROP and
↪ ADAMS
}
↪ #####

def rms_prop(grad, init, n_epochs=5000, eta=10**-3,
↪ beta=0.9, epsilon=10**-8, noise_strength=0):
    params=np.array(init)
    param_traj=np.zeros([n_epochs+1,2])
    param_traj[0,]=init#Import relevant packages
    grad_sq=0;
    for j in range(n_epochs):
        noise=noise_strength*np.random.randn(params.size)
        g=np.array(grad(params))+noise
        grad_sq=beta*grad_sq+(1-beta)*g*g
        v=eta*np.divide(g,np.sqrt(grad_sq+epsilon))
        params= params-v
        param_traj[j+1,]=params
    return param_traj

def adams(grad, init, n_epochs=5000, eta=10**-4, gamma=0.9,
↪ beta=0.99, epsilon=10**-8, noise_strength=0):
    params=np.array(init)
    param_traj=np.zeros([n_epochs+1,2])
    param_traj[0,]=init
    v=0;
    grad_sq=0;
    for j in range(n_epochs):
        noise=noise_strength*np.random.randn(params.size)
        g=np.array(grad(params))+noise
        v=gamma*v+(1-gamma)*g
        grad_sq=beta*grad_sq+(1-beta)*g*g
        v_hat=v/(1-gamma)
        grad_sq_hat=grad_sq/(1-beta)
        params=params-eta*np.divide(v_hat,np.sqrt(grad_sq_hat+epsilon))
        param_traj[j+1,]=params
    return param_traj

```

2.9 Experiments with ADAM and RMSprop

In this section, we will experiment with ADAM and RMSprop. To do so, we will use a function commonly used in optimization protocols:

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2.$$

This function has a global minimum at $(x, y) = (3, 0.5)$. We will use GD, GD with classical momentum, NAG, RMSprop, and ADAM to find minima starting at different initial conditions.

One of the things you should experiment with is the learning rate and the number of steps, N_{steps} we take. Initially, we have set $N_{\text{steps}} = 10^4$ and the learning rate for ADAM/RMSprop to $\eta = 10^{-3}$ and the learning rate for the remaining methods to 10^{-6} .

Examine the plot for these default values. What do you see?

Make a plot when the learning rate of all methods is $\eta = 10^{-6}$? How does your plot change?

Now set the learning rate for all algorithms to $\eta = 10^{-3}$? What goes wrong? Why?

```
plt.close()
#Make static plot of the results
Nsteps=10**4
lr_l=10**-3
lr_s=10**-6

init1=np.array([4,3])
fig1, ax1=contour_beales_function()

gd_trajectory1=gd(grad_beales_function,init1,Nsteps, eta=lr_s,
    ↪ noise_strength=0)
gdm_trajectory1=gd_with_mom(grad_beales_function,init1,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
NAG_trajectory1=NAG(grad_beales_function,init1,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
rms_prop_trajectory1=rms_prop(grad_beales_function,init1,Nsteps,eta=lr_l,
    ↪ beta=0.9,epsilon=10**-8,noise_strength=0)
adam_trajectory1=adams(grad_beales_function,init1,Nsteps,eta=lr_l,
    ↪ gamma=0.9, beta=0.99,epsilon=10**-8,noise_strength=0)

overlay_trajectory_contour_M(ax1,gd_trajectory1, 'GD','k')
overlay_trajectory_contour_M(ax1,gdm_trajectory1, 'GDM','m')
overlay_trajectory_contour_M(ax1,NAG_trajectory1, 'NAG','c--')
overlay_trajectory_contour_M(ax1,rms_prop_trajectory1,'RMS', 'b-.')
overlay_trajectory_contour_M(ax1,adam_trajectory1,'ADAMS', 'r')

plt.legend(loc=2)
```

```

init2=np.array([1.5,1.5])
gd_trajectory2=gd(grad_beales_function,init2,Nsteps, eta=lr_s,
    ↪ noise_strength=0)
gdm_trajectory2=gd_with_mom(grad_beales_function,init2,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
NAG_trajectory2=NAG(grad_beales_function,init2,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
rms_prop_trajectory2=rms_prop(grad_beales_function,init2,Nsteps,eta=lr_l,
    ↪ beta=0.9,epsilon=10**-8,noise_strength=0)
adam_trajectory2=adams(grad_beales_function,init2,Nsteps,eta=lr_l,
    ↪ gamma=0.9, beta=0.99,epsilon=10**-8,noise_strength=0)

overlay_trajectory_contour_M(ax1,gdm_trajectory2, 'GDM','m')
overlay_trajectory_contour_M(ax1,NAG_trajectory2, 'NAG','c--')
overlay_trajectory_contour_M(ax1,rms_prop_trajectory2,'RMS', 'b-.')
overlay_trajectory_contour_M(ax1,adam_trajectory2,'ADAMS', 'r')

init3=np.array([-1,4])

gd_trajectory3=gd(grad_beales_function,init3,Nsteps, eta=lr_s,
    ↪ noise_strength=0)
gdm_trajectory3=gd_with_mom(grad_beales_function,init3,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
NAG_trajectory3=NAG(grad_beales_function,init3,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
rms_prop_trajectory3=rms_prop(grad_beales_function,init3,Nsteps,eta=lr_l,
    ↪ beta=0.9,epsilon=10**-8,noise_strength=0)
adam_trajectory3=adams(grad_beales_function,init3,Nsteps,eta=lr_l,
    ↪ gamma=0.9, beta=0.99,epsilon=10**-8,noise_strength=0)

overlay_trajectory_contour_M(ax1,gd_trajectory3, 'GD','k')
overlay_trajectory_contour_M(ax1,gdm_trajectory3, 'GDM','m')
overlay_trajectory_contour_M(ax1,NAG_trajectory3, 'NAG','c--')
overlay_trajectory_contour_M(ax1,rms_prop_trajectory3,'RMS', 'b-.')
overlay_trajectory_contour_M(ax1,adam_trajectory3,'ADAMS', 'r')

init4=np.array([-2,-4])

gd_trajectory4=gd(grad_beales_function,init4,Nsteps, eta=lr_s,
    ↪ noise_strength=0)
gdm_trajectory4=gd_with_mom(grad_beales_function,init4,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)

```

```

NAG_trajectory4=NAG(grad_beales_function,init4,Nsteps,eta=lr_s,
    ↪ gamma=0.9,noise_strength=0)
rms_prop_trajectory4=rms_prop(grad_beales_function,init4,Nsteps,eta=lr_l,
    ↪ beta=0.9,epsilon=10**-8,noise_strength=0)
adam_trajectory4=adams(grad_beales_function,init4,Nsteps,eta=lr_l,
    ↪ gamma=0.9, beta=0.99,epsilon=10**-8,noise_strength=0)

overlay_trajectory_contour_M(ax1,gd_trajectory4, 'GD','k')
overlay_trajectory_contour_M(ax1,gdm_trajectory4, 'GDM','m')
overlay_trajectory_contour_M(ax1,NAG_trajectory4, 'NAG','c--')
overlay_trajectory_contour_M(ax1,rms_prop_trajectory4,'RMS', 'b-.')
overlay_trajectory_contour_M(ax1,adam_trajectory4,'ADAMS', 'r')

plt.show()

```

3 Lesson 2 — Clustering (DBSCAN)

The theory in this chapter is extracted from Mehta *et al.* (Physics Reports 810, 2019), §13. The full review lives under `2sem/LCPB/source/` in the repository.

Theory → What clustering tries to do, and the picture behind K-means and DBSCAN.

3.1 13. Clustering

In this section, we continue our discussion of unsupervised learning methods. Unsupervised learning is concerned with discovering structure in unlabeled data (for instance learning local structures for data visualization, see Section 12). The lack of labels make unsupervised learning much more difficult and subtle than its supervised counterpart. What is somewhat surprising is that even without labels it is still possible to uncover and exploit the hidden structure in the data. Perhaps, the simplest example of unsupervised learning is clustering. The aim of clustering is to group unlabeled data into clusters according to some similarity or distance measure. Informally, a cluster is thought of as a set of points sharing some pattern or structure. Clustering finds many applications throughout data mining (Larsen and A one, 1999), data compression and signal processing (Gersho and Gray, 2012; MacKay, 2003). Clustering can be used to identify coarse features or high level structures in an unlabeled data set. The technique also finds many applications in physical sciences, ranging from detecting celestial emission sources in astronomical surveys (Sander et al., 1998) to inferring groups of genes and proteins with similar functions in biology (Eisen et al., 1998), and building entanglement classifiers (Lu et al., 2017). Clustering is perhaps

Fig. 53

Different visualizations of a Gaussian mixture formed of $K = 30$ mixtures in a $D = 40$ dimensional space. The Gaussians have the same covariance but have means drawn uniformly at random in the space $[-10, 10]^{40}$. (a) Plot of the first two coordinates. The labels of the different Gaussians are indicated by the different colors. Note that in a realistic setting, label information is of course not available, thus making it very hard to distinguish the different clusters. (b) Random projection of the data onto a 2-dimensional space. (c) Projection onto the first 2 principal components. Only a small fraction of the variance is explained by those components (the ratio is indicated along the axis). (d) t-SNE embedding (perplexity = 60, #iteration = 1000) in a 2-dimensional latent space. t-SNE captures correctly the local structure of the data.

i Fig. 54

Visualization of the MNIST handwritten digits training data set (here $N = 60000$). (a) First two principal components. (b) t-SNE applied with a perplexity of 30, a Barnes–Hut angle of 0.5 and 1000 gradient descent iterations. In order to reduce the noise and speed-up computation, PCA was first applied to the data set to project it down to 40 dimensions. We used an open-source implementation to produce the results (Linderman et al., 2017), see <https://github.com/KlugerLab/Flt-SNE>. The simplest way to look for hidden structure in a data set and for this reason, is among the most widely used and employed data analysis and machine learning techniques. The field of clustering is vast and there exists a flurry of clustering methods suited for different purposes. Some common considerations one has to take into account when choosing a particular method is the distribution of the clusters (overlapping/noisy clusters vs. well-separated clusters), the geometry of the data (flat vs. non-flat), the cluster size distribution (multiple sizes vs. uniform sizes), the dimensionality of the data (low vs. high dimensional) and the computational efficiency of the desired method (small vs. large data set).

We begin Section 13.1 with a focus on popular practical clustering methods such as K-means clustering, hierarchical clustering and density clustering. Our goal is to highlight the strength, weaknesses and differences between these techniques, while laying out some of the theoretical framework required for clustering analysis. There exist many more

i Fig. 55

K-means with $K=3$ applied to an artificial two-dimensional data set. The cluster means at each iteration are indicated by cyan star markers. t indicates the iteration number and C the value of the objective function. (a) The algorithm is initialized by randomly partitioning the space into 3 sectors to generate an initial assignment. (b)–(c) For well separated clusters, the algorithm converges rapidly to the true clusters. (d) The objective function as a function of the iteration. C converges after $t=18$ iterations for this choice of random seed (for center initialization). clustering methods beyond those discussed in this section. 16 The methods we discuss were chosen for their pedagogical value and/or their applicability to problems in physics. In Section 13.2 we discuss Gaussian mixture models and the formulation of clustering through latent variable models.

This section introduces many of the methods we will encounter when discussing other unsupervised learning methods later in the review. Finally, in Section 13.3 we discuss the problem of clustering in high-dimensional data and possible ways to tackle this difficult problem. The reader is also urged to experiment with various clustering methods using Notebook 15.

Practice → Synthetic data on which the methods will be exercised.

```
import numpy as np
from numpy import cos, sin, pi, random
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
```

```
plt.rcParams['font.size'] = 13
mycmap = ListedColormap(["blue","red","gold" ])
from scipy.stats import ortho_group
```

```
!mkdir DATA
```

```
np.random.seed(123)

# number of data samples
N=600
# size of each sample
L=3
LL=12
# rotation
OM = ortho_group.rvs(dim=L) #random orthogonal matrix for rotation
OMM = ortho_group.rvs(dim=LL) #random orthogonal matrix for rotation of the
    ↪ 12d data
# % of points in cluster 0 and 1
p0,p1=35,45
# radii for 3d construction
R0=800
R1=400

NOISE=5

x,y,r,noise=np.zeros((N,L)),np.zeros(N),np.random.randint(0,100,
    ↪ ,N),np.random.randn(N,L)*10
xx,nnoise=np.zeros((N,LL)),np.random.randn(N,LL)*10
X,XX=np.zeros_like(x),np.zeros_like(xx)

for n in range(N):
    if r[n]<p0:
        y[n]=0
        q = random.rand()*2*pi
        x[n,:3] = (R0*cos(q), R0*sin(q), -(R0/10)*cos(6*q))
    elif r[n]<p1+p0:
        y[n]=1
        q = random.rand()*2*pi
        x[n,:3] = (-(R1/5)*sin(4*q), R1*cos(q), R1*sin(q))
        x[n,:3] = x[n,:3] + R0*np.array((0,random.randint(3) - 1, 0))
    else:
        y[n]=2
        q = random.rand()*1000.
```

```

        x[n,:3] = ((random.randint(2)-0.5)*2*q, 1.333*q-R1,
↪ 1.333*R1-0.333*q)
        x[n] = x[n]+noise[n]
        xx[n,:3] = x[n] # copy the 3d data to the first 3 dimensions of the 12d
↪ data
        nnoise[n,3:] = nnoise[n,3:] * NOISE # add noise to the 12d data
        xx[n] = xx[n] + nnoise[n] # add noise to the 12d data
        X[n] = np.dot(OM,x[n]) # rotate the 3d data
        XX[n] = np.dot(OMM,xx[n]) # rotate the 12d data
y = y.astype(int)

fig = plt.figure(figsize=(15, 5))
for i in range(2):
    ax = fig.add_subplot(1,3,i+1,projection='3d')
    ax.scatter(x[:,0], x[:,1], x[:,2], s=5, c=y, depthshade=True,
↪ cmap=mycmap)
    ax.view_init(25+15*i, 45*(i-1))

plt.draw()

np.savetxt("DATA/x_12d.dat",XX,delimiter='\t')
np.savetxt("DATA/y_12d.dat",y,fmt="%d")

x0=np.copy(x)

```

```

def generate_data(N,L,LL,R0,R1,p0,p1,NOISE):

    OM = ortho_group.rvs(dim=L) #random orthogonal matrix for rotation
    OMM = ortho_group.rvs(dim=LL) #random orthogonal matrix for rotation of
↪ the 12d data
    xx,nnoise=np.zeros((N,LL)),np.random.randn(N,LL)*10
    X,XX=np.zeros_like(x),np.zeros_like(xx)

    for n in range(N):
        if r[n]<p0:
            y[n]=0
            q = random.rand()*2*pi
            x[n,:3] = (R0*cos(q), R0*sin(q), -(R0/10)*cos(6*q))
        elif r[n]<p1+p0:
            y[n]=1
            q = random.rand()*2*pi
            x[n,:3] = (-(R1/5)*sin(4*q), R1*cos(q), R1*sin(q))

```

```

        x[n,:3] = x[n,:3] + R0*np.array((0,random.randint(3) - 1, 0))
    else:
        y[n]=2
        q = random.rand()*1000.
        x[n,:3] = ((random.randint(2)-0.5)*2*q, 1.333*q-R1,
↪ 1.333*R1-0.333*q)
        x[n] = x[n]+noise[n]
        xx[n,:3] = x[n] # copy the 3d data to the first 3 dimensions of the
↪ 12d data
        nnoise[n,3:] = nnoise[n,3:] * NOISE # add noise to the 12d data
        xx[n] = xx[n] + nnoise[n] # add noise to the 12d data
        X[n] = np.dot(OM,x[n]) # rotate the 3d data
        XX[n] = np.dot(OMM,xx[n]) # rotate the 12d data
    y = y.astype(int)
    np.savetxt(f"DATA/x_{LL}d.dat",XX,delimiter='\t')
    np.savetxt(f"DATA/y_{LL}d.dat",y,fmt="%d")
    return XX,y

```

Theory → Practical algorithms — K-means and DBSCAN in pseudocode.

3.2 13.1 Practical clustering methods

Throughout this section we focus on the Euclidean distance as a similarity measure. Other measures may be better suited for specific problems. We refer the enthusiast reader to (Rokach and Maimon, 2005) for a more in-depth discussion of the different possible similarity measures.

13.1.1. K-Means We begin our discussion with K-means clustering since this method is simple to implement and understand, and covers the core concepts of clustering. Consider a set of N unlabeled observations $\{x\}_N$ where $x \in \mathbb{R}^p$ and where p is the number of features. Also consider a set of K cluster centers called the cluster means: $\{\mu\}_K$, with $\mu \in \mathbb{R}^p$, which we will compute “empirically” in the clustering procedure. The cluster means can be thought of as the representatives of each cluster, to which data points are assigned (see Fig. 55). K-means clustering can be formulated as follows: given a fixed integer K , find the cluster means $\{\mu\}$ and the data point assignments in order to minimize the following objective function:
$$C(\{x, \mu\}) = \sum_{k=1}^K \sum_{n=1}^N r_{nk} \|x_n - \mu_k\|^2, \quad (137)$$
 where $r_{nk} \in \{0, 1\}$ is a binary variable called the assignment. The assignment r_{nk} is 1 if x_n is assigned to cluster k and 0 otherwise. Notice that $\sum_k r_{nk} = 1$ and $\sum_n r_{nk} = N_k$, where N_k is the number of points assigned to cluster k . The minimization of this objective function can be understood as trying to find the best cluster means such that the variance within each cluster is minimized. In physical terms, C is equivalent to the sum of the moments of inertia of every cluster. Indeed, as we will see below, the cluster means μ_k correspond to the centers of mass of the respective cluster.

K-Means algorithm. The K-means algorithm alternates between two steps:

Expectation: Given a set of assignments $\{r_{nk}\}$, minimize C with respect to $\{\mu_k\}$. Taking a simple derivative and setting $\frac{\partial C}{\partial \mu_k} = 0$, we find that the optimal cluster mean μ_k is the centroid of the points assigned to cluster k .

Setting μ_k to zero yields the following update rule: $\mu_k = \frac{1}{n_k} \sum_{i \in \mathcal{C}_k} x_i$ (138)

3.3 2. Maximization: Given a set of cluster means μ_k , find the assignments n_k which minimizes C . Clearly, this is

achieved by assigning each data point to their nearest cluster-mean: $i \rightarrow k = \arg \min_k \|x_i - \mu_k\|^2$ (139)

K-means clustering consists in alternating between these two steps until some convergence criterion is met. Practically, the algorithm should terminate when the change in the objective function from one iteration to another becomes smaller than a pre-specified threshold. A simple example of K-means is presented in Fig.55. A nice property of the K-means algorithm is that it is guaranteed to converge. To see this, one can verify explicitly (by taking second-order derivatives) that the expectation step always decreases C . This is also true for the assignment step. Thus, since C is bounded from below, the two-step iteration of K-means always converges to a local minimum of C . Since C is generally a non-convex function, in practice one usually needs to run the algorithm with different initial random cluster center initialization s and post-select the best local minimum. A simple implementation of K-means has an average computational complexity which scales linearly in the size of the data set (more specifically the complexity is $O(KN)$ per iteration) and is thus scalable to very large datasets. As we will see in Section 13.2, K-means is a hard-assignment limit of the Gaussian mixture model where all cluster variances are assumed to be the same. This highlights a common drawback of K-means: if the true clusters have very different variances (spreads), K-means can lead to spurious results since the underlying assumption is that the latent model has uniform variances.

13.1.2. Hierarchical clustering: Agglomerative methods Agglomerative clustering is a bottom up approach that starts from small initial clusters which are then progressively merged to form larger clusters. The merging process generates a hierarchy of clusters that can be visualized in the form of a dendrogram (see Fig.56). This hierarchy can be useful to analyze the relation between clusters and the sub components of individual clusters. Agglomerative methods are usually specified by defining a distance measure between clusters. We denote the distance between clusters X and Y by $d(X, Y)$. Different choices of distance result in different clustering

Algorithm: K-means in pseudocode

```

mu = initial_centroids(X, K) ①
for t in range(max_iter):
    labels = argmin_dist(X, mu) ②
    mu_new = group_means(X, labels, K) ③
    if max_shift(mu, mu_new) < tol: break ④
    mu = mu_new
return labels, mu ⑤

```

- ① Initialise K cluster centroids — usually with K-means++ or by drawing random points from the dataset.

- ② **Assignment step:** every point x_i is assigned to the closest centroid under the chosen distance (Euclidean by default).
- ③ **Update step:** each centroid moves to the empirical mean of the points currently assigned to it.
- ④ Stopping criterion — centroids barely move from one iteration to the next.
- ⑤ Return both the per-point labels and the final centroid positions.

Algorithm: DBSCAN in pseudocode

```

labels = [UNVISITED] * len(X) ①
cid = 0
for i, x in enumerate(X):
    if labels[i] != UNVISITED: continue
    N = region_query(X, x, eps) ②
    if len(N) < min_pts:
        labels[i] = NOISE ③
        continue
    cid += 1
    expand_cluster(X, labels, i, N, cid, ④
                  eps, min_pts)
return labels

```

- ① Mark every point as not-yet-visited.
- ② Find all points within ε of x — these are its ε -neighbours.
- ③ Below the density threshold minPts → label as noise. May be relabelled later if it becomes a border point.
- ④ Otherwise start a new cluster and recursively grow it by sweeping the density-reachable neighbourhood.

Practice → DBSCAN run, nearest-neighbour distance, t-SNE visualisation.

```

%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.ticker import NullFormatter
plt.rcParams['font.size'] = 12

# manifold.TSNE
from sklearn import manifold #, datasets
from time import time

from sklearn.neighbors import NearestNeighbors

```

```

from scipy.cluster.hierarchy import dendrogram
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score, fowlkes_mallows_score

```

In this example we know the labels, hence we can test the performances of t-SNE and DBSCAN

```

fname,fnamey="x_12d.dat","y_12d.dat"
data=np.loadtxt("DATA/"+fname, delimiter='\t') #load data and labels
y=np.loadtxt("DATA/"+fnamey,dtype=int)
N=len(data) #number of data points
D=len(data[0]) #number of features
print("N=",N," L=D=",D)
x=data

```

3.4 Find the typical minimum distance between points

This yields a typical scale, to be used as a reference for “eps”

```

nn = 10
# compute the distance to the nn nearest neighbors
nbrs = NearestNeighbors(n_neighbors=nn+1).fit(x) #, algorithm='ball_tree'
# distances and the indices of the nn nearest neighbors (10+1)
d, ind = nbrs.kneighbors(x) # returns distances and indices of nn neighbors
print(d[3,:],range(ind.max()))
dd=d.T # distances to the i-th neighbor for all points, sorted by distance
print(dd.shape)
# note: dd[0] has a trivial set of zeros
listc= ["#0000DD", "#8000FF", "#FF00FF", "#FF0080", "#DD0000",
        "#FF8000", "#E0E000", "#30D000", "#00A030", "#33A0A0"]
for i in range(1,nn+1):
    plt.plot(np.sort(dd[i]),c=listc[i-1],label=str(i))
    plt.xlabel("rank")
    plt.ylabel("distance to i-th neighbor")
plt.legend(ncol=2)
plt.show()
mean_min_dist = np.mean(dd[1]) # mean distance to the nearest neighbor
print("mean minimum distance between nn points: ",mean_min_dist)

```

3.5 t-SNE visualization, for true labels

```
np.random.seed(0)
n_samples = N # number of data points to consider
n_components = 2 # dimension of the embedded space
perplexities = [2, 4, 8, 16, 32]
Np = len(perplexities)
mycmap = ListedColormap(["blue","red","gold" ])

(fig, subplots) = plt.subplots(1,Np , figsize=(3*Np, 2.7))

X_all = [None]
for i, perplexity in enumerate(perplexities):
    ax = subplots[i]

    t0 = time()
    tsne = manifold.TSNE(n_components=n_components, init='random',
                        random_state=0, perplexity=perplexity,
    ↪ learning_rate='auto')
    Y = tsne.fit_transform(data) # apply t-SNE to the data
    t1 = time() # time taken by t-SNE
    print("t-SNE with perplexity=%d: %.2g sec" % (perplexity, t1 - t0))
    ax.set_title("Perplexity=%d" % perplexity)
    ax.scatter(Y[:, 0], Y[:, 1], c=y,cmap=mycmap) # plot the t-SNE embedding
    ax.tick_params(left=False,bottom=False)
    ax.xaxis.set_major_formatter(NullFormatter())
    ax.yaxis.set_major_formatter(NullFormatter())
    ax.axis('tight')
    X_all.append(Y)

plt.show()
```

3.6 Different random initial states (random_state=i)

```
n_samples = N
n_components = 2
mycmap = ListedColormap(["blue","red","gold" ])

(fig, subplots) = plt.subplots(1,Np , figsize=(3*Np, 2.7))

X_all = [None]
```

```

perplexity= 8

for i in range(5):
    np.random.seed(i)
    ax = subplots[i]

    t0 = time()
    tsne = manifold.TSNE(n_components=n_components,
                          init='random',random_state=i,
                          perplexity=perplexity)
    Y = tsne.fit_transform(data)
    t1 = time()
    ax.set_title("Perplexity=%d" % perplexity)
    ax.scatter(Y[:, 0], Y[:, 1], c=y,cmap=mycmap)
    ax.tick_params(left=False,bottom=False)
    ax.xaxis.set_major_formatter(NullFormatter())
    ax.yaxis.set_major_formatter(NullFormatter())
    ax.axis('tight')
    X_all.append(Y) # store the t-SNE embedding for later use

plt.show()

```

3.7 DBSCAN: find labels by applying clustering

```

from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
cpalette = ["blue","red","gold","black","magenta","green","cyan","#1CE6FF",
    ↪ "#FF34FF", "#FF4A46", "#008941", "#006FA6", "#A30059", "#0000A6",
    ↪ "#63FFAC", "#B79762", "#004D43", "#8FB0FF", "#997D87", "#5A0007",
    ↪ "#809693", "#1B4400", "#4FC601", "#3B5DFF",
    ↪ "#4A3B53", "#886F4C", "#34362D", "#B4A8BD", "#00A6AA",
    ↪ "#452C2C", "#636375", "#A3C8C9", "#FF913F", "#938A81", "#575329",
    ↪ "#00FECF", "#B05B6F"]
from collections import OrderedDict
ms = 6

def clustering(y):
    # Finds position of labels and returns a dictionary of cluster labels to
    ↪ data indices.
    yu = np.sort(np.unique(y))
    clustering = OrderedDict()
    for ye in yu:

```

```

        clustering[ye] = np.where(y == ye)[0]
    return clustering

def entropy(c, n_sample):
    # Measures the shannon entropy of a cluster
    # c is a dictionary of cluster labels to data indices
    # n_sample is the total number of data points
    h = 0.
    for kc in c.keys():
        p=len(c[kc])/n_sample
        h+=p*np.log(p)
    h*=-1.
    return h

# Normalized mutual information function
# Note that this deals with the label permutation problem
def NMI(y_true, y_pred):
    """ Computes normalized mutual information: where y_true and y_pred are
        ↪ both clustering assignments
    """

    w = clustering(y_true) # clustering of the true labels
    c = clustering(y_pred) # clustering of the predicted labels
    n_sample = len(y_true)

    Iwc = 0. # mutual information between w and c
    for kw in w.keys():
        for kc in c.keys():
            w_intersect_c=len(set(w[kw]).intersection(set(c[kc]))) # number
            ↪ of data points in the intersection of cluster kw and kc
            if w_intersect_c > 0:
                Iwc +=
                ↪ w_intersect_c*np.log(n_sample*w_intersect_c/(len(w[kw])*len(c[kc]))) #
                ↪ mutual information contribution of the intersection of cluster kw and kc
            Iwc/=n_sample
    Hc = entropy(c,n_sample) # entropy of the predicted clustering
    Hw = entropy(w,n_sample) # entropy of the true clustering

    return 2*Iwc/(Hc+Hw)

def plotting_ax(X, y, ax):
    # plotting function
    for i, yu in enumerate(np.unique(y)):
        pos = (y == yu)
        ax.scatter(X[pos,0], X[pos,1],c=cpalette[i%len(cpalette)],s=ms)

```

```

# DBSCAN has a few parameters, let's sweep over a few parameters and see
↳ what happens
np.random.seed(12345)
n_true_center=3
#X, ytrue = make_blobs(10000, 2, n_true_center) # Generating random gaussian
↳ mixture
X2D = X_all[2] # best(?) visualization of the data in 2D, obtained by t-SNE
↳ with perplexity=8
X = data
y_true = y

#eps_range = [0.01,0.1,0.5,1.0,10.]
if mean_min_dist<10:
    r=mean_min_dist # mean minimum distance between nn points
else:
    r = int(mean_min_dist)
eps_range = [r/2,r,1.5*r,2*r] # range of eps values to try

min_sample_range = [2,5,8,10]
DIMY=len(eps_range)
DIMX=len(min_sample_range)
fig, ax = plt.subplots(DIMY+1,DIMX,figsize=(14,16.8))

i,j=0,0
nmi=1
plotting_ax(X2D,y,ax[i,j])
ax[i,j].set_title('True labels')
for j in range(1,DIMX): fig.delaxes(ax[i,j])

for i, eps in enumerate(eps_range):
    for j, min_samples in enumerate(min_sample_range):
        model = DBSCAN(eps=eps, min_samples=min_samples)
        model.fit(X)
        y_hat = model.labels_ # predicted labels
        plotting_ax(X2D,y_hat,ax[i+1,j])
        nmi=NMI(y_hat, y_true)
        ax[i+1,j].set_title(r'\epsilon$=%.0f, $m_P$=%i,
↳ NMI=%.3f'%(eps,min_samples,nmi))

plt.tight_layout(h_pad=0.5)
plt.show()

```

4 Lesson 3 — Deep neural networks

The theory in this chapter is extracted from Mehta *et al.* (Physics Reports 810, 2019), §9 on feed-forward DNNs. The full review lives under `2sem/LCPB/source/`.

Theory → Neural-network basics and the feed-forward architecture.

4.1 9. An introduction to feed-forward deep neural networks (DNNs)

Over the last decade, neural networks have emerged as the one of most powerful and widely-used supervised learning techniques. Deep Neural Networks (DNNs) have a long history (Bishop, 1995a; Schmidhuber, 2015), but re-emerged to prominence after a rebranding as ‘Deep Learning’ in the mid-2000s (Hintikka et al., 2006; Hinton and Salakhutdinov, 2006). DNNs truly caught the attention of the wider machine learning community and industry in 2012 when Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton used a GPU-based DNN model (AlexNet) to lower the error rate on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) by an incredible twelve percent from 28% to 16% (Krizhevsky et al., 2012). Just three years later, a machine learning group from Microsoft achieved an error of 3.57% using an ultra-deep residual neural network (ResNet) with 152 layers (He et al., 2016)! Since then, DNNs have become the workhorse technique for many image and speech recognition based machine learning tasks. The large-scale industrial deployment of DNNs has given rise to a number of high-level libraries and packages (Caffe, Keras, PyTorch, TensorFlow, etc.) that make it easy to quickly code and deploy DNNs. Conceptually, it is helpful to divide neural networks into four categories: (i) general purpose neural networks for supervised learning, (ii) neural networks designed specifically for image processing, the most prominent example of this class being Convolutional Neural Networks (CNNs), (iii) neural networks for sequential data such as Recurrent Neural Networks (RNNs), and (iv) neural networks for unsupervised learning such as Deep Boltzmann Machines. Here, we will limit our discussions to the first two categories (unsupervised learning is discussed later in the review). Though increasingly important for many applications such as audio and speech recognition, for the sake of brevity, we omit a discussion of sequential data and RNNs from this review. For an introduction to RNNs and LSTM networks see Chris Olah’s blog, <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>, and Chapter 13 of (Bishop, 2006) as well as the introduction to RNNs in Chapter 10 of (Goodfellow et al., 2016) for sequential data. Due to the number of recent books on deep learning (see for example Michael Nielsen’s introductory online book (Nielsen, 2015) and the more advanced (Goodfellow et al., 2016)), the goal of this section is to give a high-level introduction to the basic ideas behind DNNs, and provide some practical knowledge for coding simple neural nets for supervised learning tasks (see the accompanying Notebooks). This section assumes the reader is familiar with the basic concepts introduced in earlier sections on logistic and linear regression. Throughout, we strive to

provide intuition behind the inner workings of DNNs, as well as highlight limitations of present-day algorithms. The influx of corporate and industrial interests has rapidly transformed the field in the last few years. This massive influx of money and researchers has given rise to new dogmas and best practices that change rapidly. As with most intellectual fields experiencing rapid expansion, many commonly accepted heuristics many turn out not to be as powerful as thought (Wilson et al., 2017), and widely held beliefs not as universal as once imagined (Lee et al., 2017; Zhang et al., 2016). This is especially true in modern neural networks where results are largely empirical and heuristic and lack the firm footing of many earlier machine learning methods. For this reason, in this review we have chosen to emphasize tried

and true fundamentals, while pointing out what, from our current vantage point, seem like promising new techniques. The field is rapidly evolving and readers are urged to read papers and to implement these algorithms themselves in order to gain a deep appreciation for the incredible power of modern neural networks, especially in the context of image, speech, and natural language processing, as well as limitations of the current methods. In physics, DNNs and CNNs have already found numerous applications. In statistical physics, they have been applied to detect phase transitions in 2D Ising (Tanaka and To miya, 2017a) and Potts (Li et al., 2017) models, lattice gauge theories (Wetzel and Scherzer, 2017), and different phases of polymers (Wei et al., 2017). It has also been shown that deep neural networks can be used to learn free-energy landscapes (Sidky and Whitmer, 2017). At the same time, methods from statistical physics have been applied to the field of deep learning to study the thermodynamic efficiency of learning rules (Goldt and Seifert, 2017), to explore the hypothesis space that DNNs span, make analogies between training DNNs and spin glasses (Bai et al., 2018; Baldassari et al., 2017), and to characterize phase transitions with respect to network topology in terms of errors (Li and Saeed, 2017). In relativistic hydrodynamics, deep learning has been shown to capture features of non-linear evolution and has the potential to accelerate numerical simulations (Huang et al., 2018), while in mechanics CNNs have been used to predict eigenvalues of photonic crystals (Finole et al., 2018). Deep CNNs were employed in lensing reconstruction of the cosmic microwave background (Caldeira et al., 2018). Recently, DNNs have been used to improve the efficiency of Monte-Carlo algorithms (Shen et al., 2018). Deep learning has also found interesting applications in quantum physics. Various quantum phase transitions (Arai et al., 2017; Broecker et al., 2017; Iakovlev et al., 2018; Suchsland and Wessel, 2018; van Nieuwenburg et al., 2017b) can be detected and studied using DNNs and CNNs, including the transverse-field Ising model (Ohtsuki and Ohtsuki, 2017), topological phases (Yoshio et al., 2017; Zhang et al., 2017a,b) and non-invasive topological quality control (Cao et al., 2019). DNNs found applications even in non-equilibrium many-body localization (Schindler et al., 2017; van Nieuwenburg et al., 2017a,b; Venderley et al., 2017), and the characterization of photo excited quantum states (Shinjo et al., 2019). Experimentally, DNNs were recently employed in cold atoms to identify critical points (Rem et al., 2018). Representing quantum states as DNNs (Gao and Duan, 2017; Gao et al., 2017; Levine et al., 2017; Saito and Kato, 2017) and quantum state tomography (Torlai et al., 2018) are among some of the impressive achievements to reveal the potential of deep learning to facilitate the study of quantum systems. Machine learning techniques involving neural networks were also used to study quantum and fault-tolerant error correction (Baireuther et al., 2017; Breuckmann and Ni, 2017; Chamberland and Ronagh, 2018; Davas et al., 2018; Krasnov and Jiang, 2017; Maskara et al., 2018), estimate rates of coherent and incoherent quantum processes (Greplov et al., 2017), to obtain spectra of $1/f$ -noise in spin-qubit devices (Zhang and Wang, 2018), and

the recognition of state and charge configurations and auto-tuning in quantum dots (Kalan tre et al., 2017). In quantum information theory, it has been shown that one can perform gate decomposition s with the help of neural nets(Swaddle et al.,2017).In lattice quantum chromodynamics,DN Ns have been used to learn action parameters in regions of parameter space where principal component analysis fails (Shanahan et al., 2018). CNN s were applied to data from a high-energy experiment to identify particle interactions in sampling calorimeters used commonly in neutrino physics (Aur is a no et al., 2016). Last but not least, DN Ns also found place in the study of quantum control (Yang et al., 2017),and in scattering theory to learn the s-wave scattering length(Wu et al.,2018)of potentials.

4.1.1 9.1 Neural network basics

Neural networks(also called neural nets)are neural-inspired nonlinear models for supervised learning.As we will see, neural nets can be viewed as natural,more powerful extensions of supervised learning methods such as linear and logistic regression and soft-max methods. 9.1.1. The basic building block:neurons The basic unit of a neural net is a stylized ‘‘neuron’’ i that takes a vector of d input features $x = (x_1, x_2, \dots, x_d)$ and produces a scalar output $a(x)$. A neural network consists of many such neurons stacked into layers, with the output of i one layer serving as the input for the next (see Fig. 35). The first layer in the neural net is called the input layer, the middle layers are often called ‘‘hidden layers’’,and the final layer is called the output layer. The exact function a varies depending on the type of non-linearity used in the neural network.However,in essentially i all cases a can be decomposed into a linear operation that weights the relative importance of the various inputs,and a i non-linear transformation (z)which is usually the same for all neurons.The linear transformation in almost all neural i networks takes the form of a dot product with a set of neuron-specific weights $w(i) = (w_1(i), w_2(i), \dots, w_d(i))$ followed by 1 2 d re-centering with a neuron-specific bias $b(i)$: $z(i) = w(i) \cdot x + b(i) = x^T \cdot w(i)$, (117) where $x = (1, x)$ and $w(i) = (b(i), w_1(i), \dots, w_d(i))$. In terms of $z(i)$ and the non-linear function (σ), we can write the full input–output i function as $a(x) = \sigma(z(i))$, (118) i i see Fig.35.

i Fig. 35

Basic architecture of neural networks. (A)The basic components of a neural network are stylized neurons consisting of a linear transformation that weights the importance of various inputs,followed by an on-linear activation function.(b)Neurons are arranged into layers with the output of one layer serving as the input to the next layer.

i Fig. 36

Possible non-linear activation functions for neurons. In modern DN Ns, it has become common to use non-linear functions that do not saturate for large inputs(bottom row)rather than saturating functions(top row). Historically in the neural network literature,common choices of nonlinear i ties included step-functions(perce ptr on s), sig moi ds(i.e.Fermi functions),and the hyperbolic tangent.More recently,it has become more common to use rectified linear units(ReL Us),leaky rectified linear units(leaky ReL Us),and exponential linear units(EL Us)(see Fig.36).Different choices of non-linear i ties lead to different computational and

training properties for neurons. The underlying reason for this is that we train neural nets using gradient descent based methods, see Section 4, that require us to take derivatives of the neural input-output function with respect to the weights $w(i)$ and the bias $b(i)$. Notice that the derivatives of the aforementioned non-linearities $\sigma(z)$ have very different properties. The derivative of the perceptron on is zero everywhere except where the input is zero. This discontinuous behavior makes it impossible to train perceptrons using gradient descent. For this reason, until recently the most popular choice of non-linearity was the tanh function or asigmoid/Fermi function. However, this choice of non-linearity has a major drawback. When the input weights become large, as they often do in training, the activation function saturates and the derivative of the output with respect to the weights tends to zero since $\sigma'(z) \rightarrow 0$ for $z \rightarrow \pm\infty$. Such 'vanishing gradients' are a feature of any saturating activation function (top row of Fig. 36), making it harder to train deep networks. In contrast, for a non-saturating activation function such as ReLU or ELUs, the gradients stay finite even for large inputs.

Practice → Generate the synthetic dataset the network will be trained on.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import os
# default font
plt.rcParams['font.size'] = 13

# number of data points (samples)
N=12000
# size of each sample, number of features per sample
L=8
# box size
B=10

!mkdir DATA

%run useful.py
```

```
TYPE=4

# set random seed for reproducibility
np.random.seed(12345)

print(np.random.randint(0,10),np.random.randint(0,10),np.random.randint(0,10))

# generate random 2D data points rescaled by B and initialize label to zero
```

```

x,y = (np.random.random((N,L)))*B, np.zeros(N)

for i in range(N):
    # label data according to a nonlinear function NF defined in useful.py
    y[i] = NF(x[i],B,TYPE)

plot_data(x,y)

fn=filename("data",L,TYPE)
np.savetxt(fn,x)
print(fn)
fn=filename("labels",L,TYPE)
np.savetxt(fn,y,fmt="%d")
print(fn)

```

Theory → Forward and backward pass in pseudocode.

Algorithm: forward + backward pass in pseudocode

```

a[0] = x ①
for l in range(1, L+1):
    z[l] = W[l] @ a[l-1] + b[l] ②
    a[l] = sigma(z[l]) ③
loss = C(a[L], y) ④

delta[L] = grad_a_C * sigma_prime(z[L]) ⑤
for l in range(L, 0, -1):
    grad_W[l] = delta[l] @ a[l-1].T ⑥
    grad_b[l] = delta[l]
    delta[l-1] = (W[l].T @ delta[l]) * \
        sigma_prime(z[l-1]) ⑦

```

- ① Activation of the input layer is just the input vector x .
- ② Pre-activation of layer l : linear combination of the previous layer's activations with the layer's weight matrix W^l and bias b^l .
- ③ Layer- l activation: apply the nonlinearity σ (sigmoid, ReLU, tanh, ...) elementwise.
- ④ Forward pass ends by evaluating the cost / loss on the final activation against the label y .
- ⑤ Output-layer error: gradient of the cost w.r.t. the output activations, scaled by $\sigma'(z^L)$.
- ⑥ Per-layer parameter gradients fall out by an outer product of the local error and the previous layer's activation — eq. (I) in the source.

- ⑦ Backpropagate the error one layer down — eq. (123) — by left-multiplying by $W^{l\top}$ and modulating with $\sigma'(z^{l-1})$.

Practice → Keras model definition, training, evaluation and prediction grid.

5 Keras Tuner

check https://keras.io/keras_tuner/getting_started/ or just pip install keras-tuner -q
install tensorflow

```
import numpy as np

from keras.models import Sequential
from keras.layers import Dense, Dropout

import matplotlib as mpl
import matplotlib.pyplot as plt
# default font
plt.rcParams['font.size'] = 13

from sklearn.preprocessing import StandardScaler

%run useful.py

# training/validation/test fractions
perc_train = 0.7
perc_valid = 0.15
perc_test = 0.15
# check it sums to 1
assert abs(perc_train + perc_valid + perc_test - 1.0) < 1e-12
```

5.1 Read data

```
# Keras works with numpy arrays: just use them from the start

TYPE=4
# number of features per sample
L=8
# span of each component
B=10
x = np.loadtxt(filename("data",L,TYPE), delimiter=' ')
y = np.loadtxt(filename("labels",L,TYPE), delimiter=' ')
```

```

y = y.astype("int")
print(x.shape)
N = len(x)

# dim. of a sample
L = len(x[0])
print(L)

for i in range(5):
    print(x[i],y[i])

N_train = int(perc_train * N)
N_valid = int(perc_valid * N)
N_test = N - N_train - N_valid
print(f'data: {N}\ntrain: {N_train}\nvalid: {N_valid}\ntest: {N_test}')

plot_data(x,y)

def load_data(L, TYPE):
    x = np.loadtxt(filename("data",L,TYPE), delimiter=' ')
    y = np.loadtxt(filename("labels",L,TYPE), delimiter=' ')
    y = y.astype("int")
    N = len(x)
    N_train = int(perc_train * N)
    N_valid = int(perc_valid * N)
    N_test = N - N_train - N_valid
    x_train, y_train = x[:N_train], y[:N_train]
    x_valid, y_valid = x[N_train:N_train+N_valid],
    ↪ y[N_train:N_train+N_valid]
    x_test, y_test = x[N_train+N_valid:], y[N_train+N_valid:]
    print(f'data: {N}\ntrain: {N_train}\nvalid: {N_valid}\ntest: {N_test}')
    x_train_mean = np.mean(x_train, axis=0)
    x_train_std = np.std(x_train, axis=0)
    #print("train stats before rescaling:\nmean value=", x_train_mean,
    ↪ "\nstd. dev.=", x_train_std)

    x_valid_mean = np.mean(x_valid, axis=0)
    x_valid_std = np.std(x_valid, axis=0)
    #print("validation stats before rescaling:\nmean value=", x_valid_mean,
    ↪ "\nstd. dev.=", x_valid_std)

    x_test_mean = np.mean(x_test, axis=0)
    x_test_std = np.std(x_test, axis=0)

```

```

    #print("test stats before rescaling:\nmean value=", x_test_mean, "\nstd.
    ↪ dev.=", x_test_std)

    x_train = Standardize(x_train, x_train_mean, x_train_std)
    x_valid = Standardize(x_valid, x_valid_mean, x_valid_std)
    x_test = Standardize(x_test, x_test_mean, x_test_std)
    return (x_train, y_train), (x_valid, y_valid), (x_test, y_test)

```

5.2 Split data and standardize (fit on train only)

```

def Standardize(x,m,s):
    """
    rescale each component using its mean and standard deviation
    x: data points (numpy array)
    m: mean values (numpy array)
    s: standard deviations (numpy array)
    return: rescaled data points (numpy array)
    """
    N = len(x)
    # assuming len(m)=len(s)=len(x[0])
    mm,ss = np.tile(m,(N,1)), np.tile(s,(N,1))
    return (x-mm)/ss

# split data into train/validation/test sets and check
(x_train, y_train) = (x[0:N_train],y[0:N_train])
(x_valid, y_valid) = (x[N_train:N_train+N_valid],y[N_train:N_train+N_valid])
(x_test, y_test) = (x[N_train+N_valid:],y[N_train+N_valid:])
print("Train:",len(x_train),"\t Validation:",len(x_valid),"\t
    ↪ Test:",len(x_test))

x_train_mean = np.mean(x_train, axis=0)
x_train_std = np.std(x_train, axis=0)
#print("train stats before rescaling:\nmean value=", x_train_mean, "\nstd.
    ↪ dev.=", x_train_std)

x_valid_mean = np.mean(x_valid, axis=0)
x_valid_std = np.std(x_valid, axis=0)
#print("validation stats before rescaling:\nmean value=", x_valid_mean,
    ↪ "\nstd. dev.=", x_valid_std)

x_test_mean = np.mean(x_test, axis=0)

```

```

x_test_std = np.std(x_test, axis=0)
#print("test stats before rescaling:\nmean value=", x_test_mean, "\nstd.
    ↪ dev.=", x_test_std)

x_train = Standardize(x_train, x_train_mean, x_train_std)
x_valid = Standardize(x_valid, x_valid_mean, x_valid_std)
x_test = Standardize(x_test, x_test_mean, x_test_std)
print("after rescaling (train):\nmean value=", x_train.mean(axis=0), "\nstd.
    ↪ dev.=", x_train.std(axis=0))

```

5.3 Define the model with Keras / Tensorflow

```

# reproducibility (?)
np.random.seed(12345)
import tensorflow.random as tf_r
tf_r.set_seed(12345)

# Rule of thumb for the number of parameters in a DNN: it should be smaller
    ↪ than the number of training samples, otherwise we risk overfitting.
# We use case 1
CASE=3

if CASE==1:
    model = Sequential()
    model.add(Dense(L,input_shape=(L,),activation='relu'))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1)) #Helps with overfitting, it prevents
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(1,activation='sigmoid'))
    nepoch=200
# as case 1 but one layer shorter
if CASE==2:
    model = Sequential()
    model.add(Dense(L,input_shape=(L,),activation='relu'))
    model.add(Dense(40,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(1,activation='sigmoid'))

```

```

nepoch=200
#deeper
if CASE==3:
    model = Sequential()
    model.add(Dense(L,input_shape=(L,),activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(10,activation='relu'))
    model.add(Dense(1,activation='sigmoid'))
    nepoch=200

print(model.summary())

```

5.4 Optimization method and cost function

```

model.compile(loss='binary_crossentropy',
              optimizer='RMSprop',
              #optimizer='adam',
              metrics=['accuracy'])

```

```

import visualkeras
visualkeras.graph_view(model , to_file=f'case_{CASE}_NN.png')

```

5.5 Training

```

fit = model.fit(x_train, y_train,
               epochs = nepoch, batch_size = 50,
               validation_data=(x_valid,y_valid),
               verbose=2)

```

```

fig,AX=plt.subplots(1,2,figsize=(12,5.))
ax=AX[0]
ax.plot(fit.history['accuracy'],label="train",c="b",ls="--")
ax.plot(fit.history['val_accuracy'],label="valid.",c="r")
ax.set_xlabel('epoch')
ax.set_ylabel("Accuracy")

```

```
ax.legend()
ax=AX[1]
ax.plot(fit.history['loss'],label="train",c="b",ls="--")
ax.plot(fit.history['val_loss'],label="valid.",c="r")
ax.set_xlabel('epoch')
ax.set_ylabel("Loss")
ax.legend()
```

6 Evaluation on the test set

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Final test loss: {test_loss:.4f}")
print(f"Final test accuracy: {test_acc:.4f}")
```

6.1 Grid to show predictions

```
dX = .1
X1 = np.arange(0,10+dX, dX)
LG = len(X1)
# 2D meshgrid
X, Y = np.meshgrid(X1, X1)
allXY = np.reshape((np.array((X,Y)).T), (LG**2,2))
#create synthetic inputs with L features for each grid point; initialized
↪ randomly in [0, B].
grid = np.random.rand(LG**2,L)*B
#replace first 2 features with the actual 2D coordinates
grid[:,2] = allXY
#standardize using train-set mean/std before prediction
grid_r=Standardize(grid,x_train_mean,x_train_std)

pred = model.predict(grid_r)

fig,AX=plt.subplots(1,3,figsize=(16,5.))
ax=AX[0]
ax.scatter(x[:,0],x[:,1],c=y,s=6)
ax.set_title("data")
ax=AX[1]
ax.pcolormesh(X1,X1,pred.reshape((LG, LG)))
ax.set_title("prediction  $\hat{y}$ ")
ax=AX[2]
pred01=np.copy(pred)
pred01[pred>0.5]=1
pred01[pred<=0.5]=0
ax.pcolormesh(X1,X1,pred01.reshape((LG, LG)))
ax.set_title("where  $\hat{y} > 1/2$ ")
```

```
plt.show()
print(f"remember: these are 2 out of L={L} dimensions, of which L-2 contain
    ↪ random noise")
```

```
# displays the plot
#visualkeras.layered_view(model, to_file='output.png') # saves to disk
```

7 Lesson 4 — Convolutional neural networks on time series

The theory in this chapter is extracted from Mehta *et al.* (Physics Reports 810, 2019), §10 on convolutional neural networks. The full review lives under [2sem/LCPB/source/](#).

Theory → What CNNs are and the architectural building blocks.

7.1 10. Convolutional Neural Networks(CNN S)

One of the core lessons of physics is that we should exploit symmetries and invariances when analyzing physical systems. Properties such as locality and translational invariance are often built directly into the physical laws. Our statistical physics models often directly incorporate everything we know about the physical system being analyzed. For example, we know that in many cases it is sufficient to consider only local couplings in our Hamiltonians, or work directly in momentum space if the system is translationally invariant. This basic idea, tailoring our analysis to exploit additional structure, is a key feature of modern physical theories from general relativity, through gauge theories, to critical phenomena. Like physical systems, many datasets and supervised learning tasks also possess additional symmetries and structure. For instance, consider a supervised learning task where we want to label images from some data set as being pictures of cats or not. Our statistical procedure must first learn features associated with cats. Because a cat is a physical object, we know that these features are likely to be local (groups of neighboring pixels in the two-dimensional image corresponding to whiskers, tails, eyes, etc.). We also know that the cat can be anywhere in the image. Thus, it does not really matter where in the picture these features occur (though relative positions of features likely do matter). This is a manifestation of translational invariance that is built into our supervised learning task. This example makes clear that, like many physical systems, many ML tasks (especially in the context of computer vision and image processing) also possess additional structure, such as locality and translation invariance. The all-to-all coupled neural networks in the previous section fail to exploit this additional structure. For example, consider the image of the digit ‘four’ from the MNIST data set shown in Fig. 26. In the all-to-all coupled neural networks used there, the 28×28 image was considered a one-dimensional vector of size $28^2 = 784$. This clearly throws away lots of the spatial information contained in the image. Not surprisingly, the neural networks community realized these problems and designed a class of neural network architectures, convolutional neural networks or CNNs, that take advantage of this additional structure (locality and translational invariance) (LeCun et al., 1995). Furthermore, what is especially interesting from a physics perspective is the recent finding that these CNN architectures are intimately related to models such as tensor networks (Stoudenmire, 2018; Stoudenmire and Schwab, 2016) and, in particular, MERA-like architectures that are commonly used in physical models for quantum condensed matter systems (Levine et al., 2017).

7.1.1 10.1 The structure of convolutional neural networks

A convolutional neural network is a translation ally invariant neural network that respects locality of the input data. CNN s are the backbone of many modern deep learning applications and here we just give a high-level overview of CNN s that should allow the reader to delve directly into the specialized literature. The reader is also strongly encouraged to consult the excellent, succinct notes for the Stanford CS231n Convolutional Neural Networks class developed by Andrej Karpathy and Fei-Fei Li (<https://cs231n.github.io/>). We have drawn heavily on the pedagogical style of these notes in crafting this section. There are two kinds of basic layers that makeup a CNN: a convolution layer that computes the convolution of the input with a bank of filters (as a mathematical operation, see this practical guide to image kernels: <http://setosa.io/ev/image-kernels/>), and pooling layers that coarse-grain the input while maintaining locality and spatial structure, see Fig. 42. For two-dimensional data, a layer is characterized by three numbers: height H , width W , and depth D .¹² The height and width H, W correspond to the sizes of the two-dimensional spatial (W, H) -plane (in neurons), and the depth D (marked by l, l, l the different colors in Fig. 42) – to the number of filters in that layer. All neurons corresponding to a particular filter have the same parameters (i.e. shared weights and bias). In general, we will be concerned with local spatial filters (often called a receptive field in analogy with neuroscience) that take as inputs a small spatial patch of the previous layer at all depths. For instance, a square filter of size F is a three-dimensional array of size $F \times F \times D$. The convolution consists of running this filter over all locations in the spatial plane. To demonstrate how this works in practice, let us consider the simple example consisting of a one-dimensional input of depth 1, shown in Fig. 43. In this case, a filter of size $F \times 1 \times 1$ can be specified by a vector of weights w of length F . The stride, S , encodes by how many neurons we translate the filter by when performing the convolution. In addition, it is common to pad the input with P zeros (see Fig. 43). For an input of width W , the number of neurons (outputs) in the layer is given by $(W - F + 2P)/S + 1$. We invite the reader to checkout this visualization of the convolution procedure, https://github.com/vdu moulin/conv_arithmetic/blob/master/README.md, for a square input of unit depth. After computing the filter, the output is passed through an on-linearity, a ReLU in Fig. 43. In practice, one often inserts a Batch Norm layer before the non-linearity, cf. Section 9.4.3.

i Fig. 43

Two examples to illustrate a one-dimensional convolutional layer with ReLU non-linearity. Convolutional layer for a spatial filter of size F for a one-dimensional input of width W with stride S and padding P followed by a ReLU non-linearity. width correspond to the sizes of the two-dimensional spatial (W, H) -plane (in neurons), and the depth D (marked by l, l, l the different colors in Fig. 42) – to the number of filters in that layer. All neurons corresponding to a particular filter have the same parameters (i.e. shared weights and bias). In general, we will be concerned with local spatial filters (often called a receptive field in analogy with neuroscience) that take as inputs a small spatial patch of the previous layer at all depths. For instance, a square filter of size F is a three-dimensional array of size $F \times F \times D$. The convolution consists of running this filter over all locations in the spatial plane. To demonstrate how this works in practice, let us consider the simple example consisting of a one-dimensional input of depth 1, shown in Fig. 43. In this case, a filter of size $F \times 1 \times 1$ can be specified by a vector of weights w of length F . The stride, S , encodes by how many neurons we translate the filter by when performing the convolution. In addition, it is common to pad the input with P zeros (see Fig. 43). For an input of width W , the number of neurons (outputs) in the layer is given by $(W - F + 2P)/S + 1$. We invite the reader to checkout this visualization of the convolution procedure, https://github.com/vdu moulin/conv_arithmetic/blob/master/README.md, for a square input of unit depth. After computing the filter, the output is passed through an on-linearity, a ReLU in Fig. 43. In practice, one often inserts a Batch Norm layer before the non-linearity, cf. Section 9.4.3.

These convolutional layers are interspersed with pooling layers that coarse-grain spatial information by performing a sub sampling at each depth. One common pooling operation is the max pool. In a max pool, the spatial dimensions are coarse-grained by replacing a small region (say 2×2 neurons) by a single neuron whose output is the maximum value of the output in the region. In physics, this pooling step is very similar to the decimation step of RG (Isola et al., 2018; Koch-Janusz and Ringel, 2017; Lin et al., 2017; Mehta and Schwab, 2014). This generally reduces the dimension

of outputs. For example, if the region we pool over is 2×2 , then both the height and the width of the output layer will be halved. Generally, pooling operations do not reduce the depth of the convolutional layers because pooling is performed separately at each depth. A simple example of a max-pooling operation is shown in Fig. 44. There are some studies suggesting that pooling might be unnecessary (Springenberg et al., 2014), but pooling layers remain a staple of most CNNs.

i Fig. 44

Illustration of Max Pooling. Illustration of max-pooling over a 2×2 region. Notice that pooling is done at each depth (vertical axis) separately. The number of outputs is halved along each dimension due to this coarse-graining. In a CNN, the convolution and max-pool layers are generally followed by an all-to-all connected layer and a high-level classifier such as a soft-max. This allows us to train CNNs as usual using the back prop algorithm, cf. Section 9.3. From a back prop perspective, CNNs are almost identical to fully connected neural network architectures except with tied parameters.

Apart from introducing additional structure, such as translational invariance and locality, this convolutional structure also has important practical and computational benefits. All neurons at a given layer represent the same filter, and hence can all be described by a single set of weights and biases. This reduces the number of free parameters by a factor of $H \times W$ at each layer. For example, for a layer with $D = 102$ and $H = W = 102$, this gives a reduction in parameters of nearly 10^6 ! This allows for the training of much larger models than would otherwise be possible with fully connected layers. We are familiar with similar phenomena in physics: e.g. in translationally invariant systems we can parameterize all eigenmodes by specifying only their momentum (wave number) and functional form (sin, cos, etc.), while without translation invariance much more information is required.

Practice → Generate the time-series dataset the CNN will classify.

```
import numpy as np
import matplotlib.pyplot as plt

%run plot.py
```

create the random increments of a time series as random walk

DX is the standard deviation, $bias$ is the constant average of the step

```
# random seed for reproducibility
np.random.seed(12345)

# function for the random step, using lambda construction
# int() for cleaner look and for mimicking a detector with finite resolution
jump = lambda drift, stdev: int(np.random.normal(drift, stdev))

for i in range(10):
```

```
print(jump(5,50))
```

7.1.2 Function for the added pattern

to add to part of a time series, over z bins, with amplitude a

```
def pattern(i,z,a):
    return int(a*np.sin((np.pi*i)/z))

## random seed for reproducibility
np.random.seed(12345)

# pattern parameters: Z=nr of steps, A=amplitude
Z=12
A=500

# number of data samples
N=10000
# size of each sample of the timeseries
L=60
# step parameters: introduce small positive bias
DX = 50 # standard deviation of the random step
bias = 5 # bias to make the random walk drift in the positive direction

y = [0] * N # initialize class labels with zeros
x = [[0] * L for i in range(N)] # initialize timeseries with zeros

for i in range(N):
    if i>0:
        x[i][0] = x[i-1][-1] + jump(bias,DX) # first step of the new sample
        ↪ starts from the last point of the previous one

        for j in range(1,L):
            x[i][j] = x[i][j-1] + jump(bias,DX) # random walk with bias

        y[i] = i%3 # assign class labels (0,1,2)
        ##y[i] = random.randint(0,2) # random assingment
        if y[i]>0:
            j0 = np.random.randint(0,L-1-Z) # random starting point for the
            ↪ pattern
            ###print(i,j0,j1)
            sign = 3-2*y[i] # sign of the pattern: + for class 1, - for class 2
            for j in range(Z):
```

```

        x[i][j0+j] += sign*pattern(j,Z,A) # add pattern to the
        ↪ timeseries

for i in range(min(3,N)):
    print(x[i],y[i])

Show_data(x,L,"original data")
print(np.asarray(x).shape,np.asarray(y).shape)

```

7.1.3 Save data on file

```

# command in linux
print(np.asarray(y).shape)
!mkdir DATA
str0 = f'ts_L{L}_Z{Z}_A{A}_DX{DX}_bias{bias}_N{N}.dat'
print(str0)

fname='DATA/x_'+str0
np.savetxt(fname,x,fmt="%d")
fname='DATA/y_'+str0
np.savetxt(fname,y,fmt="%d")

```

Theory → Worked example (CNNs for the 2D Ising model) and the conv → activation → pool block in pseudocode.

7.2 10.2 Example: CNN s for the 2D Ising model

The inclusion of spatial structure in CNN s is an important feature that can be exploited when designing neural networks for studying physical systems. In the accompanying notebook, we used Py torch to implement a simple CNN composed of a single convolutional layer followed by a soft-max layer. Every input data point (i.e. Ising configuration) is shaped as a two-dimensional array. We varied the output depth (i.e. the number of output channels) of the convolutional layer from unity – a single set of weights and one bias – to an output depth of 50 distinct weights and biases. The CNN was then trained using SGD for five epochs using a training set consisting of samples from far in the paramagnetic and ordered phases. The results are shown in Fig. 45. The CNN achieved a 100% accuracy on the test set for all architectures, even for a CNN with depth one. We also checked the performance of the CNN on samples drawn from the near-critical region for temperature sT slightly above and below the critical temperature T . The CNN performed admirably even on these critical c

Algorithm: a single conv $\rightarrow$ activation $\rightarrow$ pool block

```
for f in range(n_filters): ①
    for i, j in spatial_positions(X): ②
        patch = X[i:i+kH, j:j+kW]
        Z[i, j, f] = (W[f] * patch).sum() \
                    + b[f] ③
    A = relu(Z) ④
    for f in range(n_filters):
        for i, j in pool_positions(A):
            P[i, j, f] = A[i:i+pH,
                          j:j+pW, f].max() ⑤
return P
```

- ① Apply each of the n_{filters} kernels independently — this is the channel dimension of the output volume.
- ② Slide the kernel over every valid spatial position (i, j) in the input. Stride / padding are not shown explicitly.
- ③ Per-position output: elementwise product of kernel W^f and the input patch, summed and offset by a bias b^f . This is the discrete cross-correlation that CNNs misname as “convolution”.
- ④ Pointwise nonlinearity — ReLU is the de-facto default, but any of the activations from the DNN chapter applies.
- ⑤ Pooling stage: replace each pooling window with its summary statistic (max here; average pooling uses the mean).

Practice $\rightarrow$ Read and split the data, build the CNN, train and evaluate it.

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.size'] = 14

import keras
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten #, Reshape
from keras.layers import Conv1D, MaxPooling1D, AveragePooling1D

%run plot.py
```

7.2.1 Read data

```
str0 = 'ts_L60_Z12_A500_DX50_bias5_N10000.dat'
fname_x='DATA/x_'+str0
fname_y='DATA/y_'+str0

x = np.loadtxt(fname_x, delimiter=" ", dtype=float)
# number of samples, number of features
N, L = len(x), len(x[0])

Show_data(x, L, "original data")

categ = np.loadtxt(fname_y, dtype=int)
n_class = 3 # y.argmax() - y.argmin() + 1
print('data: ', N)
print(categ.shape)
y = np.zeros((N, n_class))
for i in range(N):
    y[i][categ[i]] = 1. # y contains one-hot encoding
print('y shape: ', y.shape)
```

7.2.2 Split train-validation and rescale

```
# split into training and validation sets
perc_train=0.8
N_train = int(perc_train*N)
x_train = x[:N_train]
print(x_train.shape)
y_train = y[:N_train]
x_val = x[N_train:]
y_val = y[N_train:]
N_val = len(x_val)
print('N_train=', N_train, ' N_val=', N_val, ' L=', L, ' n_class=', n_class)

# remove average value of each sample from its values
xm_train = x_train.mean(axis=1)
std_train = x_train.std(axis=1)
xm_val = x_val.mean(axis=1)
std_val = x_val.std(axis=1)
print('std_train average and mean:', max(std_train), min(std_train),
      ↪ std_train.mean(axis=0))
```

```

print('std_val average and mean:', max(std_val), min(std_val),
      ↪ std_val.mean(axis=0))

# rescale with variance on the training set
for i in range(N_train):
    #x_train[i] = (x_train[i]-xm_train[i])/ std_train[i] #not good,
    ↪ std_train is very variable across samples
    #x_train[i] = (x_train[i]-xm_train[i])/ 400 #crude rescaling, variance
    ↪ should be used
    x_train[i] = (x_train[i]-xm_train[i])/ std_train.mean(axis=0)
for i in range(N_val):
    #x_val[i] = (x_val[i]-xm_val[i])/ std_val[i]
    #x_val[i] = (x_val[i]-xm_val[i])/ 400
    x_val[i] = (x_val[i]-xm_val[i])/ std_val.mean(axis=0)
#

Show_data(x_train,L,"rescaled train data")
Show_data(x_val,L,"rescaled validation data")

#x_train=x_train.astype("float32")
#y_train=y_train.astype("float32")
#x_val=x_val.astype("float32")
#y_val=y_val.astype("float32")

# Keras wants an additional dimension with a 1 at the end
x_train = x_train.reshape(x_train.shape[0], L, 1)
x_val = x_val.reshape(x_val.shape[0], L, 1)
input_shape = (L, 1) # number of features, number of channels (1 for 1D
    ↪ data)
print(y_train.shape, y_val.shape, x_train.shape, x_val.shape)

```

7.3 Convolutional NN

```

# ...LESSON ...
# reproducibility
np.random.seed(12345)
import tensorflow.random as tf_r
tf_r.set_seed(12345)

## from keras import regularizers, initializers

```

```

reg = tf.keras.regularizers.l2(0.2) ## raised to 0.2 the regularization
    ↪ strenghts
#reg = regularizers.l1(0.1)
ini = tf.keras.initializers.RandomNormal(mean=0.0, stddev=0.05,
                                         seed=None)

NCONV = 1 #switch between 1 and 2 CNNs
NF = 5 #number of filters

# challenge: at most 600 tunable parameters
model = Sequential()
#
if NCONV==1:
    # -----2-----
    model.add(Conv1D(filters=NF, kernel_size=11,
                     kernel_initializer=ini,
                     kernel_regularizer=reg,
                     activation='relu',
                     input_shape=input_shape))
    #model.add(MaxPooling1D(3))
    model.add(AveragePooling1D(5))
    model.add(Conv1D(filters=5, kernel_size=7,
                     activation='relu'))
    model.add(Flatten())
    model.add(Dense(12, activation='relu'))
    model.add(Dropout(0.2))
#
if NCONV==2:
    # -----1-----
    model.add(Conv1D(filters=NF, kernel_size=11,
                     kernel_initializer=ini,
                     kernel_regularizer=reg, ##### TRY WITHOUT !
                     activation='relu', input_shape=input_shape))
    #model.add(MaxPooling1D(3))
    model.add(AveragePooling1D(5))
    model.add(Flatten())
    model.add(Dense(9, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(6, activation='relu'))
    model.add(Dropout(0.2))
    #model.add(Dense(10, activation='relu'))
    #model.add(Dropout(0.3))
model.add(Dense(n_class, activation='softmax')) # softmax !

print('----- Model',NCONV,'-----')
```

```

print(model.summary())

# optimizers
# .SGD(lr=0.01, momentum=0.9, nesterov=True) # decay=1e-6,
# .RMSprop()
# .Nadam()
opt = tf.keras.optimizers.Adam()

# compile the model
# categorical_crossentropy, 3 output nodes
model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer=opt, metrics=['accuracy'])

```

```
model.layers
```

```
y.shape
```

```

def Show_weights(model,l=0,label="model"):
    c=['r','y','c','b','m']
    m=['o','s','D','<','>']
    ms=10

    w = model.layers[l].get_weights()[0] # weights
    print(w.shape)
    wT=w.T
    print(wT.shape)
    M=len(wT)
    b = model.layers[l].get_weights()[1] # bias

    fig,AX=plt.subplots(1,2,figsize=(12,4.4))
    ax=AX[0]
    ax.axhline(0, c="k")
    ax.plot((0,))
    for i in range(M):
        ax.plot(wT[i][0], "-",c=c[i],marker=m[i],label=str(i),markersize=ms)
    ax.set_title(label+": filters of layer "+str(l))
    ax.set_xlabel('index')
    ax=AX[1]
    ax.axhline(0, c="k")
    for i in range(M):
        ax.plot((i),(b[i]),c=c[i],marker=m[i],label="filter
↪ "+str(i),markersize=ms)

```

```

ax.set_title(label+' : bias of layer '+str(l))
ax.set_xlabel('filter nr')
ax.set_xticks(np.arange(5))
ax.legend()
plt.show()

```

```

Show_weights(model,0)
Show_weights(model,2)

```

```

# ...LESSON ...
# Hyper-parameters
# with small minibatch it does not converge!!
BATCH_SIZE = 200
EPOCHS = 150

print('----- Model ',NCONV,'-----\nFITTING....')
fit = model.fit(x_train,y_train,batch_size=BATCH_SIZE,
                epochs=EPOCHS,
                validation_data=(x_val, y_val),
                verbose=2, shuffle=True)

```

```

def Show_history(fit):
    fig,AX=plt.subplots(1,2,figsize=(12,5.))
    ax=AX[0]
    ax.plot(fit.history['accuracy'], "b", label="train")
    ax.plot(fit.history['val_accuracy'], "r--", label="valid.")
    ax.plot((0,EPOCHS), (1/3,1/3), ":", c="gray", label="random choice")
    ax.set_xlabel('epoch')
    ax.set_ylabel("Accuracy")
    ax.set_ylim([0, 1])
    ax.legend()
    ax=AX[1]
    ax.plot(fit.history['loss'], "b", label="train")
    ax.plot(fit.history['val_loss'], "r--", label="valid.")
    ax.set_xlabel('epoch')
    ax.set_ylabel("Loss")
    ax.set_ylim([0, 1.05*np.max(fit.history['loss'])])
    ax.legend()
    plt.show()
Show_history(fit)
Show_weights(model,0)
Show_weights(model,2)

```

```

# ...LESSON ...
reg2 = tf.keras.regularizers.l1(0.5)
model2 = keras.models.clone_model(model)
model2.layers[0].kernel_regularizer = reg2
model2.layers[2].kernel_regularizer = reg2
opt2 = tf.keras.optimizers.Adam() # NEW optimizer instance
model2.compile(loss=keras.losses.categorical_crossentropy, optimizer=
    ↪ opt2, metrics=['accuracy'])

BATCH_SIZE2 = 200
EPOCHS2 = 150

fit2 = model2.fit(x_train, y_train, batch_size=BATCH_SIZE2, epochs=EPOCHS2,
    validation_data=(x_val, y_val), verbose=2, shuffle=True)

```

```

Show_history(fit)
Show_history(fit2)
Show_weights(model2, 0, label="model2")
Show_weights(model2, 2, label="model2")

```

```

# confusion table: compare true labels with predicted ones
import pandas as pd
from scipy import stats

import seaborn
from sklearn import metrics

from sklearn.metrics import classification_report
from sklearn import preprocessing

LABELS = ["absent", "positive", "negative"] # from labels y
cmap="GnBu"

def show_confusion_matrix(validations, predictions, label="Model"):

    matrix = metrics.confusion_matrix(validations, predictions)
    plt.figure(figsize=(6, 5))
    seaborn.heatmap(matrix,
        xticklabels=LABELS,
        yticklabels=LABELS,
        annot=True,
        fmt='d',
        linecolor='white',

```

```

        linewidths=1,
        cmap=cmap)
plt.title(label+' : Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

y_pred_val = model.predict(x_val)
y_pred_val2 = model2.predict(x_val)
# Take the class with the highest probability from the val predictions
max_y_pred_val = np.argmax(y_pred_val, axis=1)
max_y_val = np.argmax(y_val, axis=1)
max_y_pred_val2 = np.argmax(y_pred_val2, axis=1)

show_confusion_matrix(max_y_val, max_y_pred_val)
show_confusion_matrix(max_y_val, max_y_pred_val2, label="Model 2")

#print(classification_report(max_y_val, max_y_pred_val))

```


The first reason is statistical. When the learning set is too small, a learning algorithm can typically find several models in the hypothesis space H that all give the same performance on the training data. Provided their predictions are uncorrelated, averaging several models reduces the risk of choosing the wrong hypothesis. The second reason is computational. Many learning algorithms rely on some greedy assumption or local search that may get stuck in local optima. As such, an ensemble made of individual models built from many different starting points may provide a better approximation of the true unknown function than any of the single models. Finally, the third reason is representational. In most cases, for a learning set of finite size, the true function cannot be represented by any of the candidate models in H . By combining several models in an ensemble, it may be possible to expand the space of representable functions and to better model the true function.

The increase in representational power of ensembles can be simply visualized. For example, the classification task shown in Fig. 28 reveals that it is more advantageous to combine a group of simple hypotheses (vertical or horizontal lines) than to utilize a single arbitrary linear classifier. This of course comes with the price of introducing more parameters to our learning procedure. But if the problem itself can never be learned through a simple hypothesis, then there is no reason to avoid applying a more complex model. Since ensemble methods reduce the variance and are often easier to train than a single complex model, they are a powerful way of increasing representational power (also called expressivity in the ML literature). Our analysis also gives several intuitions for how we should construct ensembles. First, we should try to randomize ensemble construction as much as possible to reduce the correlations between predictors in the ensemble. This ensures that our variance will be reduced while minimizing an increase in bias due to correlated errors. Second, the ensembles

Fig. 29

Shown here is the procedure of empirical bootstrap. The goal is to assess the accuracy of a statistical quantity of interest, which in the main text is illustrated as the sample median $M^{(n)}(D)$. We start from a given data set D and bootstrap B size n datasets $D^{(1)}, \dots, D^{(B)}$ called the bootstrap samples. Then we compute the statistical quantity of interest on these bootstrap samples to get the median $M_n^{(k)}$, for $k=1, \dots, B$. These are then used to evaluate the accuracy of $M^{(n)}(D)$ (see also box on Bootstrap in main text). It can be shown that in the $n \rightarrow \infty$ limit the distribution of $M_n^{(k)}$ would be a Gaussian centered around $M^{(n)}(D)$ with variance σ^2 defined by Eq. (102) scales as $1/n$. This will work best for procedures where the error of the predictor is dominated by the variance and not the bias. Thus, these methods are especially well suited for unstable procedures whose results are sensitive to small changes in the training data set.

Finally, we note that although the discussion above was derived in the context of continuous predictors such as regression, the basic intuition behind using ensembles applies equally well to classification tasks. Using an ensemble allows one to reduce the variance by averaging the result of many independent classifiers. As with regression, this procedure works best for unstable predictors for which errors are dominated by variance due to finite sampling rather than bias.

8.0.1 8.2 Bagging

BAGGing, or Bootstrap AGGregation, first introduced by Leo Breiman, is one of the most widely employed and simplest ensemble-inspired methods (Breiman, 1996). Imagine we have a very large data set L that we could partition into M smaller datasets which we label $\{L_1, \dots, L_M\}$. If each partition is sufficiently large to learn a predictor, we can create an $1/M$ ensemble aggregate predictor composed of predictors trained on each subset of the data. For continuous predictors like regression, this is just the average of all the individual predictors: $\hat{g}_A(x) = \frac{1}{M} \sum_{i=1}^M g_i(x)$. (98) For classification tasks where each predictor predicts a class label $j \in \{1, \dots, J\}$, this is just a majority vote of all the predictors, $\hat{g}_A(x) = \arg \max_j \frac{1}{M} \sum_{i=1}^M I[g_i(x) = j]$, (99)

Practice → Generate the synthetic classification dataset and visualise it.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import os
```

```
# number of data
N=5000
# size of each sample
L=4
# box size
B=100
```

```
!mkdir DATA
```

```
def f(x):
    Q=np.heaviside(np.sin(np.pi*(x[1]+2*x[0]))*(2*x[1]-x[0])/2000),0)
    return Q

def filename(s):
    return "DATA/"+s+".dat"

# random seed for reproducibility
np.random.seed(12345)
print(np.random.randint(0,10),np.random.randint(0,10),np.random.randint(0,10),
      ↪ 10))

x,y = (np.random.random((N,L))-0.5)*B, np.zeros(N).astype(int)
for i in range(N):
    # label data according to a nonlinear function "f" using only the first
    ↪ two features
    y[i] = f(x[i,:2])
```

```
plt.figure(figsize = (5,5))
plt.scatter(x[:,0],x[:,1],c=y,s=4,cmap="winter")
plt.show

fn=filename("x_XGB_26")
print(fn)

np.savetxt(fn,x,fmt="%.3f")
fn=filename("y_XGB_26")
np.savetxt(fn,y,fmt="%d")
print(fn)
```

Theory → Gradient boosting in pseudocode.

Algorithm: gradient boosting in pseudocode

```
F = predict_constant(y) ①
for m in range(1, M+1):
    r = -grad_L(y, F) ②
    h = fit_weak_learner(X, r) ③
    gamma = line_search(L, y, F, h) ④
    F = F + eta * gamma * h(X) ⑤
return F ⑥
```

- ① Initialise the ensemble with a constant prediction — typically the mean for regression, the log-odds for classification.
- ② Compute the **negative gradient** of the loss L at the current model F — these are the pseudo-residuals that the next weak learner will be trained to fit.
- ③ Fit a base learner h (a shallow decision tree in XGBoost) to the residuals.
- ④ Optimise the step size γ_m that minimises the loss along the direction h — a one-dimensional line search.
- ⑤ Update the ensemble; η is the **shrinkage** / learning-rate hyper-parameter that trades training speed against overfitting.
- ⑥ After M rounds return the additive ensemble $F_M(x) = \sum_m \eta \gamma_m h_m(x)$.

Practice → Train and compare XGBoost, scikit-learn gradient boosting, and AdaBoost.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
# AdaBoost Algorithm
from sklearn.ensemble import AdaBoostClassifier
```

```

# Gradient Boosting
from sklearn.ensemble import GradientBoostingClassifier
# XGBoost
import xgboost
from xgboost import XGBClassifier
from xgboost import plot_importance, to_graphviz, plot_tree
print("XGBoost version:",xgboost.__version__)

mycmap = "winter"
mpl.rcParams['image.cmap'] = mycmap
plt.rcParams['font.size'] = 13

```

```

np.random.seed(12345)

dname="./DATA/"
str0="_XGB_26.dat"
fname=dname+'x'+str0
fnamey=dname+'y'+str0
x = np.loadtxt(fname, delimiter=" ",dtype=float)
y = np.loadtxt(fnamey)
y = y.astype(int)
N,L = len(x), len(x[0])

N_train = int(0.75*N)
x_train,y_train = x[:N_train],y[:N_train]
x_test,y_test = x[N_train:],y[N_train:]
print(f"N={N}, N_train={N_train}, L={L}")

```

8.0.1 Scatter plot on [i,j] feature plane

```

def scat(ax,x,y,i=0,j=1,s=4,title=""):
    # scatter plot of the data in the plane of features i and j
    ax.scatter(x[:,i],x[:,j],s=s,c=y)
    ax.set_xlabel(f"feature {i}")
    ax.set_ylabel(f"feature {j}")
    ax.set_title(title)

# print the first 5 data points
for n in range(5):
    print(x[n],y[n])

#print feature plots for the training and test sets

```

```

fig,AX = plt.subplots(2,2,figsize=(8.5,8.1))
scat(AX[0,0],x_train,y_train,title="Train")
scat(AX[0,1],x_train,y_train,i=2,j=3,title="Train")
scat(AX[1,0],x_test,y_test,title="Test")
scat(AX[1,1],x_test,y_test,i=2,j=3,title="Test")
fig.tight_layout()
plt.show()

```

```

def classify(clf=GradientBoostingClassifier(),show=False):
    # GradientBoostingClassifier():
    # n_estimators = 100 (default)
    # loss function = deviance(default) used in Logistic Regression
    # XGBClassifier()
    # n_estimators = 100 (default)
    # max_depth = 3 (default?)

    clf.fit(x_train,y_train) # train the model on the training set
    y_hat = clf.predict(x_test) # predict the labels of the test set

    print("errors: {:.2f}%
        ↪ Accuracy={:.3f}".format(100*(1-clf.score(x_test,
        ↪ y_test)),clf.score(x_test, y_test)))
    S=50 # size of the square grid
    dx = 1
    x_seq=np.arange(-S,S+dx,dx) # sequence of points for the grid
    nx = len(x_seq)
    x_plot=np.zeros((nx*nx,L)) # grid of points
    print('shape of x_plot=',x_plot.shape)
    q=0
    for i in range(nx):
        for j in range(nx):
            x_plot[q,:2] = [x_seq[i],x_seq[j]] # only the first two features
            ↪ are used for the artificial grid
            q+=1
    y_plot= clf.predict(x_plot) # predict the labels of the grid points

    fig,AX = plt.subplots(1,2,figsize=(8.2,4))
    scat(AX[0],x_plot[:,],y_plot,s=dx,title="predicted")
    scat(AX[1],x_train[:,],y_train,title="training set")
    fig.tight_layout()
    plt.show()

    if show:

```

```

        dump_list = clf.get_booster().get_dump() # get the list of trees in
        ↪ the model
        num_trees = len(dump_list)
        print("num_trees=", num_trees)

        fig, AX = plt.subplots(2, 1, figsize=(12, 5))
        for i in range(min(2, num_trees)):
            ax = AX[i]
            plot_tree(clf, num_trees=i, ax=ax)
        fig.savefig("DATA/tree-classif.png", dpi=400, pad_inches=0.02)
        plt.show()

```

8.0.2 Classify

If the “objective” hyperparameter is left unspecified, XGBClassifier looks at the data and chooses automatically the loss functions and the evaluation metrics (→ WARNING)

8.1 XGBoost

https://xgboost.readthedocs.io/en/stable/python/python_api.html

```

model = XGBClassifier(seed=1,
                      objective='binary:logistic',
                      importance_type="gain") #weight, cover, ...

    # learning_rate=0.4,
    # reg_lambda=0.001,
    # n_estimators=30)

classify(model, show=True)

```

```

#
    ↪ https://machinelearningmastery.com/feature-importance-and-feature-selection-with-xgboost
# feature importance

print(model.importance_type)
print(model.feature_importances_)
# plot
my_cmap = plt.get_cmap("Reds")
# rescale the feature importances to the range [0.3,1] for better
    ↪ visualization
rescale = lambda y: 0.3 + 0.7 * (y - np.min(y)) / (np.max(y) - np.min(y))
plt.bar(range(len(model.feature_importances_)), model.feature_importances_,

```

```

        color=my_cmap(rescale(model.feature_importances_))
plt.xlabel("feature")
plt.ylabel("importance")
plt.xticks(np.arange(L))
plt.title(model.importance_type)
plt.show()

rescale_r = lambda y: 0.3 + 0.7 * (np.max(y) - y) / (np.max(y) - np.min(y))
plot_importance(model,color=my_cmap(rescale_r(model.feature_importances_))
plt.show()

```

Documentation: https://xgboost.readthedocs.io/en/stable/python/python_api.html

‘weight’: the number of times a feature is used to split the data across all trees.

‘gain’: the average gain across all splits the feature is used in.

‘cover’: the average coverage across all splits the feature is used in.

‘total_gain’: the total gain across all splits the feature is used in.

‘total_cover’: the total coverage across all splits the feature is used in.

```

# if the importance type is "total_gain", then the score is sum of loss
↪ change for each split from all trees.
list_type=["weight","gain","total_gain","cover","total_cover"]
list_col=["#7565F0","k","#666666","#F0A000","gold"]

for i,t in enumerate(list_type):
    feature_imp = model.get_booster().get_score(importance_type=t)
    keys = list(feature_imp.keys())
    values = np.array(list(feature_imp.values()))
    print(i,t,values)
    values= values/np.sum(values)
    plt.bar(np.arange(L)+(i-L/2)/10,
    ↪ values,color=list_col[i],width=0.1,label=t)
plt.xlabel("feature")
plt.ylabel("importance (normalized to 1)")
plt.xticks(np.arange(L))
plt.legend()
plt.show()

```

8.2 Gradient Boosting from Scikit-learn

```
# GradientBoostingClassifier(), use the default parameters  
classify()
```

8.3 AdaBoost

```
classify(clf=AdaBoostClassifier(learning_rate=0.8,n_estimators=150))
```

Part II

Part II: Labs

9 Exercise 2

```
import numpy as np
np.random.seed(12345)
import matplotlib.pyplot as plt
import seaborn as sns
import os
import keras
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten #, Reshape
from keras.layers import Conv1D, MaxPooling1D, AveragePooling1D
from keras.optimizers import Adam, RMSprop, SGD
import keras_tuner as kt
import tensorflow.random as tf_r
tf_r.set_seed(12345)
%run plot.py
```

```
drift,stdev,N,L,Z, pattern_amplitude, pattern_period, cat = 5, 50, 10000,
↪ 60, 1, 500, 12,3
```

9.1 1. Generate data

```
def generate_data(drift,stdev,N,L,Z, pattern_amplitude, pattern_period,
↪ cat):
    os.makedirs("DATA", exist_ok=True)
    x = np.zeros(shape=(N,L,Z))
    print(x.shape)
    y = np.zeros(shape=(N))
    one_hot = np.zeros((N,cat))
    y = (np.indices(y.shape)[0] % cat).astype(int)
    #y= y.reshape(N)
    def pattern(i,z,a):
        return int(a*np.sin((np.pi*i)/z))

    jump = lambda drift, stdev: int(np.random.normal(drift,stdev))
```

```

for k in range(Z):
    d = x[:, :, k] # Shape: (N, L)
    for i in range(N):
        if i>0:
            d[i][0] = d[i-1][-1] + jump(drift,stdev) # first step of the
            ↪ new sample starts from the last point of the previous one

            for j in range(1,L):
                d[i][j] = d[i][j-1] + jump(drift,stdev) # random walk with
                ↪ bias

                #y[i] = i%3 # assign class labels (0,1,2)
                ##y[i] = random.randint(0,2) # random assingment
                if y[i]>0:
                    j0 = np.random.randint(0,L-1-pattern_period) # random
                    ↪ starting point for the pattern
                    ###print(i,j0,j1)
                    sign = cat-(cat-1)*y[i] # sign of the pattern: + for class
                    ↪ 1, - for class 2
                    for j in range(pattern_period):
                        d[i][j0+j] +=
                        ↪ sign*pattern(j,pattern_period,pattern_amplitude) # add pattern to the
                        ↪ timeseries
                    x[:, :, k] = d
            one_hot[np.arange(N), y] = 1.0
            #y = y.reshape(N,cat)
            if Z == 1:
                x = x.reshape(N,L)
                str0 = f'ts_L{L}_Z{pattern_period}_A{pattern_amplitude}_DX{stdev}_bias{
                ↪ drift}_N{N}_D{Z}.numpy'
                fname='DATA/x_'+str0
                np.save(fname,x)
                fname='DATA/y_'+str0
                np.save(fname,one_hot)
            return x, one_hot

```

```

x,y= generate_data(drift,stdev,N,L,Z, pattern_amplitude, pattern_period,
    ↪ cat)

```

```

x.shape, y.shape

```

```

Show_data(x,L,"Original Data")

```

```

def load_data(drift, stdev, N, L, Z, pattern_amplitude, pattern_period,
    ↪ perc_train=0.7, perc_valid=0.15, perc_test=0.15):
    str0 = f'ts_L{L}_Z{pattern_period}_A{pattern_amplitude}_DX{stdev}_bias{
    ↪ drift}_N{N}_D{Z}.npy'

    x = np.load(f'DATA/x_{str0}') # Load once outside the loop
    y = np.load(f'DATA/y_{str0}')

    N = x.shape[0]
    N_train = int(perc_train * N)
    N_valid = int(perc_valid * N)

    # Split 3D arrays directly
    x_train, y_train = x[:N_train], y[:N_train]
    x_valid, y_valid = x[N_train:N_train+N_valid],
    ↪ y[N_train:N_train+N_valid]
    x_test, y_test = x[N_train+N_valid:], y[N_train+N_valid:]

    # Calculate mean/std across the 'L' dimension (axis=1)
    # This preserves the (N, Z) shape for scaling
    def scale(data,Z):
        # 1. Ensure 3D shape: (N, L, Z)
        N, L = data.shape[0], data.shape[1]
        data = data.reshape(N, L, Z)
        for z in range(Z):
            data_m = data[:, :, z].mean(axis=1)
            data_std = data[:, :, z].std(axis=1)

            #print('std_train average and mean:', max(std_train),
            ↪ min(std_train), std_train.mean(axis=0))

        # rescale with variance on the training set
        for i in range(N):
            #x_train[i] = (x_train[i]-xm_train[i])/ std_train[i] #not
            ↪ good, std_train is very variable across samples
            #x_train[i] = (x_train[i]-xm_train[i])/ 400 #crude
            ↪ rescaling, variance should be used
            data[i] = (data[i]-data_m[i])/ data_std.mean(axis=0)

    return data

x_train = scale(x_train, Z)

```

```

x_valid = scale(x_valid, Z)
x_test = scale(x_test, Z)
if Z == 1:
    x_train = x_train.reshape(x_train.shape[0], L,1)
    x_valid = x_valid.reshape(x_valid.shape[0], L,1)
    x_test = x_test.reshape(x_test.shape[0], L,1)
else:
    x_train = x_train.reshape(x_train.shape[0], L, Z)
    x_valid = x_valid.reshape(x_valid.shape[0], L, Z)
    x_test = x_test.reshape(x_test.shape[0], L, Z)
return (x_train, y_train), (x_valid, y_valid), (x_test, y_test)

```

```

(x_train, y_train), (x_valid, y_valid), (x_test, y_test) =
    ↪ load_data(drift,stdev,N,L,Z, pattern_amplitude, pattern_period)
y_train.shape, y_valid.shape, x_train.shape, x_valid.shape

```

```

#print(x_train[:, :, 0]-x_train[:, :, 1])

```

```

Show_data(x_train,L, "Train Data first dimension (Normalized)")
#Show_data(x_train[:, :, 1],L, "Train Data second dimension (Normalized)")

```

9.2 2. Study of the amplitude

```

amplitude_range = np.linspace(1,15,10)
for A in amplitude_range:
    x,y= generate_data(drift,stdev,N,L,Z, stdev*A, pattern_period, cat)

```

```

def base_model(NCONV, NF, input_shape, n_class):
    reg = tf.keras.regularizers.l2(0.2) ## raised to 0.2 the regularization
    ↪ strenghts
    #reg = regularizers.l1(0.1)
    ini = tf.keras.initializers.RandomNormal(mean=0.0, stddev=0.05,
                                              seed=None)

    # challenge: at most 600 tunable parameters
    model = Sequential()
    #
    if NCONV==1:
        # -----2-----

```

```

model.add(Conv1D(filters=NF, kernel_size=11,
                 kernel_initializer=ini,
                 kernel_regularizer=reg,
                 activation='relu',
                 input_shape=input_shape))
#model.add(MaxPooling1D(3))
model.add(AveragePooling1D(5))
model.add(Conv1D(filters=5, kernel_size=7,
                 activation='relu'))
model.add(Flatten())
model.add(Dense(12, activation='relu'))
model.add(Dropout(0.2))

#
if NCONV==2:
    # -----1-----
    model.add(Conv1D(filters=NF, kernel_size=11,
                     kernel_initializer=ini,
                     kernel_regularizer=reg, ##### TRY WITHOUT !
                     activation='relu', input_shape=input_shape))
    #model.add(MaxPooling1D(3))
    model.add(AveragePooling1D(5))
    model.add(Flatten())
    model.add(Dense(9, activation='relu'))
    model.add(Dropout(0.2))
    model.add(Dense(6, activation='relu'))
    model.add(Dropout(0.2))
    #model.add(Dense(10, activation='relu'))
    #model.add(Dropout(0.3))
    model.add(Dense(n_class, activation='softmax')) # softmax !
    model.compile(loss=keras.losses.categorical_crossentropy,
                  optimizer=Adam(), metrics=['accuracy'])
    return model

```

```

base_model2 = base_model(NCONV=1, NF=5, input_shape=(L,Z), n_class=cat)
print('----- Base Model',1,'-----')
print(base_model2.summary())

```

```

def Show_weights(model, l=0, label="Base Model"):
    # Extract weights: shape is (kernel_size, channels, filters)
    weights = model.layers[l].get_weights()[0]
    biases = model.layers[l].get_weights()[1]

    kernel_size, n_channels, n_filters = weights.shape

```

```

print(f"Kernel Size: {kernel_size}, Channels: {n_channels}, Filters:
    ↪ {n_filters}")

fig, AX = plt.subplots(1, 2, figsize=(14, 5))

# --- Plot 0: Filters ---
ax = AX[0]
ax.axhline(0, color="black", linewidth=1, alpha=0.5)

# We loop through each filter
for f in range(n_filters):
    # We loop through each channel (your 2 parallel series)
    for c in range(n_channels):
        # weights[:, c, f] gives the M weights for channel 'c' in filter
        ↪ 'f'
        label_str = f"Filter {f} - Ch {c}" if n_filters < 4 else None
        ax.plot(weights[:, c, f], label=label_str, marker='o',
    ↪ alpha=0.7)

ax.set_title(f'{label}: Weights of Layer {l}')
ax.set_xlabel('Weight Index (M)')
if n_filters < 6: ax.legend() # Only show legend if readable

# --- Plot 1: Biases ---
ax = AX[1]
ax.axhline(0, color="black", linewidth=1, alpha=0.5)
ax.scatter(np.arange(n_filters), biases, s=100, c='red', edgecolors='k')

ax.set_title(f'{label}: Biases of Layer {l}')
ax.set_xlabel('Filter Index')
ax.set_xticks(np.arange(n_filters))

plt.tight_layout()
plt.show()

Show_weights(base_model2,0)
Show_weights(base_model2,2)

```

```

BATCH_SIZE = 200
EPOCHS = 150

```

```

print('----- Base Model ',1,'-----\nFITTING....')
fit_or = base_model2.fit(x_train,y_train,batch_size=BATCH_SIZE,

```

```

epochs=EPOCHS,
validation_data=(x_valid, y_valid),
verbose=2, shuffle=True)

```

```

def Show_history(fit, cat=cat):
    fig,AX=plt.subplots(1,2,figsize=(12,5.))
    ax=AX[0]
    ax.plot(fit.history['accuracy'], "b", label="train")
    ax.plot(fit.history['val_accuracy'], "r--", label="valid.")
    ax.plot((0,EPOCHS), (1/cat, 1/cat), ":", c="gray", label="random choice")
    ax.set_xlabel('epoch')
    ax.set_ylabel("Accuracy")
    ax.set_ylim([0, 1])
    ax.legend()
    ax=AX[1]
    ax.plot(fit.history['loss'], "b", label="train")
    ax.plot(fit.history['val_loss'], "r--", label="valid.")
    ax.set_xlabel('Epoch')
    ax.set_ylabel("Loss")
    ax.set_ylim([0, 1.05*np.max(fit.history['loss'])])
    ax.legend()
    plt.show()
Show_history(fit_or)
Show_weights(base_model2,0)
Show_weights(base_model2,2)

```

```

A_sam_fits = {}
for A in amplitude_range:
    (x_train_al, y_train_al), (x_valid_al, y_valid_al), (x_test_al,
↪ y_test_al) = load_data(drift, stdev, N, L, Z, stdev*A, pattern_period)
    base_model_A1 = base_model(NCONV=1, NF=5, input_shape=(L,Z),
↪ n_class=cat)
    fit = base_model_A1.fit(x_train_al, y_train_al, batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        validation_data=(x_valid_al, y_valid_al),
        verbose=0, shuffle=True) #, callbacks=[early_stop])
    A_sam_fits[A] = (fit, base_model_A1)

```

```

def show_confusion_matrix(validations, predictions, label="Model"):

    matrix = metrics.confusion_matrix(validations, predictions)
    plt.figure(figsize=(6, 5))

```

```

sns.heatmap(matrix,
             xticklabels=["absent", "positive", "negative"],
             yticklabels=["absent", "positive", "negative"],
             annot=True,
             fmt='d',
             linecolor='white',
             linewidths=1,
             cmap="GnBu",
             ax=ax_conf)
plt.title(label+' : Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

```

```

from sklearn import metrics
acc_list = []
loss_list = []
for key, (fit, model) in A_sam_fits.items():
    # Create a 2x2 grid with shared x-axes for the top/bottom pairs
    fig, AX = plt.subplots(2, 3, figsize=(20, 8), sharex='col',
                           gridspec_kw={'height_ratios': [3, 1], 'hspace':
⇨ 0.1})

    fig.suptitle(f"Accuracy and Loss Comparison: Data with {float(key):.2f}
⇨ times the noise ", fontsize=14, y=0.95)

    # Metrics to loop through (Accuracy and Loss)
    metrics1 = ['accuracy', 'loss']
    colors = {'train': 'b', 'valid': 'r'}
    ax_conf = AX[0, -1]
    (x_train, y_train), (x_valid, y_valid), (x_test, y_test) =
⇨ load_data(drift, stdev, N, L, Z, float(key)*stdev, pattern_period)
    y_pred_val = model.predict(x_valid)
    # Take the class with the highest probability from the val predictions
    max_y_pred_val = np.argmax(y_pred_val, axis=1)
    max_y_val = np.argmax(y_valid, axis=1)
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    acc_list.append(test_acc)
    loss_list.append(test_loss)
    matrix = metrics.confusion_matrix(max_y_val, max_y_pred_val)
    sns.heatmap(matrix,
                xticklabels=["absent", "positive", "negative"],

```

```

        yticklabels=["absent","positive","negative"],
        annot=True,
        fmt='d',
        linecolor='white',
        linewidths=1,
        cmap="GnBu",
        ax=ax_conf)
    ax_conf.set_title(f"Confusion Matrix: (Test Acc: {test_acc:.2f}) (Test
↪ Loss: {test_loss:.2f})")
    for i, metric in enumerate(metrics1):
        # --- Upper Panel: Absolute Values ---
        ax_main = AX[0, i]
        # Plot Original (Base) Model
        ax_main.plot(fit_or.history[metric], label="base (train)",
↪ c=colors['train'], alpha=0.5, ls=":")
        ax_main.plot(fit_or.history[f'val_{metric}'], label="base (valid)",
↪ c=colors['valid'], alpha=0.5, ls=":")
        # Plot Current Model
        ax_main.plot(fit.history[metric], label="train", c=colors['train'],
↪ ls="--")
        ax_main.plot(fit.history[f'val_{metric}'], label="valid.",
↪ c=colors['valid'])

        ax_main.set_ylabel(metric.capitalize())
        ax_main.legend(fontsize='small')
        ax_main.grid(True, alpha=0.3)

        # --- Lower Panel: Difference (Model - Base) ---
        ax_diff = AX[1, i]
        diff_train = [m - b for m, b in zip(fit.history[metric],
↪ fit_or.history[metric])]
        diff_valid = [m - b for m, b in zip(fit.history[f'val_{metric}'],
↪ fit_or.history[f'val_{metric}'])]

        ax_diff.plot(diff_train, c=colors['train'], ls="--")
        ax_diff.plot(diff_valid, c=colors['valid'], ls="--")
        ax_diff.axhline(0, color='black', lw=1) # Zero line for reference

        ax_diff.set_ylabel(rf"$\Delta$ {metric.shortname()} if
↪ hasattr(metric, 'shortname') else metric}")
        ax_diff.set_xlabel('epoch')
        ax_diff.grid(True, alpha=0.3)

    plt.tight_layout(rect=[0, 0, 1, 0.93])

```

```

plt.savefig(f"images/n_samples_comparison_{key}.png")
plt.show()

fig, AX = plt.subplots(1, 2, figsize=(20, 8), sharey='col',
                        gridspec_kw={'hspace': 0.1})
sns.barplot(x=np.round(amplitude_range, 2), y=acc_list, ax=AX[0],
            ↪ hue=acc_list)
AX[0].set_xlabel("Signal Amplitude (times the original stdev)")
AX[0].set_ylabel("Test Accuracy")
sns.barplot(x=np.round(amplitude_range, 2), y=loss_list, ax=AX[1],
            ↪ hue=loss_list)
AX[1].set_xlabel("Signal Amplitude (times the original stdev)")
AX[1].set_ylabel("Test loss")
plt.show()

```

```

def build_model(hp,NCONV=1, NF=5, input_shape=(L,Z), n_class=cat,
    ↪ lr_min=1e-5, lr_max=1e-1, dropout_min=0.1, dropout_max=0.4):

    optim_hyp = hp.Choice('optimizer', ['adam', 'rmsprop','sgd'])
    reg = hp.Choice('regularizer', ['l1', 'l2','l1_l2', 'none'])
    dropout_hyp = round(hp.Float('dropout', dropout_min, dropout_max,
    ↪ step=0.1),3)
    lr_hyp = hp.Choice('learning_rate', lr_min, lr_max, sampling='log')

    if reg == 'l1':
        reg = tf.keras.regularizers.l1(dropout_hyp)
    elif reg == 'l2':
        reg = tf.keras.regularizers.l2(dropout_hyp)
    elif reg == 'l1_l2':
        reg = tf.keras.regularizers.l1_l2(l1=dropout_hyp, l2=dropout_hyp)
    else:
        reg = None

    if optim_hyp == 'adam':
        optimizer = Adam(learning_rate=lr_hyp)
    elif optim_hyp == 'rmsprop':
        optimizer = RMSprop(learning_rate=lr_hyp)
    else:
        # Nesterov is a parameter of SGD
        optimizer = SGD(learning_rate=lr_hyp, nesterov=True)

    ini = tf.keras.initializers.RandomNormal(mean=0.0, stddev=0.05,
                                              seed=None)

    # challenge: at most 600 tunable parameters
    model = Sequential()
    #
    if NCONV==1:
        # -----2-----
        model.add(Conv1D(filters=NF, kernel_size=11,

```

```

        kernel_initializer=ini,
        kernel_regularizer=reg,
        activation='relu',
        input_shape=input_shape))
#model.add(MaxPooling1D(3))
model.add(AveragePooling1D(5))
model.add(Conv1D(filters=NF, kernel_size=7,
                 activation='relu'))
model.add(Flatten())
model.add(Dense(12, activation='relu'))
model.add(Dropout(dropout_hyp))
#
if NCONV==2:
    # -----1-----
    model.add(Conv1D(filters=NF, kernel_size=11,
                    kernel_initializer=ini,
                    kernel_regularizer=reg, ##### TRY WITHOUT !
                    activation='relu', input_shape=input_shape))
    #model.add(MaxPooling1D(3))
    model.add(AveragePooling1D(5))
    model.add(Flatten())
    model.add(Dense(9, activation='relu'))
    model.add(Dropout(dropout_hyp))
    model.add(Dense(6, activation='relu'))
    model.add(Dropout(dropout_hyp))
    #model.add(Dense(10, activation='relu'))
    #model.add(Dropout(0.3))
    model.add(Dense(n_class, activation='softmax')) # softmax !
    model.compile(loss=keras.losses.categorical_crossentropy,
                  optimizer=optimizer, metrics=['accuracy'])
    return model

```

```

tuner = kt.RandomSearch(
    hypermodel=build_model,
    objective="val_accuracy",
    max_trials=100, # 3 choices for activation, 3 for optimizer, 3 for
    ↪ dropout, 5 for learning rate
    executions_per_trial=2,
    overwrite=False, # Set to False to keep previous results
    directory="ParameterTuning",
    project_name=f"CNN_Tuning{int(Z)}",
)
tuner.search_space_summary()

```

```
tuner.search(x_train, y_train, epochs=40, validation_data=(x_valid,
    ↪ y_valid))
```

```
import pandas as pd
# 1. Get all trials after the search is done
#trials = tuner.oracle.get_best_trials(num_trials=100)
#
## 2. Parse the results into a list of dictionaries
#results_list = []
#for trial in trials:
#    data = trial.hyperparameters.values.copy() # Gets 'activation',
#    ↪ 'optimizer', dropout, 'learning_rate'
#    data['val_accuracy'] = trial.score          # Gets the resulting
#    ↪ accuracy
#    results_list.append(data)
#df = pd.DataFrame(results_list)
##df['learning_rate'] = round(df['learning_rate'],3)
##df['learning_rate'] = df['learning_rate'].astype(str) # Ensure
#    ↪ learning_rate is float for plotting
#df['groups'] = pd.qcut(df['learning_rate'], q=10)
#print(df['groups'].unique())
```

```
label_dict = {
    'optimizer': 'Optimizer',
    'regularizer': 'Regularizer',
    'dropout': 'Dropout Rate',
    'learning_rate': 'Learning Rate'
}

def results_plot(tuner,num_trials=100,column="regularizer", row="optimizer",
    ↪ hue="dropout", x_var="learning_rate", label_dict=label_dict):
    """
    Plots the results of the hyperparameter tuning using Seaborn's
    ↪ FacetGrid.
    Each subplot corresponds to a combination of 'column' and 'row'
    ↪ hyperparameters,
    with bars colored by 'hue' and grouped by 'x_var'.
    Parameters:
    - tuner: The Keras Tuner object after the search is complete.
    - num_trials: Number of top trials to retrieve and plot.
    - column: Hyperparameter to use for column facets (e.g., 'regularizer').
    - row: Hyperparameter to use for row facets (e.g., 'optimizer').
    - hue: Hyperparameter to use for bar colors (e.g., 'dropout').
```

```

- x_var: Hyperparameter to use for x-axis grouping (e.g.,
↳ 'learning_rate').
"""
trials = tuner.oracle.get_best_trials(num_trials=num_trials)

# 2. Parse the results into a list of dictionaries
results_list = []
for trial in trials:
    data = trial.hyperparameters.values.copy() # Gets 'activation',
↳ 'optimizer', dropout, 'learning_rate'
    data['val_accuracy'] = trial.score # Gets the resulting
↳ accuracy
    results_list.append(data)
df = pd.DataFrame(results_list)
g = sns.FacetGrid(df, col=column, row=row, margin_titles=True, height=4,
↳ hue=hue, palette="Set2")

def annotate(data, **kws):
    # Get the specific dropout value for this group

    unique_vals = sorted(df[hue].unique())
    pos = np.linspace(1, 0.5, num=len(unique_vals))
    offset_map = dict(zip(unique_vals, pos))

    dropout_val = kws.get('label')
    y_pos = offset_map.get(dropout_val, 0.5)
    dropout_val = kws.get('label', 'N/A')
    n = data['val_accuracy'].max()
    ax = plt.gca()

    ax.text(0.85, y_pos, f"Score: {n:.3f}",
            transform=ax.transAxes,
            fontsize=9,
            color=kws.get('color', 'black'))

if df[x_var].unique().shape[0] <= 10: # Only annotate if there are 10
↳ or fewer unique x values

    g.map_dataframe(
        sns.barplot,
        x=x_var,
        y="val_accuracy",

```

```

        errorbar=None,
        orient="y"

    )
else:
    df[f'groups_of_{x_var}'] = pd.qcut(df[x_var], q=10)
    g.map_dataframe(
        sns.barplot,
        x="val_accuracy",          # Numbers go on X
        y=f'groups_of_{x_var}',    # Categories go on Y
        errorbar=None,
        orient="h"                # Use "h" for clarity, or leave it to
    ↪ auto-detect
    )

    g.map_dataframe(annotate)
    # 1. Adjust the plot boundaries to prevent legend/title overlap
    g.fig.subplots_adjust(right=0.8, top=0.85, wspace=0.3)

    # 2. Refine the margin titles' styling to make them less intrusive
    # This sets the font weight and padding for the facet labels
    g.set_titles(col_template="{col_name}", row_template="{row_name}",
    ↪ fontweight='bold')

    # 3. Optional: Move the actual margin title objects if they still
    ↪ overlap
    # We use g.fig.texts because margin_titles=True places them as
    ↪ figure-level text
    for txt in g.fig.texts:
        # Check if it's a row title (typically positioned on the right)

        if txt.get_y() > 0.1:
            txt.set_x(txt.get_y() + 1) # Nudge it right
    g.set(xlim=(0, 1.05))
    g.add_legend()
    plt.show()

```

```

results_plot(tuner,num_trials=100,column="regularizer", row="optimizer",
    ↪ hue="dropout", x_var="learning_rate")

```

11 LCPB 25-26 Exercise 3, XGBoost (XGB)

Last Name	First Name	Student Number
Miguel	Ordén	2183906
Adrián	Orellana	218002
Carlos	Hita	2184935
Álvaro	Méndez	2185566

11.1 Importing the libraries

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import pandas as pd
import os
from sklearn import metrics
from sklearn import model_selection
# AdaBoost Algorithm
from sklearn.ensemble import AdaBoostClassifier
# Gradient Boosting
from sklearn.ensemble import GradientBoostingClassifier
# XGBoost
import xgboost
from xgboost import XGBClassifier
from xgboost import plot_importance, to_graphviz, plot_tree
print("XGBoost version:", xgboost.__version__)
import keras_tuner as kt
mycmap = "winter"
mpl.rcParams['image.cmap'] = mycmap
plt.rcParams['font.size'] = 13
```

11.2 Generating and loading the data

```
N=5000
# size of each sample
L=4
# box size
B=100

def f(x):
    Q=np.heaviside(np.sin(np.pi*(x[1]+2*x[0]))*(2*x[1]-x[0])/2000),0)
    return Q

def generate_data(N,L,B,f,s):
    os.makedirs("DATA",exist_ok=True)
    print(np.random.randint(0,10),np.random.randint(0,10),
          ↪ np.random.randint(0,10))
    def filename(s):
        return "DATA/"+s+".dat"
    x,y = (np.random.random((N,L))-0.5)*B, np.zeros(N).astype(int)
    for i in range(N):
        # label data according to a nonlinear function "f" using only the
        ↪ first two features
        y[i] = f(x[i,:2])

    #plt.figure(figsize = (5,5))
    #plt.scatter(x[:,0],x[:,1],c=y,s=4,cmap="winter")
    #plt.show

    fn=filename(f"x_{s}")

    np.savetxt(fn,x,fmt="%.3f")
    fn=filename(f"y_{s}")
    np.savetxt(fn,y,fmt="%d")
    return x,y

def load_data(s,fac=0.75):
    def filename(s):
        return "DATA/"+s+".dat"
    x = np.loadtxt(filename(f"x_{s}"), delimiter=" ",dtype=float)
    y = np.loadtxt(filename(f"y_{s}"))
    y = y.astype(int)
    N,L = len(x), len(x[0])
```

```

N_train = int(fac*N)
x_train,y_train = x[:N_train],y[:N_train]
x_test,y_test = x[N_train:],y[N_train:]
return x_train,y_train,x_test,y_test, N,L

x, y = generate_data(N,L,B,f,"XGB_26")
x_train,y_train,x_test,y_test, N,L = load_data("XGB_26")
print("N =",N,"L =",L)
print("x_train.shape =",x_train.shape,"y_train.shape =",y_train.shape)
print("x_test.shape =",x_test.shape,"y_test.shape =",y_test.shape)

```

11.3 Exercise 1: Grid search for different parameters (Model complexity)

Now we will use keras tuner to explore the parameter space to see which combinations of parameters gives us the best performance. In this case we explore three different models (as suggested in the course notebook):

- XGBClassifier
- GradientBoostingClassifier
- AdaBoostClassifier

```

import inspect
import keras_tuner as kt
from sklearn import metrics, model_selection

# Since we will try different models and we don't know which models support
↳ which parameters, we create a flexible function that checks the model's
↳ supported parameters before trying to sample them. This way, we can
↳ maintain a single "Master Config" and not worry about compatibility
↳ issues.
def build_model(hp, clf, params_config):
    sampled_params = {}

    # 1. Create a dummy instance to see ALL supported parameters
    # This is much more reliable than inspect.signature for XGBoost
    try:
        temp_instance = clf()
        all_supported_params = temp_instance.get_params().keys()
    except Exception:
        # Fallback to signature if get_params fails
        all_supported_params = inspect.signature(clf).parameters.keys()

```

```

# 2. Loop through our desired tuning config
for p_name, p_settings in params_config.items():
    # Check if the model actually supports this parameter
    if p_name in all_supported_params:
        if p_settings['type'] == 'Int':
            sampled_params[p_name] = hp.Int(
                p_name, p_settings['min'], p_settings['max'],
                ↪ step=p_settings.get('step', 1)
            )
        elif p_settings['type'] == 'Float':
            sampled_params[p_name] = hp.Float(
                p_name, p_settings['min'], p_settings['max'],
                ↪ sampling=p_settings.get('sampling', 'linear')
            )
        elif p_settings['type'] == 'Categorical':
            sampled_params[p_name] = hp.Choice(
                p_name, p_settings['values']
            )

    # This print will now show the full list!
    print(f"Building {clf.__name__} with: {list(sampled_params.keys())}")

    return clf(**sampled_params)

# 2. Define the "Master Config" for all possible parameters
tuning_config = {
    'objective' : {'type': 'Categorical', 'values': ['binary:logistic']},
    'importance_type' : {'type': 'Categorical', 'values': ['gain']},
    'n_estimators': {'type': 'Int', 'min': 50, 'max': 300, 'step': 50},
    'max_depth': {'type': 'Int', 'min': 3, 'max': 10},
    'learning_rate': {'type': 'Categorical', 'values': [1e-5, 1e-4, 1e-3,
        ↪ 1e-2, 1e-1]},
    'reg_lambda': {'type': 'Categorical', 'values': [0.001, 0.01, 0.1, 0.5,
        ↪ 1.0, 1.5, 2.0]},
}

# 3. The Optimized Loop
n_trials = 100
tunners = {}
classifiers = {
    "XGBClassifier": XGBClassifier,

```

```

    "GradientBoostingClassifier": GradientBoostingClassifier,
    "AdaBoostClassifier": AdaBoostClassifier,
}

for name, clf in classifiers.items():
    print(f"Starting tuning for {name}...")
    tuner = kt.tuners.SklearnTuner(
        oracle=kt.oracles.BayesianOptimizationOracle(
            objective=kt.Objective('score', 'max'),
            max_trials=n_trials
        ),
        # Pass the classifier and the full config; the function will filter
        ↳ them
        hypermodel=lambda hp, c=clf: build_model(hp, c, tuning_config),
        scoring=metrics.make_scorer(metrics.accuracy_score),
        cv=model_selection.StratifiedKFold(5),
        directory='ParameterTuning',
        project_name=name+"_cat_lambda",
        overwrite=False
    )

    tuner.search(x_train, y_train)
    tuners[name] = tuner

```

```

import matplotlib.cm as cm
import matplotlib.colors as mcolors

label_dict = {
    'optimizer': 'Optimizer',
    'regularizer': 'Regularizer',
    'dropout': 'Dropout Rate',
    'learning_rate': 'Learning Rate'
}

def results_plot(tuner, num_trials=None, column="regularizer",
    ↳ row="optimizer",
    hue="dropout", x_var="learning_rate",
    ↳ label_dict=label_dict,
    sig_figs=3, title=None):

    if num_trials is None:
        num_trials = len(tuner.oracle.trials)
    trials = tuner.oracle.get_best_trials(num_trials=num_trials)

```

```

if title is None:
    title = f"Parameter search for a model with
↳ {len(tuner.oracle.trials)} trials"

results_list = []
for trial in trials:
    data = trial.hyperparameters.values.copy()
    data['val_accuracy'] = round(trial.score, 3) if trial.score is not
↳ None else 0
    results_list.append(data)

df = pd.DataFrame(results_list)

# 1. Handling Missing Columns (Safety for different models)
for col in [column, row, hue, x_var]:
    if col is not None and col not in df.columns:
        df[col] = "N/A"
global_max = df['val_accuracy'].max()
# 2. Rounding Logic
def round_df_to_sig_figs(df_in, n_sigs):
    df_numeric = df_in.select_dtypes(include=[np.number])
    vals = df_numeric.values
    mask = (vals != 0) & (~np.isnan(vals))
    log_10 = np.floor(np.log10(np.abs(np.where(mask, vals, 1))))
    decimals = (n_sigs - 1) - log_10
    rounded = np.where(mask, np.round(vals * 10**decimals) /
↳ 10**decimals, vals)
    df_in[df_numeric.columns] = rounded
    return df_in

df = round_df_to_sig_figs(df, sig_figs)

# 3. FacetGrid Setup with Hue Logic
colormap = None
norm = None

if hue is not None and hue in df.columns and df[hue].nunique() > 1:
    if df[hue].nunique() <= 9:
        # Discrete Case
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↳ height=4, hue=hue, palette="Set2")
    else:
        # Continuous Case (Fixing the Palette/Hue swap)

```

```

        norm = mcolors.Normalize(vmin=df[hue].min(), vmax=df[hue].max())
        colormap = cm.get_cmap("cubehelix")
        unique_hues = sorted(df[hue].unique())
        palette_dict = {val: mcolors.to_hex(colormap(norm(val))) for val
↪ in unique_hues}

        # FIXED: hue is the COLUMN NAME, palette is the DICTIONARY
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↪ height=4, hue=hue, palette=palette_dict)
    else:
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↪ height=4)

# 4. Annotate Function

def annotate_scores(data, **kws):
    if data.empty:
        return

    ax = plt.gca()
    # 1. Identify which variable is on the y-axis
    # In your logic, this is either x_var or the 'groups_of' binned
    ↪ column
    y_var_name = (x_var if df[x_var].unique().shape[0] <= 10 else
↪ f"groups_of_{x_var}")

    # 2. Find the row with the maximum accuracy in THIS facet
    best_row_idx = data['val_accuracy'].argmax()
    max_score = data['val_accuracy'].iloc[best_row_idx]
    best_y_label = data[y_var_name].iloc[best_row_idx]

    # 3. Get the vertical position of that specific category
    # Matplotlib's categorical axes use integers 0, 1, 2... for
    ↪ positions

    # We find where our 'best_y_label' sits in the current y-ticks
    # A more robust way is using the category index from the plot
    y_pos = None
    ticks = ax.get_yticks()
    labels = [t.get_text() for t in ax.get_yticklabels()]

    try:
        # Match the label string to find the correct tick index

```

```

        label_idx = labels.index(str(best_y_label))
        y_pos = ticks[label_idx]
    except (ValueError, IndexError):
        # Fallback to middle if label matching fails
        y_pos = 0.5
        transform = ax.transAxes
    else:
        # If we found the tick, we use Data coordinates for Y and Axes
        #   ↪ for X
        transform = ax.get_yaxis_transform()

    #ax.axvline(x=y_pos, color="black", linestyle="--")
    # 4. Draw the text inside the bar or at the end of it
    if max_score < global_max:
        ax.text(0.95, y_pos, f"Score: {max_score:.3f}",
                transform=transform,
                fontsize=9, fontweight='bold',
                ha='right', va='center',
                color=kws.get('color', 'black'),
                bbox=dict(facecolor='white', alpha=0.8,
        ↪ edgecolor='none'))
    else:
        ax.text(0.95, y_pos, f"Score: {max_score:.3f}",
                transform=transform,
                fontsize=9, fontweight='bold',
                ha='right', va='center',
                color=kws.get('color', 'black'),
                bbox=dict(facecolor='white', alpha=0.8, edgecolor='black',
        ↪ boxstyle='round,pad=0.2'))

    # 5. Plotting
    y_axis_var = x_var if x_var in df.columns else df.columns[0]

    if df[x_var].unique().shape[0] <= 10: # Only annotate if there are 10
        ↪ or fewer unique x values

        g.map_dataframe(
            sns.barplot,
            x="val_accuracy",
            y=y_axis_var,
            errorbar=None,
            orient="h"

        )

```

```

else:
    df[f'groups_of_{x_var}'] = pd.cut(df[x_var], bins=10,
    ↪ duplicates='drop')
    g.map_dataframe(
        sns.barplot,
        x="val_accuracy",          # Numbers go on X
        y=f'groups_of_{x_var}',    # Categories go on Y
        errorbar=None,
        orient="h"                 # Use "h" for clarity, or leave it to
    ↪ auto-detect
    )
    g.map_dataframe(annotate_scores)

# 6. Final Layout and Margin Title Fixes
# Push right margin to make room for titles
g.fig.subplots_adjust(right=0.8, top=0.85, wspace=0.4)
g.fig.suptitle(title, fontsize=14, fontweight='bold')
g.set_titles(col_template=f"{column}="+f"{col_name}",
    ↪ row_template=f"{row}="+f"{row_name}", fontweight='bold')

# FIX: Reposition Row Titles to the far right
for txt in g.fig.texts:
    if txt.get_rotation() == -90:
        txt.set_x(0.92) # Absolute figure coordinate
        txt.set_va('center')

# Add Colorbar for Continuous Hue
if colormap and norm:
    sm = plt.cm.ScalarMappable(cmap=colormap, norm=norm)
    sm.set_array([])
    cbar_ax = g.fig.add_axes([0.85, 0.15, 0.02, 0.6]) # [left, bottom,
    ↪ width, height]
    g.fig.colorbar(sm, cax=cbar_ax, label=label_dict.get(hue, hue))
else:
    g.add_legend()
g.set(xlim=(0, 1.1))
plt.show()

```

```

results_plot(tunners['XGBClassifier'], column="n_estimators",
    ↪ row="max_depth", hue="reg_lambda",
    ↪ x_var="learning_rate", title=f"Parameter search for XGBClassifier with
    ↪ {len(tunners['XGBClassifier'].oracle.trials)} trials")

```

```

results_plot(tunners['AdaBoostClassifier'],column="n_estimators", row=None,
    ↪ hue=None, x_var="learning_rate",title=f"Parameter search for
    ↪ AdaBoostClassifier with
    ↪ {len(tunners['AdaBoostClassifier'].oracle.trials)} trials")

```

```

results_plot(tunners['GradientBoostingClassifier'],column="n_estimators",
    ↪ row=None, hue="max_depth", x_var="learning_rate",title=f"Parameter
    ↪ search for GradientBoostingClassifier with
    ↪ {len(tunners['GradientBoostingClassifier'].oracle.trials)} trials")

```

11.4 Select different features

```

def scat(ax, x, y, i=0, j=1, s=4, title="", xlabel=None, ylabel=None):
    """
    Scatter plot of data in the plane of features i and j.
    """
    # Use c=y to color points by their class label
    ax.scatter(x[:, i], x[:, j], s=s, c=y, cmap='viridis', alpha=0.7)
    ax.set_xlabel(xlabel if xlabel else f"Feature {i}")
    ax.set_ylabel(ylabel if ylabel else f"Feature {j}")
    ax.set_title(title)

def classify(clf, x_train, y_train, x_test, y_test, Lp, grid_range=50, dx=1,
    ↪ show=False, tree=False,name = "", params=""):
    """
    Trains a classifier, predicts on a grid to show decision boundaries,
    and compares it to the training set.
    """
    # 1. Train and Evaluate
    clf.fit(x_train, y_train)
    accuracy = clf.score(x_test, y_test)
    error_rate = 100 * (1 - accuracy)

    print(f"Errors: {error_rate:.2f}% | Accuracy: {accuracy:.3f}")

    # 2. Create Artificial Grid for Decision Boundary Visualization
    # We create a 2D grid for the first two features
    s_points = np.arange(-grid_range, grid_range + dx, dx)
    xx, yy = np.meshgrid(s_points, s_points)

    # Flatten grid and prepare x_plot with Lp features (filling rest with
    ↪ zeros)

```

```

grid_coords = np.c_[xx.ravel(), yy.ravel()]
x_plot = np.zeros((grid_coords.shape[0], Lp))
x_plot[:, :2] = grid_coords

print(f'Shape of x_plot: {x_plot.shape}')
if show:
    # 3. Predict on the grid
    y_plot = clf.predict(x_plot)

    # 4. Visualization
    fig, AX = plt.subplots(1, 2, figsize=(10, 4))

    # Left: Decision Boundary (Predicted)
    scat(AX[0], x_plot, y_plot, i=0, j=1, s=dx*2, title="Decision
↪ Boundary (Predicted)")

    # Right: Actual Training Data
    scat(AX[1], x_train, y_train, i=0, j=1, s=10, title="Actual Training
↪ Set")

    fig.tight_layout()
    fig.suptitle(f"Results for model {name} with params \n {params}",
↪ y=1.2)
    plt.show()

if tree:
    dump_list = clf.get_booster().get_dump() # get the list of trees in
↪ the model
    num_trees = len(dump_list)
    print("num_trees=", num_trees)

    fig, AX = plt.subplots(2, 1, figsize=(12, 5))
    for i in range(min(2, num_trees)):
        ax=AX[i]
        plot_tree(clf, num_trees=i, ax=ax)
    fig.savefig("DATA/tree-classif.png", dpi=400, pad_inches=0.02)
    plt.show()

```

```

def plot_feature_importance(model, feature_names=None, title=None,
↪ ascending=False):
    """
    Plots feature importances for any scikit-learn or XGBoost model.
    - ascending: If True, smaller importances get darker colors.

```

```

"""
importances = model.feature_importances_
num_features = len(importances)

# Use provided feature names or default to indices
if feature_names is None:
    feature_names = [f"F{i}" for i in range(num_features)]

# 1. Setup Colors using your Rescale logic
my_cmap = plt.get_cmap("Reds")
min_imp, max_imp = np.min(importances), np.max(importances)

# Avoid division by zero if all importances are equal
diff = (max_imp - min_imp) if max_imp != min_imp else 1

if not ascending:
    # Standard: Higher importance = Darker Red
    rescale = lambda y: 0.3 + 0.7 * (y - min_imp) / diff
else:
    # Reversed: Lower importance = Darker Red
    rescale = lambda y: 0.3 + 0.7 * (max_imp - y) / diff

# 2. Create Plot
plt.figure(figsize=(8, 4))
bars = plt.bar(range(num_features), importances,
               color=my_cmap(rescale(importances)),
               edgecolor='black', linewidth=0.5)

# 3. Styling
plt.xlabel("Features")
plt.ylabel("Importance Score")

# Determine the type of importance used (XGBoost specific)
imp_type = getattr(model, 'importance_type', 'Gini/MDI')
plt.title(title if title else f"Feature Importance ({imp_type})")

plt.xticks(range(num_features), feature_names, rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

def plot_xgb_multi_importance(model, feature_names=None):

```

```

"""
Compares different XGBoost importance types.
Handles cases where booster.feature_names is None.
"""
list_type = ["weight", "gain", "total_gain", "cover", "total_cover"]
list_col = ["#7565F0", "k", "#666666", "#F0A000", "gold"]

booster = model.get_booster()

# FIX: If booster.feature_names is None, generate generic names
# based on the model's expected input shape
if booster.feature_names is not None:
    all_features = booster.feature_names
else:
    # Fallback: get count from the actual feature_importances_ array
    num_features = len(model.feature_importances_)
    all_features = [f"f{i}" for i in range(num_features)]

L = len(all_features)

# Use the names passed in the function call for the X-axis labels
if feature_names is None:
    display_names = all_features
else:
    display_names = feature_names

plt.figure(figsize=(12, 6))

for i, t in enumerate(list_type):
    score_dict = booster.get_score(importance_type=t)

    # XGBoost get_score keys are strings like 'f0', 'f1' regardless of
    # ↪ names
    # so we map the scores carefully
    values = np.array([score_dict.get(f, 0) for f in all_features])

    total = np.sum(values)
    if total > 0:
        values = values / total

    width = 0.12
    offset = (i - len(list_type)/2) * width + (width/2)

    plt.bar(np.arange(L) + offset, values,

```

```

        color=list_col[i], width=width, label=t, alpha=0.8)

plt.xlabel("Features", fontweight='bold')
plt.ylabel("Importance (Normalized to 1)", fontweight='bold')
plt.title("XGBoost: Comparison of Importance Metrics", fontsize=14)

# Use our display_names for the ticks
plt.xticks(np.arange(L), display_names, rotation=45)
plt.legend(title="Importance Type", bbox_to_anchor=(1.05, 1), loc='upper
↳ left')
plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```

```

from itertools import combinations
Lp = 3
features = list(combinations(range(L), Lp))
for (n, tuner), ( _, clf) in zip(tunners.items(), classifiers.items()):
    best_params = tuner.oracle.get_best_trials(num_trials=n_trials)[0]
↳ ].hyperparameters.values.copy()
    for f in features:
        x_train_subset = x_train[:, f]
        x_test_subset = x_test[:, f]
        print(f"Prediction for model {n} trained with the features {f}")
        model = clf(**best_params)
        classify(
            clf=model,
            x_train=x_train_subset,
            y_train=y_train,
            x_test=x_test_subset,
            y_test=y_test,
            Lp=Lp,
            grid_range=50,
            name=n+f" with features {f}",
            params = best_params,
            show=True,
        )
        plot_feature_importance(model, feature_names=[f"F{idx}" for idx in
↳ f], title=f"Feature Importance for {n} with features {f}")
        if n == "XGBClassifier":
            plot_xgb_multi_importance(model, feature_names=[f"F{idx}" for
↳ idx in f])

```

11.5 compare it to FFNN with cross validation

```
from keras.models import Sequential
from keras.layers import Dense, Dropout

def FFNN(x_train,x_test,y_train,y_test,epochs=200):
    model = Sequential()
    model.add(Dense(L,input_shape=(L,),activation='relu'))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1)) #Helps with overfitting, it prevents
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(1,activation='sigmoid'))

    model.compile(loss='binary_crossentropy',
                  optimizer='RMSprop',
                  #optimizer='adam',
                  metrics=['accuracy'])
    model.fit(x_train, y_train,
              epochs = epochs, batch_size = 50,
              validation_data=(x_test,y_test),
              verbose=2)
    accuracy, loss = model.evaluate(x_test, y_test, verbose=0)

    return accuracy, loss
```

```
from sklearn.model_selection import KFold
k=5
fac = np.linspace(0.1, 0.95, 20)

results = {}
for (n, tuner),( _, clf) in zip(tunners.items(),clasifiers.items()):

    accuracies_fnn = []
    losses_fnn = []
    for f in fac:
        print(f"Training {n} with training size factor: {f:.2f}")
        total_acc = 0
        total_loss = 0
```

```

x_train, y_train, x_test, y_test, N, L = load_data("XGB_26", fac=f)
for train_index, test_index in KFold(n_splits=k, shuffle=True,
    ↪ random_state=1234).split(x_train):

    x_tr_fold, x_val_fold = x_train[train_index],
    ↪ x_train[test_index]
    y_tr_fold, y_val_fold = y_train[train_index],
    ↪ y_train[test_index]
    best_params = tuner.oracle.get_best_trials(num_trials=
    ↪ n_trials)[0].hyperparameters.values.copy()
    model = clf(**best_params)
    model.fit(x_tr_fold, y_tr_fold)
    accuracy = model.score(x_val_fold, y_val_fold)
    error_rate = (1 - accuracy)
    total_acc += accuracy
    total_loss += error_rate

    avg_acc = total_acc / k
    avg_loss = total_loss / k
    accuracies_fnn.append(avg_acc)
    losses_fnn.append(avg_loss)
    results[n] = (accuracies_fnn, losses_fnn)

accuracies_fnn = []
losses_fnn = []
for f in fac:
    print(f"Training FFNN with training size factor: {f:.2f}")
    x_train, y_train, x_test, y_test, N, L = load_data("XGB_26", fac=f)

    total_acc = 0
    total_loss = 0

    for train_index, test_index in KFold(n_splits=k, shuffle=True,
        ↪ random_state=1234).split(x_train):
        x_tr_fold, x_val_fold = x_train[train_index], x_train[test_index]
        y_tr_fold, y_val_fold = y_train[train_index], y_train[test_index]
        acc, loss = FFNN(x_tr_fold, x_val_fold, y_tr_fold, y_val_fold,
        ↪ epochs=10)
        print(f"Accuracy: {acc:.4f}, Loss: {loss:.4f}\n")
        total_acc += acc
        total_loss += loss
    avg_acc = total_acc / k
    avg_loss = total_loss / k

```

```

    accuracies_fnn.append(avg_acc)
    losses_fnn.append(avg_loss)
results["FNN"] = (accuracies_fnn, losses_fnn)

```

```

import matplotlib.pyplot as plt

fig, AX = plt.subplots(2, 2, figsize=(20, 8), sharex='col',
                        gridspec_kw={'height_ratios': [3, 1], 'hspace':
↪ 0.1})
colors = {
    'XGBClassifier': 'purple',
    'GradientBoostingClassifier': 'green',
    'AdaBoostClassifier': 'r'
}

base_acc = results["FNN"][0]
base_loss = results["FNN"][1]

# Plot Original (Base) Model
AX[0,0].plot(fac, base_acc, label="FFNN", c='b', alpha=0.5, ls="--")
AX[0,1].plot(fac, base_loss, label="FFNN", c='b', alpha=0.5, ls="--")
for key, (accc, loss) in results.items():
    # Create a 2x2 grid with shared x-axes for the top/bottom pairs
    if key == "FNN":
        continue
    #print(f"Plotting results for {key}...")
    #print(f"Accuracies: {accc}")
    #print(f"Losses: {loss}")

    fig.suptitle("Accuracy and Loss Comparison as a function of the train
↪ size ", fontsize=14, y=0.95)

    # Metrics to loop through (Accuracy and Loss)

    best_params = tuner.oracle.get_best_trials(num_trials=n_trials)[0]
    ↪ .hyperparameters.values.copy()

    ax_main = AX[0]
    # Plot Current Model
    AX[0,0].plot(fac, accc, label=key, c=colors[key], ls="--")

```

```

AX[0,1].plot(fac,loss, label=key, c=colors[key], ls="--")

AX[0,0].set_ylabel("Accuracy")
AX[0,0].legend(fontsize='small')
AX[0,0].grid(True, alpha=0.3)
AX[0,1].set_ylabel("Loss")
AX[0,1].legend(fontsize='small')
AX[0,1].grid(True, alpha=0.3)

# --- Lower Panel: Difference (Model - Base) ---

diff_train = [m - b for m, b in zip(accc, base_acc)]
diff_valid = [m - b for m, b in zip(loss, base_loss)]

AX[1,0].plot(fac,diff_train, c=colors[key], ls="--")
AX[1,1].plot(fac,diff_valid, c=colors[key], ls="--")
AX[1,1].axhline(0, color='black', lw=1) # Zero line for reference
AX[1,0].axhline(0, color='black', lw=1) # Zero line for reference

AX[1,0].set_ylabel(r"$\Delta$ (Model - FFNN)")
AX[1,1].set_xlabel('Fraction of Training Data Used')
AX[1,0].set_xlabel('Fraction of Training Data Used')
AX[1,1].grid(True, alpha=0.3)
AX[1,0].grid(True, alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.93])
#plt.savefig(f"size_images/n_samples_comparison_{key}.png")
plt.show()

```

12 Grid search for different parameters

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import pandas as pd
import os
from sklearn import metrics
from sklearn import model_selection
# AdaBoost Algorithm
from sklearn.ensemble import AdaBoostClassifier
# Gradient Boosting
from sklearn.ensemble import GradientBoostingClassifier
# XGBoost
import xgboost
from xgboost import XGBClassifier
from xgboost import plot_importance, to_graphviz, plot_tree
print("XGBoost version:", xgboost.__version__)
import keras_tuner as kt
mycmap = "winter"
mpl.rcParams['image.cmap'] = mycmap
plt.rcParams['font.size'] = 13
```

```
N=5000
# size of each sample
L=4
# box size
B=100
```

```
def f(x):
    Q=np.heaviside(np.sin(np.pi*(x[1]+2*x[0]))*(2*x[1]-x[0])/2000),0)
    return Q

def generate_data(N,L,B,f,s):
    os.makedirs("DATA",exist_ok=True)
```

```

print(np.random.randint(0,10),np.random.randint(0,10),
      ↪),np.random.randint(0,10))
def filename(s):
    return "DATA/"+s+".dat"
x,y = (np.random.random((N,L))-0.5)*B, np.zeros(N).astype(int)
for i in range(N):
    # label data according to a nonlinear function "f" using only the
    ↪ first two features
    y[i] = f(x[i,:2])

#plt.figure(figsize = (5,5))
#plt.scatter(x[:,0],x[:,1],c=y,s=4,cmap="winter")
#plt.show

fn=filename(f"x_{s}")

np.savetxt(fn,x,fmt="%.3f")
fn=filename(f"y_{s}")
np.savetxt(fn,y,fmt="%d")
return x,y

def load_data(s,fac=0.75):
    def filename(s):
        return "DATA/"+s+".dat"
    x = np.loadtxt(filename(f"x_{s}"), delimiter=" ",dtype=float)
    y = np.loadtxt(filename(f"y_{s}"))
    y = y.astype(int)
    N,L = len(x), len(x[0])

    N_train = int(fac*N)
    x_train,y_train = x[:N_train],y[:N_train]
    x_test,y_test = x[N_train:],y[N_train:]
    return x_train,y_train,x_test,y_test, N,L

x, y = generate_data(N,L,B,f,"XGB_26")
x_train,y_train,x_test,y_test, N,L = load_data("XGB_26")
print("N =",N,"L =",L)
print("x_train.shape =",x_train.shape,"y_train.shape =",y_train.shape)
print("x_test.shape =",x_test.shape,"y_test.shape =",y_test.shape)

```

```

import inspect
import keras_tuner as kt

```

```

from sklearn import metrics, model_selection

# 1. New dynamic build_model
def build_model(hp, clf, params_config):
    sampled_params = {}

    # 1. Create a dummy instance to see ALL supported parameters
    # This is much more reliable than inspect.signature for XGBoost
    try:
        temp_instance = clf()
        all_supported_params = temp_instance.get_params().keys()
    except Exception:
        # Fallback to signature if get_params fails
        all_supported_params = inspect.signature(clf).parameters.keys()

    # 2. Loop through our desired tuning config
    for p_name, p_settings in params_config.items():
        # Check if the model actually supports this parameter
        if p_name in all_supported_params:
            if p_settings['type'] == 'Int':
                sampled_params[p_name] = hp.Int(
                    p_name, p_settings['min'], p_settings['max'],
↪ step=p_settings.get('step', 1)
                )
            elif p_settings['type'] == 'Float':
                sampled_params[p_name] = hp.Float(
                    p_name, p_settings['min'], p_settings['max'],
↪ sampling=p_settings.get('sampling', 'linear')
                )
            elif p_settings['type'] == 'Categorical':
                sampled_params[p_name] = hp.Choice(
                    p_name, p_settings['values']
                )

    # This print will now show the full list!
    print(f"Building {clf.__name__} with: {list(sampled_params.keys())}")

    return clf(**sampled_params)

# 2. Define the "Master Config" for all possible parameters
tuning_config = {
    'objective' : {'type': 'Categorical', 'values': ['binary:logistic']},
    'importance_type' : {'type': 'Categorical', 'values': ['gain']},
    'n_estimators': {'type': 'Int', 'min': 50, 'max': 300, 'step': 50},

```

```

    'max_depth': {'type': 'Int', 'min': 3, 'max': 10},
    'learning_rate': {'type': 'Float', 'min': 1e-5, 'max': 1e-3, 'sampling':
        ⇨ 'log'},
    'reg_lambda': {'type': 'Float', 'min': 1e-3, 'max': 1e-1, 'sampling':
        ⇨ 'log'}

}

# 3. The Optimized Loop
n_trials = 200
tunners = {}
clasifiers = {
    "XGBClassifier": XGBClassifier,
    "GradientBoostingClassifier": GradientBoostingClassifier,
    "AdaBoostClassifier": AdaBoostClassifier,
}

for name, clf in classifiers.items():
    print(f"Starting tuning for {name}...")
    tuner = kt.tuners.SklearnTuner(
        oracle=kt.oracles.BayesianOptimizationOracle(
            objective=kt.Objective('score', 'max'),
            max_trials=n_trials
        ),
        # Pass the classifier and the full config; the function will filter
        ⇨ them
        hypermodel=lambda hp, c=clf: build_model(hp, c, tuning_config),
        scoring=metrics.make_scorer(metrics.accuracy_score),
        cv=model_selection.StratifiedKFold(5),
        directory='ParameterTuning',
        project_name=name,
        overwrite=False
    )

    tuner.search(x_train, y_train)
    tunners[name] = tuner

```

```

import matplotlib.cm as cm
import matplotlib.colors as mcolors

label_dict = {
    'optimizer': 'Optimizer',

```

```

        'regularizer': 'Regularizer',
        'dropout': 'Dropout Rate',
        'learning_rate': 'Learning Rate'
    }

def results_plot(tuner, num_trials=None, column="regularizer",
    ↪ row="optimizer",
                hue="dropout", x_var="learning_rate",
    ↪ label_dict=label_dict,
                sig_figs=3, title=None):

    if num_trials is None:
        num_trials = len(tuner.oracle.trials)
    trials = tuner.oracle.get_best_trials(num_trials=num_trials)

    if title is None:
        title = f"Parameter search for a model with
    ↪ {len(tuner.oracle.trials)} trials"

    results_list = []
    for trial in trials:
        data = trial.hyperparameters.values.copy()
        data['val_accuracy'] = round(trial.score, 3) if trial.score is not
    ↪ None else 0
        results_list.append(data)

    df = pd.DataFrame(results_list)

    # 1. Handling Missing Columns (Safety for different models)
    for col in [column, row, hue, x_var]:
        if col is not None and col not in df.columns:
            df[col] = "N/A"
    global_max = df['val_accuracy'].max()

    # 2. Rounding Logic
    def round_df_to_sig_figs(df_in, n_sigs):
        df_numeric = df_in.select_dtypes(include=[np.number])
        vals = df_numeric.values
        mask = (vals != 0) & (~np.isnan(vals))
        log_10 = np.floor(np.log10(np.abs(np.where(mask, vals, 1))))
        decimals = (n_sigs - 1) - log_10
        rounded = np.where(mask, np.round(vals * 10**decimals) /
    ↪ 10**decimals, vals)
        df_in[df_numeric.columns] = rounded
        return df_in

```

```

df = round_df_to_sig_figs(df, sig_figs)

# 3. FacetGrid Setup with Hue Logic
colormap = None
norm = None

if hue is not None and hue in df.columns and df[hue].nunique() > 1:
    if df[hue].nunique() <= 9:
        # Discrete Case
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↪ height=4, hue=hue, palette="Set2")
    else:
        # Continuous Case (Fixing the Palette/Hue swap)
        norm = mcolors.Normalize(vmin=df[hue].min(), vmax=df[hue].max())
        colormap = cm.get_cmap("cubehelix")
        unique_hues = sorted(df[hue].unique())
        palette_dict = {val: mcolors.to_hex(colormap(norm(val))) for val
↪ in unique_hues}

        # FIXED: hue is the COLUMN NAME, palette is the DICTIONARY
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↪ height=4, hue=hue, palette=palette_dict)
    else:
        g = sns.FacetGrid(df, col=column, row=row, margin_titles=True,
↪ height=4)

# 4. Annotate Function

def annotate_scores(data, **kws):
    if data.empty:
        return

    ax = plt.gca()
    # 1. Identify which variable is on the y-axis
    # In your logic, this is either x_var or the 'groups_of' binned
    ↪ column
    y_var_name = (x_var if df[x_var].unique().shape[0] <= 10 else
↪ f"groups_of_{x_var}")

    # 2. Find the row with the maximum accuracy in THIS facet
    best_row_idx = data['val_accuracy'].argmax()
    max_score = data['val_accuracy'].iloc[best_row_idx]
    best_y_label = data[y_var_name].iloc[best_row_idx]

```

```

# 3. Get the vertical position of that specific category
# Matplotlib's categorical axes use integers 0, 1, 2... for
    ↪ positions

# We find where our 'best_y_label' sits in the current y-ticks
# A more robust way is using the category index from the plot
y_pos = None
ticks = ax.get_yticks()
labels = [t.get_text() for t in ax.get_yticklabels()]

try:
    # Match the label string to find the correct tick index
    label_idx = labels.index(str(best_y_label))
    y_pos = ticks[label_idx]
except (ValueError, IndexError):
    # Fallback to middle if label matching fails
    y_pos = 0.5
    transform = ax.transAxes
else:
    # If we found the tick, we use Data coordinates for Y and Axes
    ↪ for X
    transform = ax.get_yaxis_transform()

# 4. Draw the text inside the bar or at the end of it
if max_score < global_max:
    ax.text(0.95, y_pos, f"Score: {max_score:.3f}",
            transform=transform,
            fontsize=9, fontweight='bold',
            ha='right', va='center',
            color=kws.get('color', 'black'),
            bbox=dict(facecolor='white', alpha=0.8,
    ↪ edgecolor='none'))
else:
    ax.text(0.95, y_pos, f"Score: {max_score:.3f}",
            transform=transform,
            fontsize=9, fontweight='bold',
            ha='right', va='center',
            color=kws.get('color', 'black'),
            bbox=dict(facecolor='white', alpha=0.8, edgecolor='black',
    ↪ boxstyle='round,pad=0.2'))

# 5. Plotting

```

```

y_axis_var = x_var if x_var in df.columns else df.columns[0]

if df[x_var].unique().shape[0] <= 10: # Only annotate if there are 10
    ↪ or fewer unique x values

    g.map_dataframe(
        sns.barplot,
        x="val_accuracy",
        y=y_axis_var,
        errorbar=None,
        orient="h"

    )
else:
    df[f'groups_of_{x_var}'] = pd.cut(df[x_var], bins=10,
    ↪ duplicates='drop')
    g.map_dataframe(
        sns.barplot,
        x="val_accuracy", # Numbers go on X
        y=f'groups_of_{x_var}', # Categories go on Y
        errorbar=None,
        orient="h" # Use "h" for clarity, or leave it to
    ↪ auto-detect
    )
    g.map_dataframe(annotate_scores)

# 6. Final Layout and Margin Title Fixes
# Push right margin to make room for titles
g.fig.subplots_adjust(right=0.8, top=0.85, wspace=0.4)
g.fig.suptitle(title, fontsize=14, fontweight='bold')
g.set_titles(col_template=f"{column}="+f"{col_name}",
↪ row_template=f"{row}="+f"{row_name}", fontweight='bold')

# FIX: Reposition Row Titles to the far right
for txt in g.fig.texts:
    if txt.get_rotation() == -90:
        txt.set_x(0.92) # Absolute figure coordinate
        txt.set_va('center')

# Add Colorbar for Continuous Hue
if colormap and norm:
    sm = plt.cm.ScalarMappable(cmap=colormap, norm=norm)
    sm.set_array([])

```

```

        cbar_ax = g.fig.add_axes([0.85, 0.15, 0.02, 0.6]) # [left, bottom,
↪ width, height]
        g.fig.colorbar(sm, cax=cbar_ax, label=label_dict.get(hue, hue))
    else:

        g.add_legend()
    g.set(xlim=(0, 1.1))
    plt.show()

```

```

results_plot(tunners['XGBClassifier'],column="n_estimators",
↪ row="max_depth", hue="reg_lambda",
↪ x_var="learning_rate",title=f"Parameter search for XGBClassifier with
↪ {len(tunners['XGBClassifier'].oracle.trials)} trials")
results_plot(tunners['AdaBoostClassifier'],column="n_estimators", row=None,
↪ hue=None, x_var="learning_rate",title=f"Parameter search for
↪ AdaBoostClassifier with
↪ {len(tunners['AdaBoostClassifier'].oracle.trials)} trials")
results_plot(tunners['GradientBoostingClassifier'],column="n_estimators",
↪ row=None, hue="max_depth", x_var="learning_rate",title=f"Parameter
↪ search for GradientBoostingClassifier with
↪ {len(tunners['GradientBoostingClassifier'].oracle.trials)} trials")

```

12.1 Select different features

```

def scat(ax, x, y, i=0, j=1, s=4, title="", xlabel=None, ylabel=None):
    """
    Scatter plot of data in the plane of features i and j.
    """
    # Use c=y to color points by their class label
    ax.scatter(x[:, i], x[:, j], s=s, c=y, cmap='viridis', alpha=0.7)
    ax.set_xlabel(xlabel if xlabel else f"Feature {i}")
    ax.set_ylabel(ylabel if ylabel else f"Feature {j}")
    ax.set_title(title)

def classify(clf, x_train, y_train, x_test, y_test, Lp, grid_range=50, dx=1,
↪ show=False, tree=False,name = "", params=""):
    """
    Trains a classifier, predicts on a grid to show decision boundaries,
    and compares it to the training set.
    """
    # 1. Train and Evaluate
    clf.fit(x_train, y_train)

```

```

accuracy = clf.score(x_test, y_test)
error_rate = 100 * (1 - accuracy)

print(f"Errors: {error_rate:.2f}% | Accuracy: {accuracy:.3f}")

# 2. Create Artificial Grid for Decision Boundary Visualization
# We create a 2D grid for the first two features
s_points = np.arange(-grid_range, grid_range + dx, dx)
xx, yy = np.meshgrid(s_points, s_points)

# Flatten grid and prepare x_plot with Lp features (filling rest with
    ↪ zeros)
grid_coords = np.c_[xx.ravel(), yy.ravel()]
x_plot = np.zeros((grid_coords.shape[0], Lp))
x_plot[:, :2] = grid_coords

print(f'Shape of x_plot: {x_plot.shape}')
if show:
    # 3. Predict on the grid
    y_plot = clf.predict(x_plot)

    # 4. Visualization
    fig, AX = plt.subplots(1, 2, figsize=(10, 4))

    # Left: Decision Boundary (Predicted)
    scat(AX[0], x_plot, y_plot, i=0, j=1, s=dx*2, title="Decision
    ↪ Boundary (Predicted)")

    # Right: Actual Training Data
    scat(AX[1], x_train, y_train, i=0, j=1, s=10, title="Actual Training
    ↪ Set")

    fig.tight_layout()
    fig.suptitle(f"Results for model {name} with params \n {params}",
    ↪ y=1.2)
    plt.show()

if tree:
    dump_list = clf.get_booster().get_dump() # get the list of trees in
    ↪ the model
    num_trees = len(dump_list)
    print("num_trees=", num_trees)

    fig, AX = plt.subplots(2, 1, figsize=(12, 5))

```

```

for i in range(min(2,num_trees)):
    ax=AX[i]
    plot_tree(clf, num_trees=i, ax=ax)
fig.savefig("DATA/tree-classif.png", dpi=400, pad_inches=0.02)
plt.show()

```

```

def plot_feature_importance(model, feature_names=None, title=None,
↪ ascending=False):
    """
    Plots feature importances for any scikit-learn or XGBoost model.
    - ascending: If True, smaller importances get darker colors.
    """
    importances = model.feature_importances_
    num_features = len(importances)

    # Use provided feature names or default to indices
    if feature_names is None:
        feature_names = [f"F{i}" for i in range(num_features)]

    # 1. Setup Colors using your Rescale logic
    my_cmap = plt.get_cmap("Reds")
    min_imp, max_imp = np.min(importances), np.max(importances)

    # Avoid division by zero if all importances are equal
    diff = (max_imp - min_imp) if max_imp != min_imp else 1

    if not ascending:
        # Standard: Higher importance = Darker Red
        rescale = lambda y: 0.3 + 0.7 * (y - min_imp) / diff
    else:
        # Reversed: Lower importance = Darker Red
        rescale = lambda y: 0.3 + 0.7 * (max_imp - y) / diff

    # 2. Create Plot
    plt.figure(figsize=(8, 4))
    bars = plt.bar(range(num_features), importances,
                    color=my_cmap(rescale(importances)),
                    edgecolor='black', linewidth=0.5)

    # 3. Styling
    plt.xlabel("Features")
    plt.ylabel("Importance Score")

```

```

# Determine the type of importance used (XGBoost specific)
imp_type = getattr(model, 'importance_type', 'Gini/MDI')
plt.title(title if title else f"Feature Importance ({imp_type})")

plt.xticks(range(num_features), feature_names, rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

def plot_xgb_multi_importance(model, feature_names=None):
    """
    Compares different XGBoost importance types.
    Handles cases where booster.feature_names is None.
    """
    list_type = ["weight", "gain", "total_gain", "cover", "total_cover"]
    list_col = ["#7565F0", "k", "#666666", "#F0A000", "gold"]

    booster = model.get_booster()

    # FIX: If booster.feature_names is None, generate generic names
    # based on the model's expected input shape
    if booster.feature_names is not None:
        all_features = booster.feature_names
    else:
        # Fallback: get count from the actual feature_importances_ array
        num_features = len(model.feature_importances_)
        all_features = [f"f{i}" for i in range(num_features)]

    L = len(all_features)

    # Use the names passed in the function call for the X-axis labels
    if feature_names is None:
        display_names = all_features
    else:
        display_names = feature_names

    plt.figure(figsize=(12, 6))

    for i, t in enumerate(list_type):
        score_dict = booster.get_score(importance_type=t)

```

```

# XGBoost get_score keys are strings like 'f0', 'f1' regardless of
    ↪ names
# so we map the scores carefully
values = np.array([score_dict.get(f, 0) for f in all_features])

total = np.sum(values)
if total > 0:
    values = values / total

width = 0.12
offset = (i - len(list_type)/2) * width + (width/2)

plt.bar(np.arange(L) + offset, values,
        color=list_col[i], width=width, label=t, alpha=0.8)

plt.xlabel("Features", fontweight='bold')
plt.ylabel("Importance (Normalized to 1)", fontweight='bold')
plt.title("XGBoost: Comparison of Importance Metrics", fontsize=14)

# Use our display_names for the ticks
plt.xticks(np.arange(L), display_names, rotation=45)
plt.legend(title="Importance Type", bbox_to_anchor=(1.05, 1), loc='upper
    ↪ left')
plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```

```

from itertools import combinations
Lp = 3
features = list(combinations(range(L), Lp))
for (n, tuner), (_, clf) in zip(tuners.items(), classifiers.items()):
    best_params = tuner.oracle.get_best_trials(num_trials=n_trials)[0]
    ↪ ].hyperparameters.values.copy()
    for f in features:
        x_train_subset = x_train[:, f]
        x_test_subset = x_test[:, f]
        print(f"Prediction for model {n} trained with the features {f}")
        model = clf(**best_params)
        classify(
            clf=model,
            x_train=x_train_subset,
            y_train=y_train,

```

```

        x_test=x_test_subset,
        y_test=y_test,
        Lp=Lp,
        grid_range=50,
        name=n+f" with features {f}",
        params = best_params,
        show=True,
    )
    plot_feature_importance(model, feature_names=[f"F{idx}" for idx in
↪ f], title=f"Feature Importance for {n} with features {f}")
    if n == "XGBClassifier":
        plot_xgb_multi_importance(model, feature_names=[f"F{idx}" for
↪ idx in f])

```

12.2 compare it to FFNN with cross validation

```

from keras.models import Sequential
from keras.layers import Dense, Dropout

def FFNN(x_train,x_test,y_train,y_test,epochs=200):
    model = Sequential()
    model.add(Dense(L,input_shape=(L,),activation='relu'))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1)) #Helps with overfitting, it prevents
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(20,activation='relu'))
    model.add(Dropout(0.1))
    model.add(Dense(1,activation='sigmoid'))

    model.compile(loss='binary_crossentropy',
                  optimizer='RMSprop',
                  #optimizer='adam',
                  metrics=['accuracy'])
    model.fit(x_train, y_train,
              epochs = epochs, batch_size = 50,
              validation_data=(x_test,y_test),
              verbose=2)
    accuracy, loss = model.evaluate(x_test, y_test, verbose=0)

    return accuracy, loss

```

```

from sklearn.model_selection import KFold
k=5
fac = np.linspace(0.1, 0.95, 20)

results = {}
for (n, tuner), (_, clf) in zip(tuners.items(), classifiers.items()):

    accuracies_fnn = []
    losses_fnn = []
    for f in fac:
        print(f"Training {n} with training size factor: {f:.2f}")
        total_acc = 0
        total_loss = 0
        x_train, y_train, x_test, y_test, N, L = load_data("XGB_26", fac=f)
        for train_index, test_index in KFold(n_splits=k, shuffle=True,
            ↪ random_state=1234).split(x_train):

            x_tr_fold, x_val_fold = x_train[train_index],
            ↪ x_train[test_index]
            y_tr_fold, y_val_fold = y_train[train_index],
            ↪ y_train[test_index]
            best_params = tuner.oracle.get_best_trials(num_trials=
            ↪ n_trials)[0].hyperparameters.values.copy()
            model = clf(**best_params)
            model.fit(x_tr_fold, y_tr_fold)
            accuracy = model.score(x_val_fold, y_val_fold)
            error_rate = (1 - accuracy)
            total_acc += accuracy
            total_loss += error_rate

        avg_acc = total_acc / k
        avg_loss = total_loss / k
        accuracies_fnn.append(avg_acc)
        losses_fnn.append(avg_loss)
    results[n] = (accuracies_fnn, losses_fnn)

accuracies_fnn = []
losses_fnn = []
for f in fac:
    print(f"Training FFNN with training size factor: {f:.2f}")
    x_train, y_train, x_test, y_test, N, L = load_data("XGB_26", fac=f)

```

```

total_acc = 0
total_loss = 0

for train_index, test_index in KFold(n_splits=k, shuffle=True,
    ↪ random_state=1234).split(x_train):
    x_tr_fold, x_val_fold = x_train[train_index], x_train[test_index]
    y_tr_fold, y_val_fold = y_train[train_index], y_train[test_index]
    acc, loss = FFNN(x_tr_fold, x_val_fold, y_tr_fold, y_val_fold,
    ↪ epochs=10)
    print(f"Accuracy: {acc:.4f}, Loss: {loss:.4f}\n")
    total_acc += acc
    total_loss += loss
avg_acc = total_acc / k
avg_loss = total_loss / k
accuracies_fnn.append(avg_acc)
losses_fnn.append(avg_loss)
results["FNN"] = (accuracies_fnn, losses_fnn)

```

```

results["XGBClassifier"]

```

```

import matplotlib.pyplot as plt

fig, AX = plt.subplots(2, 2, figsize=(20, 8), sharex='col',
    gridspec_kw={'height_ratios': [3, 1], 'hspace':
    ↪ 0.1})
colors = {
    'XGBClassifier': 'purple',
    'GradientBoostingClassifier': 'green',
    'AdaBoostClassifier': 'r'
}

base_acc = results["FNN"][0]
base_loss = results["FNN"][1]

# Plot Original (Base) Model
AX[0,0].plot(fac, base_acc, label="FFNN", c='b', alpha=0.5, ls="--")
AX[0,1].plot(fac, base_loss, label="FFNN", c='b', alpha=0.5, ls="--")
for key, (acc, loss) in results.items():
    # Create a 2x2 grid with shared x-axes for the top/bottom pairs
    if key == "FNN":
        continue

```

```

print(f"Plotting results for {key}...")
print(f"Accuracies: {accc}")
print(f"Losses: {loss}")

fig.suptitle("Accuracy and Loss Comparison as a function of the train
↪ size ", fontsize=14, y=0.95)

# Metrics to loop through (Accuracy and Loss)

best_params = tuner.oracle.get_best_trials(num_trials=n_trials)[0]
↪ ].hyperparameters.values.copy()

ax_main = AX[0]
# Plot Current Model
AX[0,0].plot(fac,accc, label=key, c=colors[key], ls="--")
AX[0,1].plot(fac,loss, label=key, c=colors[key], ls="--")

AX[0,0].set_ylabel("Accuracy")
AX[0,0].legend(fontsize='small')
AX[0,0].grid(True, alpha=0.3)
AX[0,1].set_ylabel("Loss")
AX[0,1].legend(fontsize='small')
AX[0,1].grid(True, alpha=0.3)

# --- Lower Panel: Difference (Model - Base) ---

diff_train = [m - b for m, b in zip(accc, base_acc)]
diff_valid = [m - b for m, b in zip(loss, base_loss)]

AX[1,0].plot(fac,diff_train, c=colors[key], ls="--")
AX[1,1].plot(fac,diff_valid, c=colors[key], ls="--")
AX[1,1].axhline(0, color='black', lw=1) # Zero line for reference
AX[1,0].axhline(0, color='black', lw=1) # Zero line for reference

AX[1,0].set_ylabel(r"$\Delta$ (Model - FFNN)")
AX[1,1].set_xlabel('Fraction of Training Data Used')
AX[1,1].grid(True, alpha=0.3)
AX[1,0].grid(True, alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.93])
#plt.savefig(f"size_images/n_samples_comparison_{key}.png")
plt.show()

```

```

%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.ticker import NullFormatter
import seaborn as sns
import pandas as pd
from scipy.stats import ortho_group
plt.rcParams['font.size'] = 12

# manifold.TSNE
from sklearn import manifold #, datasets
from time import time

from sklearn.neighbors import NearestNeighbors

from scipy.cluster.hierarchy import dendrogram
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score, fowlkes_mallows_score
from sklearn.cluster import DBSCAN

```

```

N=1000
L=3
LL=12
R0=10
R1=20
p0=0.3
p1=0.3
NOISE=1

```

```

from numpy import random, cos, sin, pi
def generate_data(N,L,LL,R0,R1,p0,p1,NOISE):
    x,y,r,noise=np.zeros((N,L)),np.zeros(N),np.random.randint(0,100
    ↪ ,N),np.random.randn(N,L)*10

```

```

OM = ortho_group.rvs(dim=L) #random orthogonal matrix for rotation
OMM = ortho_group.rvs(dim=LL) #random orthogonal matrix for rotation of
↪ the 12d data
xx,nnoise=np.zeros((N,LL)),np.random.randn(N,LL)*10
X,XX=np.zeros_like(x),np.zeros_like(xx)

for n in range(N):
    if r[n]<p0:
        y[n]=0
        q = random.rand()*2*pi
        x[n,:3] = (R0*cos(q), R0*sin(q), -(R0/10)*cos(6*q))
    elif r[n]<p1+p0:
        y[n]=1
        q = random.rand()*2*pi
        x[n,:3] = (-(R1/5)*sin(4*q), R1*cos(q), R1*sin(q))
        x[n,:3] = x[n,:3] + R0*np.array((0,random.randint(3) - 1, 0))
    else:
        y[n]=2
        q = random.rand()*1000.
        x[n,:3] = ((random.randint(2)-0.5)*2*q, 1.333*q-R1,
↪ 1.333*R1-0.333*q)
        x[n] = x[n]+noise[n]
        xx[n,:3] = x[n] # copy the 3d data to the first 3 dimensions of the
↪ 12d data
        nnoise[n,3:] = nnoise[n,3:] * NOISE # add noise to the 12d data
        xx[n] = xx[n] + nnoise[n] # add noise to the 12d data
        X[n] = np.dot(OM,x[n]) # rotate the 3d data
        XX[n] = np.dot(OMM,xx[n]) # rotate the 12d data
y = y.astype(int)
np.savetxt(f"DATA/x_{LL}d.dat",XX,delimiter='\t')
np.savetxt(f"DATA/y_{LL}d.dat",y,fmt="%d")
return XX,y

```

```

X,y=generate_data(N=1000,L=3,LL=12,R0=10,R1=20,p0=0.3,p1=0.3,NOISE=1)

```

```

# Source - https://stackoverflow.com/a/14019260
# Posted by Dmitry Dubovitsky, modified by community. See post 'Timeline'
↪ for change history
# Retrieved 2026-03-29, License - CC BY-SA 3.0

import random
r = lambda: random.randint(0,255)
print()

```

```

def get_random_color(n):
    r = lambda: random.randint(0,255)
    return ['%02X'%02X'%02X' % (r(),r(),r()) for i in range(n)]

def nn_dist(X,nn, plot=True):

    # compute the distance to the nn nearest neighbors
    nbrs = NearestNeighbors(n_neighbors=nn+1).fit(X) #,
    ↪ algorithm='ball_tree'
    # distances and the indices of the nn nearest neighbors (10+1)
    d, ind = nbrs.kneighbors(X) # returns distances and indices of nn
    ↪ neighbors
    print(d[3,:],range(ind.max()))
    dd=d.T # distances to the i-th neighbor for all points, sorted by
    ↪ distance
    print(dd.shape)
    # note: dd[0] has a trivial set of zeros
    if plot:
        listc= get_random_color(nn)
        for i in range(1,nn+1):
            plt.plot(np.sort(dd[i]),c=listc[i-1],label=str(i))
            plt.xlabel("Rank")
            plt.ylabel("Distance to i-th neighbor")
        plt.legend(ncol=2)
        plt.show()
    mean_min_dist = np.mean(dd[1]) # mean distance to the nearest neighbor
    print("mean minimum distance between nn points: ",mean_min_dist)
    return mean_min_dist
mean_min_dist = nn_dist(X,nn=10, plot=True)

```

```

from collections import OrderedDict

def round_df_to_sig_figs(df_in, n_sigs):
    df_numeric = df_in.select_dtypes(include=[np.number])
    vals = df_numeric.values
    mask = (vals != 0) & (~np.isnan(vals))
    log_10 = np.floor(np.log10(np.abs(np.where(mask, vals, 1))))
    decimals = (n_sigs - 1) - log_10
    rounded = np.where(mask, np.round(vals * 10**decimals) /
    ↪ 10**decimals, vals)
    df_in[df_numeric.columns] = rounded
    return df_in

```

```

def heat_map(mean_min_dist,eps_range,min_sample_range,X,y_true):
    eps_range = mean_min_dist*eps_range
    def clustering(y):
        # Finds position of labels and returns a dictionary of cluster
        ↪ labels to data indices.
        yu = np.sort(np.unique(y))
        clustering = OrderedDict()
        for ye in yu:
            clustering[ye] = np.where(y == ye)[0]
        return clustering

    def entropy(c, n_sample):
        # Measures the shannon entropy of a cluster
        # c is a dictionary of cluster labels to data indices
        # n_sample is the total number of data points
        h = 0.
        for kc in c.keys():
            p=len(c[kc])/n_sample
            h+=p*np.log(p)
        h*=-1.
        return h

    # Normalized mutual information function
    # Note that this deals with the label permutation problem
    def NMI(y_true, y_pred):
        """ Computes normalized mutual information: where y_true and y_pred
        ↪ are both clustering assignments
        """
        w = clustering(y_true) # clustering of the true labels
        c = clustering(y_pred) # clustering of the predicted labels
        n_sample = len(y_true)

        Iwc = 0. # mutual information between w and c
        for kw in w.keys():
            for kc in c.keys():
                w_intersect_c=len(set(w[kw]).intersection(set(c[kc]))) #
                ↪ number of data points in the intersection of cluster kw and kc
                if w_intersect_c > 0:
                    Iwc +=
                ↪ w_intersect_c*np.log(n_sample*w_intersect_c/(len(w[kw])*len(c[kc]))) #
                ↪ mutual information contribution of the intersection of cluster kw and kc
            Iwc/=n_sample
        Hc = entropy(c,n_sample) # entropy of the predicted clustering

```

```

        Hw = entropy(w,n_sample) # entropy of the true clustering

    return 2*Iwc/(Hc+Hw)

results_matrix=np.zeros((len(eps_range),len(min_sample_range)))
for i, eps in enumerate(eps_range):
    for j, min_samples in enumerate(min_sample_range):
        model = DBSCAN(eps=eps, min_samples=min_samples)
        model.fit(X)
        y_hat = model.labels_ # predicted labels
        nmi=NMI(y_hat, y_true)
        results_matrix[i, j] = nmi
df_eps_samp = pd.DataFrame({"eps":eps_range})
df_eps_samp = round_df_to_sig_figs(df_eps_samp,3)
eps_range = df_eps_samp['eps']

df = pd.DataFrame(
    results_matrix,
    index=eps_range,
    columns=min_sample_range
)

fig, ax = plt.subplots(1,1, figsize=(16,16))
if len(eps_range) <= 10 and len(min_sample_range) <= 10:
    df = round_df_to_sig_figs(df,2)
    sns.heatmap(data=df, annot=True, fmt=".3f", ax=ax)
else:
    sns.heatmap(data=df,ax=ax)
plt.show()
ind = np.unravel_index(np.argmax(results_matrix, axis=None),
↪ results_matrix.shape)
return ind

eps_range=np.linspace(0.5, 2, 9)
min_sample_range=np.linspace(1, 50,9).astype(int)
ind = heat_map(mean_min_dist,eps_range,min_sample_range,X,y)

```

```

from sklearn.decomposition import PCA

```

```

def PCA_plot(LL,X):
    pca = PCA(n_components=LL)
    pca.fit(X)

```

```
y=pca.explained_variance_ratio_  
df = pd.DataFrame({'explained_var': y})  
df_hue = round_df_to_sig_figs(df,2)  
sns.barplot(x=range(LL),y=y,errorbar=None, hue=df_hue["explained_var"],  
↪ palette='Set2')  
plt.show()
```

```
PCA_plot(LL,X)
```

14 Summary

In summary, this book has no content whatsoever.

References

Knuth, Donald E. 1984. “Literate Programming.” *Comput. J.* (USA) 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.